

CINEVERSE-LARAVEL

Technical Documentation

Technologies

Vue 3 · Laravel 12 · Inertia 2 · TypeScript · PHP · MySQL 8 · TMDB API

Project Technical Documentation · Laravel 12 + Vue 3 + Inertia.js 2

Author: Gastón Rizzo

Table of Contents

1. CineVerse Overview	3
2. Technologies, Environment, and Core Concepts	4
3. Inertia: How Laravel and Vue Connect in CineVerse	8
4. TMDB: What It Is, How Access Is Obtained, and How CineVerse Uses It	14
5. General Architecture of CineVerse Laravel	22
6. Project Structure	23
7. Configuration Files and Environment Variables	25
8. MySQL Database, Migrations, and Eloquent	28
9. Test Data and Seeders	30
10. How a User Action Flows Through CineVerse	31
11. TypeScript in CineVerse	32
12. How the Vue Frontend Is Organized	35
13. How Data Is Stored and Reused in the Frontend	38
14. Navigation Without Page Reloads: SPA and Vue Router	41
15. Languages: vue-i18n and Language Switching	42
16. Communication Between Vue, TMDB, and Laravel	47
17. Movie Explorer and Search System	50
18. How Filters Are Stored and Reflected in the URL	56
19. How More Movies Are Loaded	59
20. Home Page	61
21. Movie Details	63
22. Trailers, Similar Movies, and Images	65
23. Actors and Filmography	67
24. Favorites	68
25. How the Laravel Backend Is Organized	70
26. Authentication and Account: Breeze, Sessions, and Recovery	75
27. Reviews and Profile	81
28. Validation, Security, CSRF, and RateLimiter	87
29. Interface States and Error Handling	89
30. Logs: Error Recording and Diagnostics	93
31. Local Installation of CineVerse Laravel	97
32. Running CineVerse Laravel in Development	101
33. Deploying CineVerse Laravel to a Server	103
34. Tests and Technical Checks	104

1. CineVerse Overview

CineVerse is a web application designed to provide information about movies and actors. From the browser, users can browse trends and new releases, search for titles, combine filters, open a movie's detail page, explore cast and filmographies, play trailers, save favorites, and switch the interface between Spanish and English. After creating an account and verifying their email address, visitors can also publish reviews, edit or delete them, and view their activity from their profile.

The version documented here is the Laravel integration of the project. The frontend is still built with Vue 3 and TypeScript, but it no longer lives as an application separate from a plain PHP backend. Vue is included in the same Laravel repository under `resources/js/cineverse`, and browser entry is handled through Laravel, Inertia, and Vite.

Two Types of Information

Movie-related information is not copied into MySQL. Movies, people, posters, credits, and videos are queried from TMDB when the interface needs them. MySQL stores CineVerse's own data: users, reviews, sessions, password-reset tokens, and Laravel infrastructure tables.

1.1 What Changes Compared with the Plain PHP Version

The important change is not CineVerse's appearance, but the server architecture. The former flow `public/index.php` → custom Router → Middleware → Controller → Service → Model → PDO is no longer the foundation of the backend. Laravel provides its own router, middleware, controllers, Form Requests, authentication, Eloquent, migrations, sessions, and exception handling. This replaces several manual solutions with standard framework mechanisms without sacrificing the original frontend.

The way the project is installed also changes. Instead of importing an SQL file that creates the entire environment, the schema is rebuilt through migrations. Test data is loaded with seeders from NDJSON files. The repository does not need to include `vendor`, `node_modules`, `.env`, or `public/build` because those elements are rebuilt locally.

1.2 What Was Deliberately Preserved

Vue Router remains the interface's internal router. This is an important design decision: Inertia did not replace CineVerse's SPA navigation. Laravel delivers a single Inertia page named CineVerse, and that page mounts CineVerseApp; from that point on, Vue Router decides whether `HomeView`, `MoviesView`, `MovieDetailView`, `LoginView`, `ProfileView`, or another view should be displayed.

Vue also retains direct access to TMDB. Laravel does not act as a proxy for the movie catalog. This separation lets movie and person queries continue to use TanStack Vue Query and frontend services, while Laravel focuses on identity, security, and application-owned data.

1.3 How to Read This Document

Each mechanism is introduced through a concrete situation before technical names are discussed. When Carlos searches for a movie, opens a tab, signs in, or publishes a review, the document follows the action from the View or Component that receives the interaction through the Service, the HTTP route, Laravel or TMDB, the database when applicable, and finally the data's return to the interface.

When actual source code is shown, the block is preceded by “Location in the project” and identifies the file, the type of component, and, when applicable, the responsible class, method, or function. Arrow diagrams are labeled as explanatory flows: they summarize how the components interact, but they are not presented as literal lines from a file. This distinction makes it possible to study the project without confusing real code with a teaching diagram.

Reading rule: first explain what the user does, then where it happens, what data comes in, what code is involved, what response comes back, and what the user ultimately sees.

Laravel is the PHP framework that organizes the server in this version. It does not replace the Vue frontend or the TMDB catalog: it mainly replaces the backend infrastructure that CineVerse previously implemented with its own Router, sessions, validators, PDO Models, Response class, and logging.

1.4 Concepts Repeated Throughout the Document

Before following specific files, it is useful to establish what each type of component means within CineVerse Laravel. These names are not theoretical labels: they make it possible to follow a complete flow and understand the responsibility of each file.

Concept	What It Means in CineVerse Laravel	Real Example
View	Full screen normally associated with an internal Vue Router route.	MoviesView.vue, LoginView.vue, ProfileView.vue
Component	Reusable visual part of a screen. It can receive data from its parent component (props) and emit events to notify it that an action occurred.	MovieCard.vue, MovieHero.vue, MovieReviews.vue
Composable	Vue function that groups reusable reactive logic and can coordinate queries, filters, state, and services without becoming a screen itself.	useMoviesQuery.ts, useMovieDetail.ts, usePersonDetail.ts
Frontend Service	TypeScript module that knows how to communicate with an API or with Laravel. It does not render the interface.	movies.service.ts, person.service.ts, auth.service.ts, reviews.service.ts
Store	State shared across different parts of the frontend.	useAuthStore, useFavoritesStore
API	HTTP interface that allows data to be requested from or sent to another system.	TMDB API and Laravel JSON endpoints
Endpoint	Specific address within an API that represents an operation.	/search/movie, POST /login, POST /api/reviews
JSON	Structured text format used to exchange data.	Paginated TMDB response or JSON response from ReviewController
Query parameter	Data that appears after ? in a URL.	page=1&language=es

Reading rule. A View should not be confused with a Service: the View coordinates what the user sees; the Service knows the HTTP destination. A Composable connects both worlds without becoming a screen.

2. Technologies, Environment, and Core Concepts

CineVerse combines technologies that run on the server with technologies that run in the browser. Each has a specific role: Laravel organizes the server layer; Inertia connects that layer to Vue; Vue Router preserves internal navigation; MySQL stores application-owned data; TMDB provides the catalog; Vite prepares and serves the frontend files—known as assets; Tailwind CSS provides the global foundation imported by app.css; and several frontend libraries handle carousels, icons, filter controls, drag-and-drop, and state persistence.

Technology	Project Version/Constraint	Primary Responsibility
PHP	composer.json: ^8.2	Runs Laravel and Composer.
Laravel	framework ^12.0; lock 12.62.0	Routes, sessions, validation, controllers, Eloquent, migrations, and exceptions.

Technology	Project Version/Constraint	Primary Responsibility
Inertia Laravel	^2.0	Delivers Pages/CineVerse.vue from Laravel.
Vue 3	^3.4.0	Components, reactivity, and interface views.
Vue Router	^5.1.0	Internal routes such as /es, /movies, /profile, etc.
TypeScript	^5.6.3	Static frontend types and builds with vue-tsc.
Vite	^7.0.7	Frontend asset server during development and production bundle build.
Pinia	^3.0.4	Global authentication and favorites state.
TanStack Vue Query	^5.101.2	Queries, caching, pagination, and remote-data states.
vue-i18n	^11.4.6	ES/EN interface.
Axios	^1.11.0	HTTP communication with TMDB and Laravel.
Laravel Breeze	^2.4, require-dev	Initial authentication code structure (scaffolding), adapted by CineVerse.
Sanctum	^4.0	Installed, although the main routes use web sessions.
Ziggy	^2.0	Makes Laravel routes available in Vue.
MySQL	configurable through .env	Users, reviews, sessions, cache, jobs, and tokens.
TMDB API v3	VITE_TMDB_BASE_URL	Movies, people, credits, videos, and images.
Tailwind CSS	4.2.1; @tailwindcss/vite 4.2.1	Global base/reset imported by resources/css/app.css; the CineVerse interface primarily uses custom CSS. tailwind.config.js also registers @tailwindcss/forms.
pinia-plugin-persistedstate	^4.7.1	Persists the favorites store and other configured state in localStorage.
Swiper	^14.0.5	Home page carousels.
lucide-vue-next	^1.0.0	Reusable icons in Vue components.
vue-multiselect	^3.5.0	Multi-select controls in the movie explorer.
vue-slider-component	^4.1.0-beta.7	Year and ratings sliders in the filters.
vuedraggable	^4.1.0	Drag-and-drop reordering of customizable explorer panels.

2.1 What Is a Framework

A framework is a working foundation that already provides a structure and tools for solving recurring application tasks. In a PHP project without a framework, a team may manually implement routing, sessions, validation, database access, error handling, and many other pieces. Laravel provides standard mechanisms for those tasks, and CineVerse uses them as the backbone of its server layer.

Key idea: Laravel does not replace Vue or TMDB. In CineVerse, it primarily replaces the backend infrastructure that was previously implemented with custom PHP classes.

2.2 What Laravel Is and What It Does in CineVerse

Laravel is a web framework for PHP. It receives HTTP requests, matches the URL and method against registered routes, runs middleware, hands the operation to the appropriate Controller, validates data, manages

authentication and sessions, uses Eloquent to work with MySQL, and finally builds an HTTP response. The version installed in this project is Laravel 12.62.0, resolved by composer.lock from laravel/framework ^12.0.

```
Browser / Vue
-> Laravel receives the request
-> Router finds a route
-> middleware
-> Request / Form Request
-> Controller
-> Auth / Eloquent / Notification / other framework services
-> MySQL or direct response
-> HTTP Response
-> Vue
```

The same application may traverse only part of that flow. For example, loading a movie from TMDB does not pass through Eloquent or MySQL; creating a review, by contrast, reaches Laravel, validates the session and data, and ultimately writes to the reviews table.

2.3 What Changed Compared with the Plain PHP Backend

The previous version of CineVerse had its own Router, Request and Response classes, manual session and CSRF handling, business Services, PDO Models, and a custom AppLogger. The Laravel version preserves the same functional needs, but many of those responsibilities move to framework mechanisms.

Previous PHP Backend	CineVerse Laravel
Custom Router	Laravel Router
Session.php	web guard session
AuthService / AuthController	Auth + Controllers + LoginRequest
Validator / AuthValidator	validation + Form Requests
PDO Models	Eloquent Models / Query Builder
Response::json()	response()->json()
AppLogger + manual Monolog	Log facade + Laravel channel + Monolog
Schema SQL	migrations
test-data script	seeders

The comparison does not mean that Laravel removes the project's own logic. CineVerse-specific rules still exist: one review per user and movie, mandatory email verification before login, a local favorites limit, email language, and the response format used by the frontend. What changes is the infrastructure used to enforce those rules.

2.4 Artisan: Laravel's Command-Line Interface

Artisan is the command-line interface included with Laravel. The artisan file is located at the project root. When `php artisan <command>` is executed, PHP starts Laravel in console mode and the framework performs the requested task.

<code>php artisan serve</code>	-> development server
<code>php artisan key:generate</code>	-> generates APP_KEY
<code>php artisan migrate</code>	-> runs pending migrations
<code>php artisan db:seed</code>	-> runs DatabaseSeeder
<code>php artisan migrate:fresh --seed</code>	-> deliberately rebuilds the schema and loads test data
<code>php artisan test</code>	-> runs Laravel's PHPUnit test suite

Artisan is not a separate server or an additional library. It is an entry point to the same Laravel project for console tasks. That is why commands such as migrate know about config/database.php, .env, and the classes under database/migrations.

2.5 PHP, Composer, Autoloading, and vendor

composer.json requires PHP ^8.2. This means the project needs PHP 8.2 or a later compatible version under Composer's semantic versioning rules. The environment screenshot used for this documentation shows PHP 8.2.12. Composer reads composer.json/composer.lock, downloads libraries into vendor, and runs Laravel package-discovery scripts.

vendor is not stored in Git. This has an important consequence when installing from a clean repository: having composer.json does not mean Laravel can already run; vendor must first be rebuilt with composer install.

2.6 Node.js, npm, and Vite

package.json defines npm run dev as vite and npm run build as vue-tsc && vite build. The project does not declare an engines field with a specific Node.js version, so this document does not invent a minimum version. npm install rebuilds node_modules from package-lock.json and makes the local Vite executable required by npm run dev available.

Vite receives two entry points from vite.config.js: resources/css/app.css and resources/js/app.ts. It also knows the aliases @/features, @/core, @/layouts, @/shared, and others, which point to the integrated frontend structure under resources/js/cineverse.

2.7 Inertia: The Bridge Between Laravel and Vue

Inertia is the layer that connects Laravel's web routes to the Vue application. In CineVerse it does not replace Vue Router: Laravel and Inertia deliver the application, and once it is mounted, Vue Router decides which internal screen should be displayed.

The complete connection is explored in depth in section 3, which follows the path from routes/web.php to MovieDetailView.vue using the project's actual source code.

2.8 Blade: Where It Is Used in This Version

Blade is Laravel's templating system. CineVerse's main interface is not built with Blade; it is built with Vue. Blade appears in specific server-side functions.

- resources/views/app.blade.php acts as Inertia's root HTML document and loads the Vite assets.
- resources/views/emails/*.blade.php contains the HTML templates for email verification and password recovery.
- resources/views/errors/database.blade.php displays a controlled HTML page when MySQL is unavailable during web navigation.

This distinction prevents a common misunderstanding: a Laravel application can use Blade and Vue at the same time, but not necessarily for the same screens.

2.9 Eloquent, Migrations, and Seeders: Quick Map

Eloquent is Laravel's ORM. An ORM makes it possible to work with rows in a relational database through models and queries built in PHP. CineVerse has models such as User and Review. Eloquent does not replace MySQL: it generates and executes queries against MySQL and transforms the results into objects that Laravel can use.

Conceptual diagram based on Eloquent queries used by CineVerse Controllers; it is not a literal block copied from a single file.

```
Review::query()
-> where("user_id", $userId)
-> orderByDesc("created_at")
-> get()
-> SQL executed against MySQL
-> collection of Review models
```

A migration describes changes to the database structure, while a seeder loads test data. database/migrations contains the reproducible history of the schema, and database/seeders contains the scenario with 5,000 users and 24,322 reviews.

2.10 Middleware, Request, Form Request, and Controller

These pieces appear in almost every backend flow and should be distinguished from the outset.

- Middleware: runs around a route and can allow, transform, or stop the request before it reaches the Controller. auth, guest, signed, and throttle are real examples used by the project.
- Request: represents the current HTTP request; it can read JSON, query parameters, files, the authenticated user, the session, and headers.

- **Form Request:** a specialized Laravel class that encapsulates preparation, authorization, and validation. `LoginRequest` also performs authentication and uses `RateLimiter`.
- **Controller:** coordinates the final action of a route. It receives the `Request` or `Form Request`, uses models/facades, and returns a response.

```
POST /login
-> guest middleware
-> LoginRequest prepares and validates
-> AuthenticatedSessionController::store()
-> session()->regenerate()
-> HTTP 204
```

2.11 Facades and Framework Services Used in CineVerse

Laravel provides simple access to framework services through classes known as facades. They appear in CineVerse because they make it possible to use already-configured features—authentication, hashing, attempt limits, password recovery, logging, and signed URLs—without building those mechanisms from scratch.

- **Auth:** signs a user in or out through the web guard, Laravel's mechanism for associating a request with the authenticated user stored in the session.
- **Hash:** creates and verifies password hashes. A hash is a cryptographic representation that makes it possible to verify a password without storing its original text.
- **RateLimiter:** counts attempts and decides when to temporarily block a login.
- **Password:** uses the Password Broker, Laravel's mechanism for coordinating the creation and validation of password-reset tokens.
- **Log:** writes records through the channels defined in `config/logging.php`.
- **URL:** generates the temporary signed URL used for email verification.

These facades are not databases or separate processes. They are ways to access services that Laravel already provides; their behavior is based on the configuration in `config/*.php` and the framework's providers.

2.12 Breeze: Initial Authentication Scaffolding, Not a Separate Engine

Laravel Breeze appears in `require-dev`. Its role was to provide an initial authentication code structure—what development terminology calls scaffolding—including controllers such as `AuthenticatedSessionController` and `RegisteredUserController`, `LoginRequest`, auth routes, and `Feature/Auth` tests. CineVerse took that foundation and adapted it. Breeze does not remain running as an independent layer: at runtime, login is handled by `Auth`, `Hash`, `RateLimiter`, the web guard session, and `App\Models\User`.

CineVerse also changed behavior from the initial Breeze scaffolding. Registration creates the account and sends a verification email, but it does not automatically sign the user in. `LoginRequest` requires a verified email before `Auth::login`. The visible screens are CineVerse's own components, not the pages originally generated by Breeze.

2.13 Sanctum Is Installed, but It Does Not Define the Main Login Flow

`laravel/sanctum` is present in `composer.json` and `config/sanctum.php` exists, but the functional account and review routes use `web` and `auth` middleware. `config/auth.php` defines the web guard with the session driver: the guard determines how Laravel identifies the user for a request, and the session driver means that identity is preserved through the session. Therefore, describing CineVerse as an application authenticated with Sanctum tokens would be incorrect for this version: the main flow uses Laravel sessions and cookies.

3. Inertia: How Laravel and Vue Connect in CineVerse

Inertia is the component that allows Laravel to deliver an application built with Vue without turning every screen into a separate Blade template. In CineVerse it acts as an entry bridge: Laravel receives browser navigation, decides to deliver the CineVerse Inertia page, and the existing Vue application is mounted from that page.

The term bridge does not mean that a single class performs all the work. The connection is built across several project components: `routes/web.php` chooses the Inertia page; `HandleInertiaRequests` configures the root view and shared data; `app.blade.php` prepares the HTML document; `app.ts` starts Inertia and Vue;

Pages/CineVerse.vue adapts the page Inertia expects to the actual application; and cineverse/App.vue finally hands the central view over to Vue Router.

3.1 What Problem Inertia Solves in This Project

CineVerse has a Laravel backend and a Vue interface within the same repository. The browser needs to enter through Laravel because Laravel controls the web routes, session, middleware, and initial HTTP response. The main interface, however, is not written in Blade; it is built with Vue components. Inertia acts as a bridge between Laravel and Vue and allows both parts to be integrated into the same application.

In this project, Inertia does not replace Vue Router. Its main role ends after delivering and mounting the CineVerse application. Once Vue is running in the browser, Vue Router interprets internal routes such as /es/movies/550, /es/profile, or /en/favorites and decides which View should appear inside RouterView.

```
Browser
-> Laravel
-> Inertia
-> CineVerse Vue page
-> cineverse/App.vue
-> Vue Router
-> specific View
```

3.2 What Inertia::render("CineVerse") Means Exactly

The connection begins in routes/web.php. The root route redirects to /es. The main /{locale} route and the /{locale}/{any} catch-all deliver the same logical Inertia page: CineVerse. The visible authentication routes /{locale}/{authPage} also deliver it to guests, but redirect to the home page if a session already exists. The name CineVerse is not yet MovieDetailView, HomeView, or ProfileView; it is the name of the page that Inertia must resolve in the frontend.

Location in the project: routes/web.php · **Laravel web routes** · /{locale} routes and catch-all

Used by: browser / Inertia. Responsibility: deliver the CineVerse Inertia page.

```
Route::get('/{locale}', function (string $locale) {
    return Inertia::render('CineVerse', [
        'locale' => $locale,
    ]);
})->whereIn('locale', ['es', 'en']);

Route::get('/{locale}/{any}', function (string $locale) {
    return Inertia::render('CineVerse', [
        'locale' => $locale,
    ]);
})->whereIn('locale', ['es', 'en'])
->where('any', '.*');
```

If the browser requests /es/movies/550, Laravel does not look for an Inertia page called MovieDetail. The catch-all route recognizes locale="es" and any="movies/550", but still responds with Inertia::render('CineVerse'). The deep URL is preserved; Vue Router then uses it to select MovieDetailView.

3.3 HandleInertiaRequests: The Laravel Class That Configures the Integration

app/Http/Middleware/HandleInertiaRequests.php extends InertiaMiddleware. This is the Laravel-specific class where the project configures global Inertia behavior. It does not mount Vue by itself, but it defines which Blade view serves as the root document and which global props are shared with Inertia pages.

Location in the project: app/Http/Middleware/HandleInertiaRequests.php · **Inertia Middleware** · HandleInertiaRequests::share()

Used by: web group. Responsibility: define rootView and share auth.user.

```
class HandleInertiaRequests extends Middleware
{
    protected $rootView = 'app';

    public function share(Request $request): array
    {
        return [
            ...parent::share($request),
            'auth' => [
                'user' => $request->user(),
            ],
        ];
    }
}
```

protected \$rootView = 'app' makes resources/views/app.blade.php the root view. The share() method adds auth.user to Inertia's global data. That shared value does not replace the GET /user request that useAuthStore later uses to reconstruct the session and obtain csrf_token; they are two different mechanisms that coexist in the project.

3.4 app.blade.php: The HTML Shell Returned by Laravel

The resources/views/app.blade.php view is the initial HTML document. It does not build Home, Movies, or Profile. Its job is to prepare the container where Inertia will mount Vue and to load the required assets through Vite.

Location in the project: resources/views/app.blade.php · **Root Blade view** · app.blade.php

Used by: HandleInertiaRequests::\$rootView. Responsibility: load Vite and insert Inertia's mount point.

```
<title inertia>{{ config('app.name', 'Laravel') }}</title>

@routes
@vite(['resources/js/app.ts', "resources/js/Pages/{{ $page['component'] }}.vue"])
@inertiaHead

<body>
    @inertia
</body>
```

@vite loads resources/js/app.ts and the current Inertia page component. The \$page['component'] expression contains the logical name sent by Inertia; when Laravel called Inertia::render('CineVerse'), that component is CineVerse. @inertia generates the mount point and the initial page information the Inertia client needs to start the application.

3.5 app.ts: Where the Name Sent by Laravel Becomes a Vue Component

resources/js/app.ts is the integrated frontend entry point. createInertiaApp() receives the page sent by Laravel, resolves its Vue component, and creates the single main Vue instance used by CineVerse.

Location in the project: resources/js/app.ts · **Vue/Inertia entry point** · createInertiaApp()

Used by: resources/views/app.blade.php / Inertia. Responsibility: resolve Pages/CineVerse.vue and install Pinia, Router, Vue Query, i18n, and Ziggy.

```
createInertiaApp({
  resolve: (name) =>
    resolvePageComponent(
      `./Pages/${name}.vue`,
      import.meta.glob<DefineComponent>('./Pages/**/*.vue'),
    ),

  setup({ el, App, props, plugin }) {
    const pinia = createPinia();
    pinia.use(piniaPluginPersistedState);

    const app = createApp({
      render: () => h(App, props),
    });

    app
      .use(plugin)
      .use(pinia)
      .use(router)
      .use(VueQueryPlugin)
      .use(i18n)
      .use(ZiggyVue);

    useAuthStore().initialize();

    router.isReady().then(() => {
      app.mount(el);
    });
  },
});
```

The key point is resolve. Laravel sent the name CineVerse, so name equals "CineVerse". The template `./Pages/\${name}.vue` becomes ./Pages/CineVerse.vue, and resolvePageComponent loads that file.

```
Laravel:
Inertia::render('CineVerse')

    -> page name

app.ts:
name = 'CineVerse'

    -> resolve()

./Pages/CineVerse.vue
```

setup() receives four important pieces. el is the HTML element where Vue will be mounted; App is the root component provided by Inertia; props contains the page data; plugin installs the Inertia integration within Vue. The App received by setup() should not be confused with resources/js/cineverse/App.vue: they are different components with similar names.

createApp({ render: () => h(App, props) }) creates the Vue application that renders Inertia's root component with the received props. That same instance then registers Pinia, Vue Router, Vue Query, vue-i18n, and Ziggy. There are not two separate Vue applications: Inertia and CineVerse ultimately run inside the same instance.

3.6 Pages/CineVerse.vue: The CineVerse-Specific Adapter

Inertia looks for its pages under resources/js/Pages, while CineVerse's original architecture lives under resources/js/cineverse. Rather than moving the entire application into Pages, the project uses a minimal adapter.

Location in the project: resources/js/Pages/CineVerse.vue · **Inertia Vue page** · CineVerse.vue

Used by: resources/js/app.ts. *Responsibility:* adapt the page resolved by Inertia to cineverse/App.vue.

```
<script setup lang="ts">
import CineVerseApp from '@/cineverse/App.vue';
</script>

<template>
  <CineVerseApp />
</template>
```

This file is the most visible connection between Inertia's convention and the frontend's own structure. Inertia can resolve Pages/CineVerse.vue by name, and that page in turn mounts resources/js/cineverse/App.vue. This allows HomeView, MoviesView, ProfileView, and the other views to retain their original organization.

3.7 cineverse/App.vue: Where the Bridge Ends and Vue Router Begins

Location in the project: resources/js/cineverse/App.vue · **Root component of the CineVerse SPA** · template with RouterView. **Used by** Pages/CineVerse.vue when it mounts CineVerseApp. AppHeader and AppFooter remain outside RouterView; RouterView is where Vue Router places the View that corresponds to the current URL.

```
<div class="app-content">
  <RouterView v-slot="{ Component, route }">
    <component
      :is="Component"
      :key="routeKey(route)"
    />
  </RouterView>
</div>
```

For /es/movies/550, Vue Router finds the MovieDetail route and selects MovieDetailView; Inertia no longer performs that work. **Location in the project:** resources/js/cineverse/core/router/index.ts · **global router** · MovieDetail route definition. The following fragment comes from that file and shows the route with :lang and the movie's numeric id.

```
{
  path: "[:lang(en|es)/movies/:id(\\d+)",
  name: "MovieDetail",
  component: MovieDetailView,
}
```

The responsibility split is therefore: Laravel decides which application to deliver during a full HTTP navigation; Inertia connects that response to Vue; Vue Router decides which internal View to display once the application is mounted.

3.8 Complete Example: Carlos Pastes /es/movies/550 and Presses Enter

This flow begins outside the SPA. CineVerse is not yet mounted in the browser, so the request reaches Laravel first.

```

1. Browser -> GET /es/movies/550
2. Laravel compares the URL with routes/web.php
3. /{locale}/{any} matches
4. locale = "es" and any = "movies/550"
5. Laravel executes Inertia::render("CineVerse", ["locale" => "es"])
6. HandleInertiaRequests selects rootView = "app"
7. Laravel returns resources/views/app.blade.php
8. @vite loads resources/js/app.ts
9. @inertia prepares the page mount point
10. createInertiaApp receives component = "CineVerse"
11. resolve() loads ./Pages/CineVerse.vue
12. setup() creates the Vue instance and registers the plugins
13. CineVerse.vue mounts cineverse/App.vue
14. Vue Router reads the URL /es/movies/550
15. Vue Router selects MovieDetailView
16. RouterView mounts MovieDetailView
17. useMovieDetail starts the TMDB queries

```

The movie was not selected by Laravel or Inertia. Laravel only ensured that the correct application reached the browser. MovieDetailView is selected when Vue Router interprets the same URL after the application has been mounted.

3.9 What Changes During Internal Navigation Without F5

Now suppose CineVerse is already open and Carlos clicks another MovieCard. The Vue application already exists. Vue Router can change the route and replace the View inside RouterView without asking Laravel for the CineVerse page again for every screen.

```

MovieCard
-> router.push({ name: "MovieDetail", lang, id })
-> Vue Router updates the URL
-> RouterView receives the new route
-> MovieDetailView mounts or remounts according to routeKey
-> useMovieDetail requests the required data

```

During this internal navigation, the complete routes/web.php -> Inertia::render() -> app.blade.php -> createInertiaApp() flow does not run again. If the user presses F5 on the new URL, then a full HTTP navigation begins again and Laravel must deliver the application once more.

3.10 Inertia and Vue Router Are Not Competing Routers

The possible confusion comes from the fact that Laravel, Inertia, and Vue Router all participate around the same URL. They operate at different times, however. Laravel resolves the HTTP entry point; Inertia delivers the Vue page; Vue Router resolves the internal screen.

```

FULL LOAD / RELOAD
Browser -> Laravel -> Inertia -> Vue -> Vue Router -> View

INTERNAL NAVIGATION
View / component -> Vue Router -> RouterView -> new View

```

CineVerse chose to keep Vue Router because the frontend was already a complete SPA before it was integrated with Laravel. That is why Pages/Home.vue, Pages/Movies.vue, and Pages/Profile.vue do not exist as separate Inertia pages. There is a single bridge page, Pages/CineVerse.vue, and the original Vue Router architecture continues behind it.

3.11 What Data Laravel Can Share Through Inertia

HandleInertiaRequests shares auth.user through share(). Those props can travel with the Inertia page and be available in the frontend. In this version, CineVerse also keeps an explicit GET /user contract so that useAuthStore can reconstruct user and csrfToken from the Laravel session after a reload.

```
Laravel / Inertia:
share() -> auth.user

CineVerse auth store:
GET /user -> user + csrf_token -> Pinia
```

The presence of both paths does not mean there are two authentication systems. The actual identity still belongs to the Laravel session. Inertia can share context from the initial request, while Pinia maintains a reactive representation for the interface.

3.12 What Inertia Does Not Do in CineVerse

Inertia does not query TMDB, store reviews, replace Eloquent, authenticate users by itself, replace Axios, or decide which movie MovieDetailView displays. It does not replace Vue Router in this architecture either.

Its central responsibility can be summarized as follows: Laravel decides to deliver the CineVerse page; Inertia carries that decision to Vue and enables the application to mount; Vue Router then takes control of the internal views.

4. TMDB: What It Is, How Access Is Obtained, and How CineVerse Uses It

TMDB provides an external API that allows CineVerse to query structured information about movies and people. An API is an interface through which one program can request data from another system; within that API, each specific address representing an operation is called an endpoint. CineVerse centralizes the shared TMDB access configuration in `tmdbApi.ts`, while `movies.service.ts` and `person.service.ts` specify which endpoint they need to query, which parameters to send, and what type of response they expect.

4.1 What Is The Movie Database (TMDB)

TMDB stands for The Movie Database. CineVerse does not install TMDB or copy its website: it uses its v3 API, an external Internet service that makes it possible to automatically query information about movies and people through HTTP requests. MySQL stores CineVerse's own data; TMDB provides the audiovisual catalog.

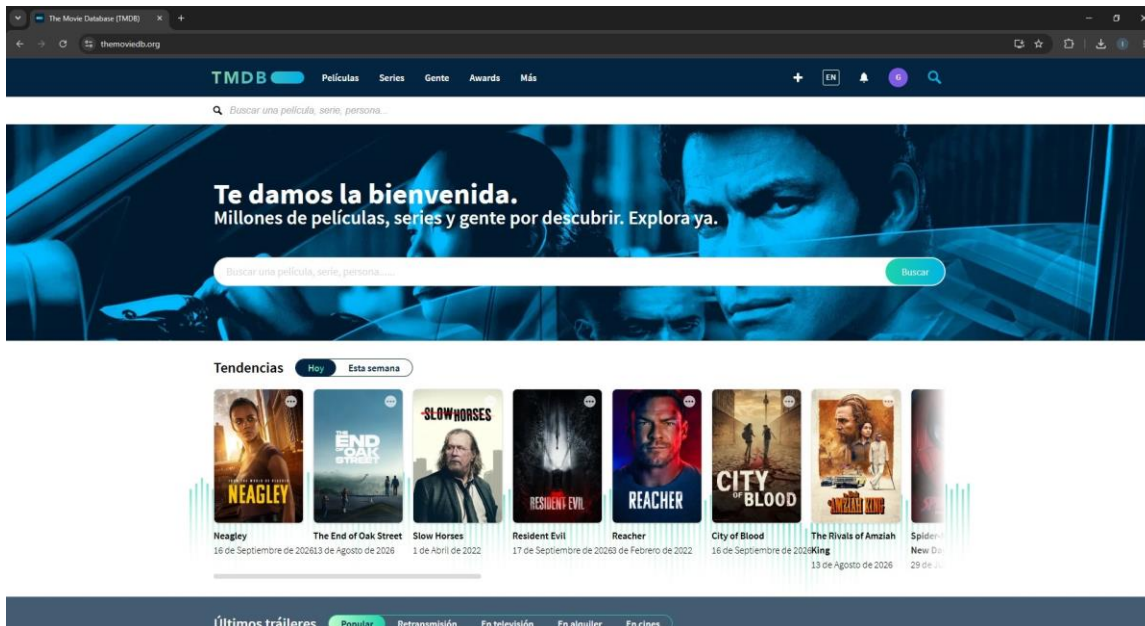


Figure. The Movie Database public website. CineVerse consumes the API; it does not copy this HTML page.

Address / System	Function
www.themoviedb.org	Website visited by a person.
api.themoviedb.org	API to which tmdbApi.ts sends HTTP requests.
image.tmdb.org	Image server used for posters, backdrops, and photos.
CineVerse MySQL	Users, sessions, reviews, and other application-owned data; it does not contain the full TMDB catalog.

4.2 What Information CineVerse Gets from TMDB

TMDB provides titles, synopses, dates, genres, ratings, posters, backdrops, cast, crew, videos, similar movies, and person data. CineVerse selects the fields it needs and combines several endpoints when a screen requires different kinds of information.

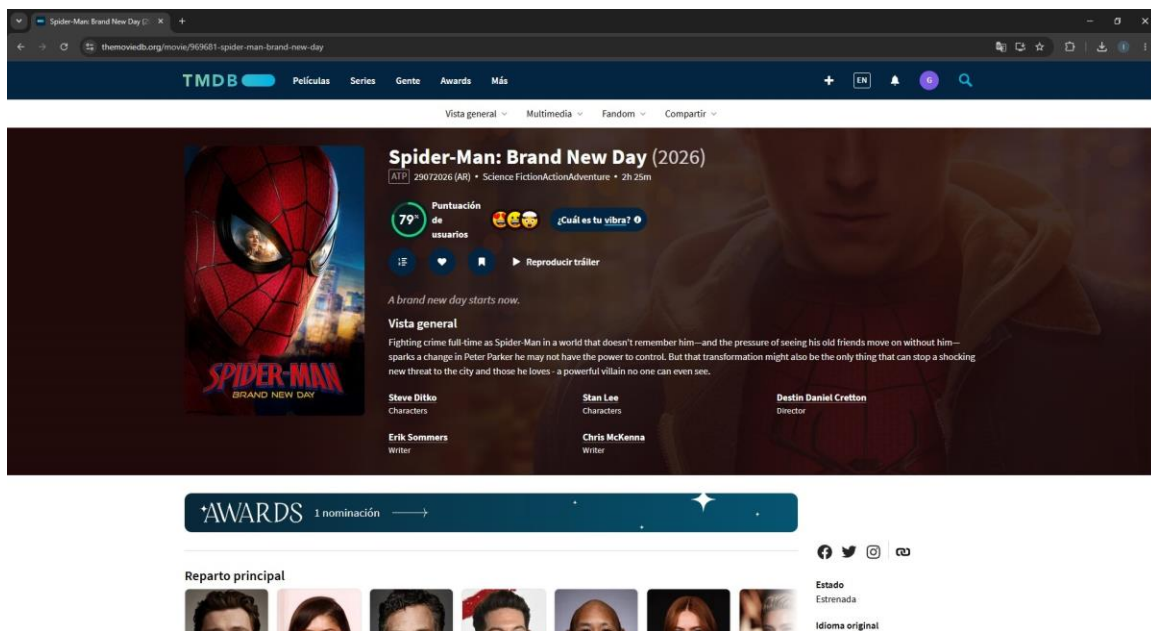


Figure. Public movie page on TMDB.

The separation from MySQL is visible in reviews: the table stores movie_id and movie_title, but there is no local movies table containing a complete copy of the catalog.

4.3 How a Developer Obtains Access and Which Credential CineVerse Uses

API access is managed through a TMDB account. The API settings provide the credentials that are later copied into the local environment. A credential is a piece of information that an application presents to a service to demonstrate that it is authorized. In CineVerse, VITE_TMDB_TOKEN contains the API Read Access Token provided by TMDB.

An access token is a character string that identifies the authorization used to query an API. CineVerse does not send it in the URL: tmdbApi.ts places it in the HTTP Authorization header. An HTTP header is additional information sent with the request, separate from the address and the main data.

The word Bearer is not another token. It is the name of the scheme used in the Authorization header to indicate that the value that follows is an access token. Therefore, Authorization: Bearer <token> simply means, “this request presents this token as its credential.”

The other important variable is VITE_TMDB_BASE_URL. It contains the base URL https://api.themoviedb.org/3, the common initial portion of the API addresses. The services then append a specific endpoint, such as /movie/top_rated, to form the complete request address.

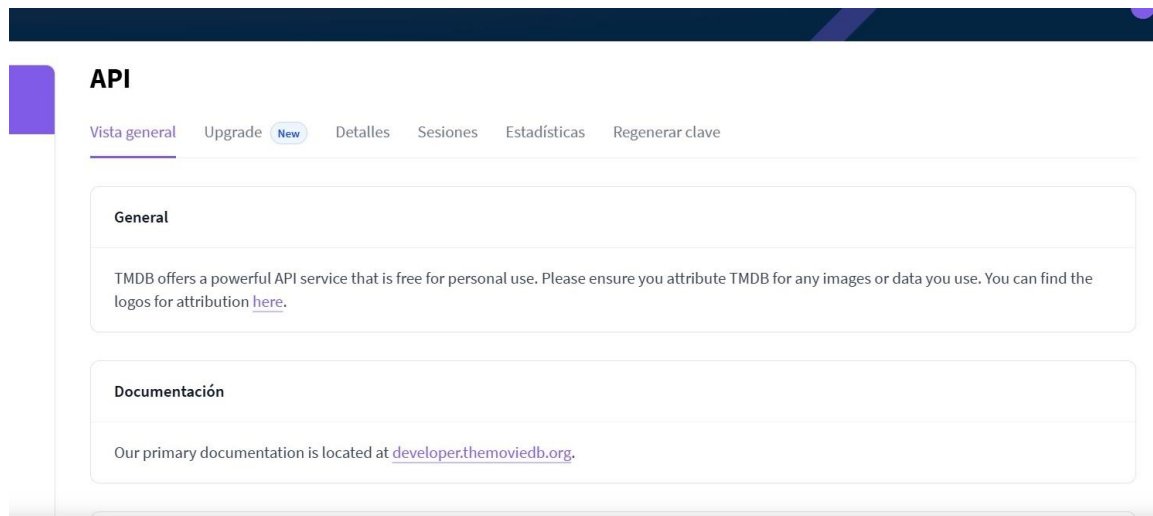


Figure. API settings section of a TMDb account.

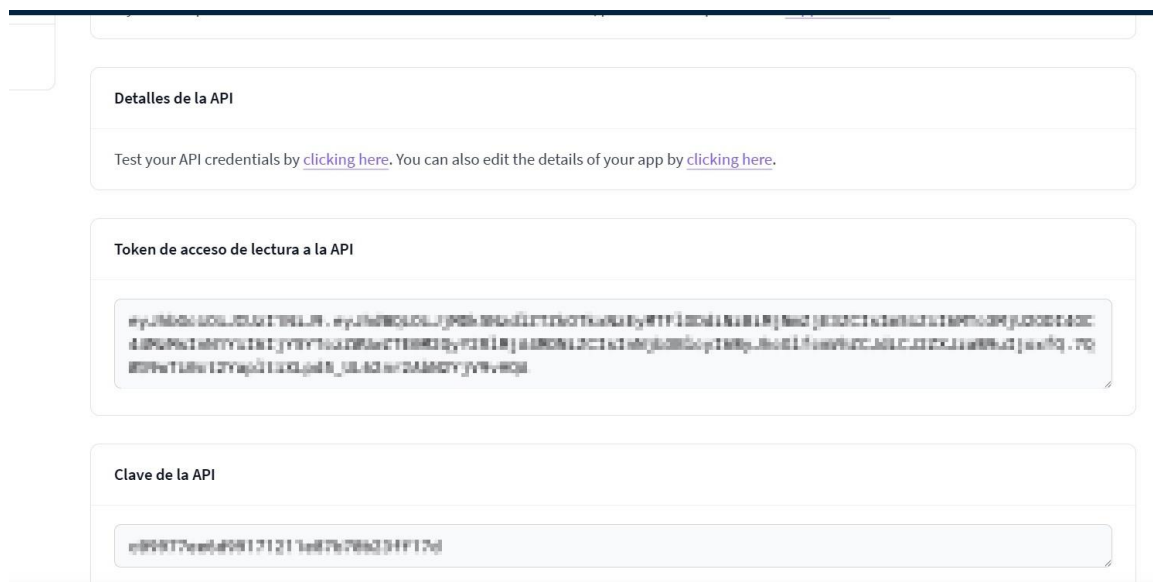


Figure. TMDb API credentials with sensitive values redacted.

TMDb documentation also contains examples that use a v3 API Key in the URL's `api_key` parameter. CineVerse does not use that mechanism in its code: it uses the API Read Access Token and sends it through `Authorization: Bearer <token>`.

Location in the project: `.env` and `.env.example` contain `VITE_TMDB_BASE_URL` and `VITE_TMDB_TOKEN`; `resources/js/cineverse/core/api/tmdbApi.ts` reads those variables and creates the shared Axios client. The following fragment shows how the base URL and token reach that configuration.

```
# .env
VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"
VITE_TMDB_TOKEN="<API Read Access Token>"

// resources/js/cineverse/core/api/tmdbApi.ts
const api = axios.create({
  baseURL: import.meta.env.VITE_TMDB_BASE_URL,
  headers: {
    Authorization: `Bearer ${import.meta.env.VITE_TMDB_TOKEN}`,
    "Content-Type": "application/json",
  },
});
```

axios.create() does not search for any movie yet. It only creates a reusable HTTP client. baseUrl defines the common initial portion of the addresses, and headers defines headers that will accompany requests, including Authorization. The services later decide which endpoint and parameters each operation needs.

Summary: VITE_TMDB_BASE_URL indicates where the TMDB API begins; VITE_TMDB_TOKEN contains the access credential; Bearer indicates how that credential is presented in the Authorization header. Because VITE_ variables become available in browser JavaScript, they should not be treated as private server secrets.*

4.4 Which Files Actually Communicate with TMDB

```
View / Component
-> Composable
-> movies.service.ts / person.service.ts
-> tmdbApi.ts
-> Axios
-> HTTPS
-> TMDB API
-> JSON
-> Vue Query / Composable
-> interface
```

Location in the project: resources/js/cineverse/core/api/tmdbApi.ts · shared HTTP client.

resources/js/cineverse/features/movies/services/movies.service.ts · movie endpoints.

resources/js/cineverse/features/person/services/person.service.ts · person details and filmography.

4.5 Complete Example: The User Opens “Top Rated”

The example begins with a visible action. The user enters the movie explorer and chooses Top Rated. useMoviesQuery detects listType="top_rated" and selects the predefined-list path; movies.service.ts then requests the corresponding page from TMDB.

```
User selects "Top Rated"
-> MoviesView.vue keeps listType="top_rated"
-> useMoviesQuery.ts executes its query function (queryFn)
-> getMovieList("top_rated", page, language)
-> movies.service.ts
-> api.get("/movie/top_rated", { params: { page, language } })
-> tmdbApi.ts adds baseUrl + Authorization
-> TMDB returns page + results + total_pages + total_results
-> Vue Query keeps the received page
-> MovieResults combines the loaded results
-> MovieCard renders each movie
```

TMDB defines pagination in its response. CineVerse decides which page to request, when to request the next one, and how to keep the pages already obtained together.

4.5.1 What HTTP Request This Flow Builds

Teaching example equivalent to a getMovieList() call; it is not a literal block with a fixed "top_rated" value. The actual implementation is in resources/js/cineverse/features/movies/services/movies.service.ts · Frontend Service · getMovieList().

```
const { data } = await api.get<PaginatedResponse<Movie>>(
  "/movie/top_rated",
  { params: { page: 1, language: "es" } }
);
```

In that flow, getMovieList() uses the shared Axios instance. VITE_TMDB_BASE_URL provides the base URL https://api.themoviedb.org/3; the service provides the /movie/top_rated endpoint; and page=1 together with language=es are added as parameters. The result is a request equivalent to GET

`https://api.themoviedb.org/3/movie/top_rated?page=1&language=es`. `tmdbApi.ts` also adds the Authorization: Bearer <token> header.

4.5.2 What CineVerse Sends and What It Receives from TMDB

`tmdbApi.ts` creates a single Axios instance for TMDB. It configures `VITE_TMDB_BASE_URL` as the base URL and Authorization: Bearer <token> as a shared header. `movies.service.ts` then only needs to specify the endpoint and parameters for each query.

A GET search does not send a complete movie or a JSON object containing all filters. Values such as query, page, and language become URL parameters. The credential travels separately inside the HTTP Authorization header.

```
GET https://api.themoviedb.org/3/search/movie?query=Batman&page=1&language=es
Authorization: Bearer <VITE_TMDB_TOKEN>
Content-Type: application/json
```

TMDB responds with JSON. For paginated endpoints, CineVerse expects a `PaginatedResponse<Movie>` structure: page identifies the page received, results contains the movies, and `total_pages/total_results` describe the pagination reported by TMDB.

The following block is a reduced example of the shape expected by the code; the values are illustrative and are not a capture of a specific TMDB response:

```
{
  "page": 1,
  "results": [
    {
      "id": 123,
      "title": "Example Movie",
      "release_date": "2026-01-01",
      "poster_path": "/poster.jpg",
      "backdrop_path": "/backdrop.jpg",
      "vote_average": 7.8,
      "popularity": 120.5,
      "genre_ids": [28, 12],
      "overview": "Example synopsis"
    }
  ],
  "total_pages": 20,
  "total_results": 400
}
```

Axios places that JSON in data. The `PaginatedResponse<Movie>` type helps TypeScript understand the structure the program expects, but it does not modify or validate the response arriving over the network by itself. The code then uses `data.results` to obtain the movies and their fields.

4.5.3 Manually Testing an Endpoint in Postman

A TMDB call can be tested outside CineVerse to inspect the URL, parameters, HTTP status, and returned JSON. The following screenshot uses a v3 API Key as a URL parameter only to demonstrate the endpoint manually in Postman. CineVerse's actual code uses the mechanism explained earlier: an API Read Access Token inside Authorization: Bearer <token>.

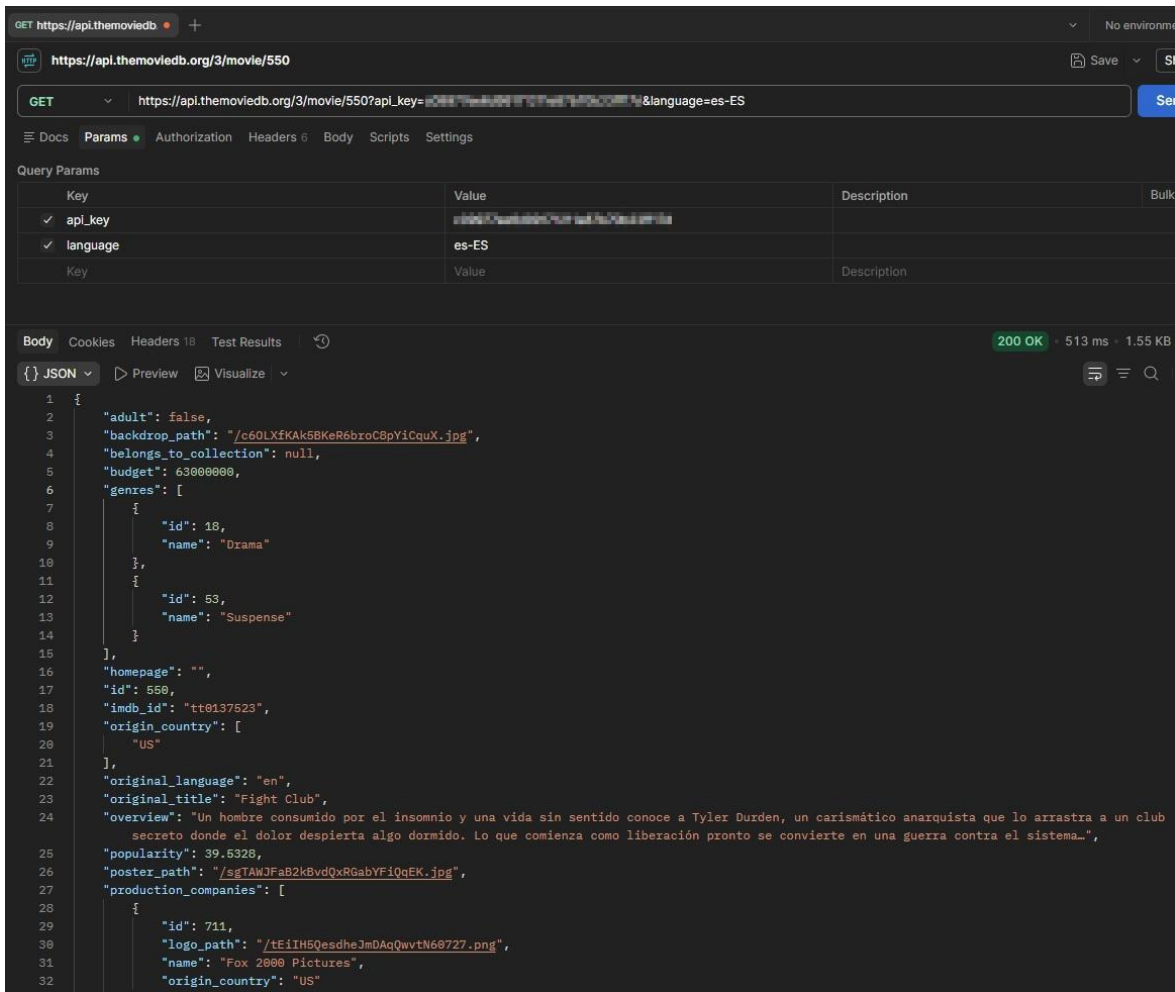


Figure. Manual GET /3/movie/550 test in Postman and 200 OK response.

4.6 What TMDb Returns and How TypeScript Interprets It

TMDb responds with JSON. A detail endpoint returns an object; paginated endpoints such as `top_rated` or `search/movie` include `page`, `results`, `total_pages`, and `total_results`. CineVerse declares TypeScript types to express the structure it expects.

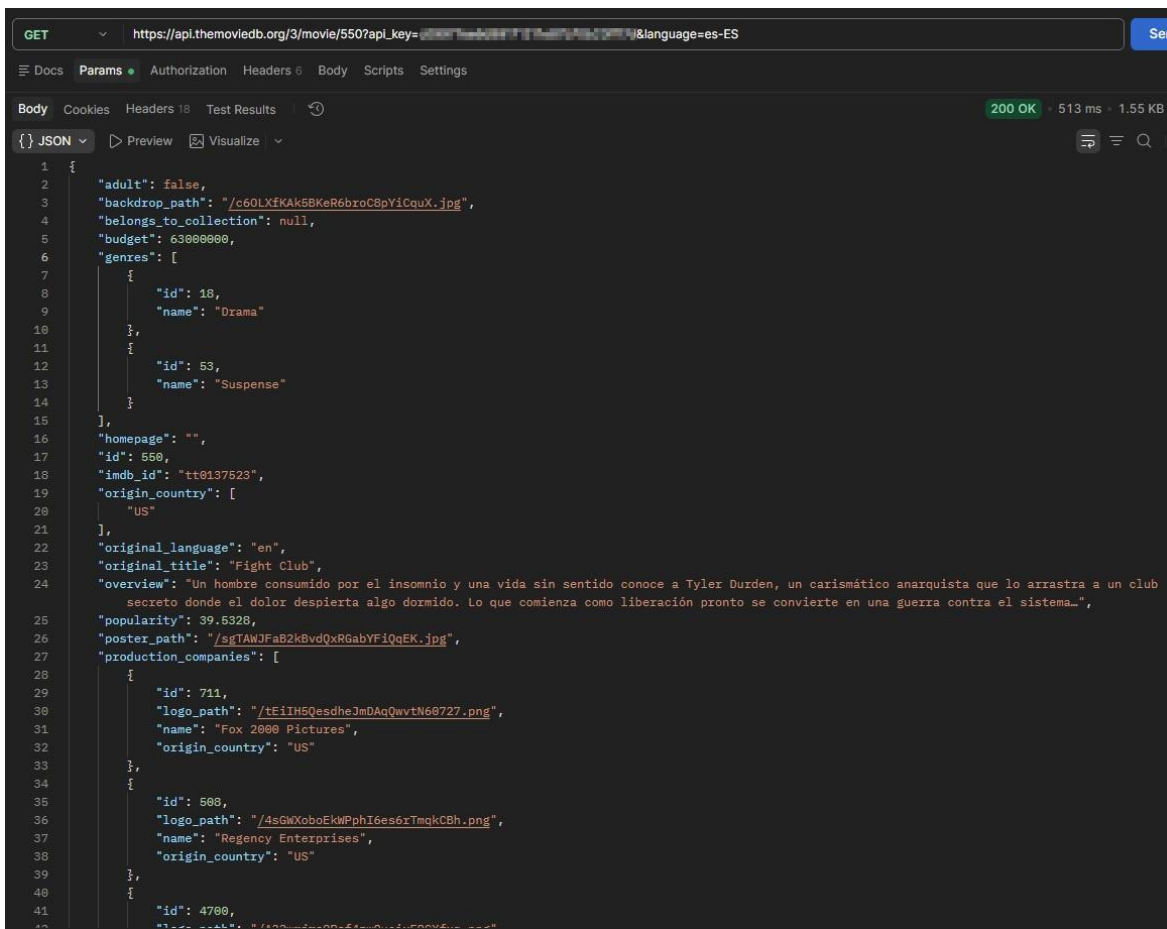


Figure. Fragment of a TMDB JSON response containing movie data.

Location in the project: resources/js/cineverse/share/d/types/paginated-response.ts · generic
PaginatedResponse<T> interface. It is used by movies.service.ts and other services that type paginated TMDB responses.

```
export interface PaginatedResponse<T> {
  page: number;
  results: T[];
  total_pages: number;
  total_results: number;
}
```

PaginatedResponse<Movie> tells TypeScript that results is Movie[]. The type helps during development, but it does not validate JSON that has already arrived over the network; the application still needs to handle error states and optional values.

4.7 Endpoints Used by the Project

TMDB Endpoint	Use
/trending/movie/day	Home page trends.
/movie/popular	Popular catalog.
/movie/now_playing	Now playing.
/movie/upcoming	Upcoming releases.
/movie/top Rated	Top rated.

TMDB Endpoint	Use
/search/movie	Text search.
/discover/movie	Advanced filters without text.
/genre/movie/list	Available genre catalog.
/movie/{id}	Movie details.
/movie/{id}/credits	Cast and crew.
/movie/{id}/videos	Videos/trailers.
/movie/{id}/similar	Similar movies.
/person/{id}?append_to_response=movie_credits	Actor details + filmography in one request.

4.8 Images

TMDB does not send the image itself inside the JSON. It returns a relative path, such as /abc123.jpg. The frontend combines that path with the image server image.tmdb.org and chooses an appropriate size. That server acts as a CDN, a network designed to deliver image files efficiently from different locations.

4.9 What Happens When Data Is Missing in a Language

TMDB does not always provide every field in every language. CineVerse applies several fallback mechanisms to try to display an image, credits, or videos even when the first response is empty.

- `getMovieDetail` requests the alternate language only if both poster and backdrop are empty.
- `getMovieCredits` requests the alternate language if both cast and crew are empty.
- `getMovieVideos` requests /videos again without language when there are no results.
- `getSimilarMovies` currently uses the fixed value `language="en-US"`, regardless of the UI language.

4.10 People and `append_to_response`

TMDB uses the person resource for actors and other people connected with film. CineVerse requests a person's details and filmography in a single request by using `append_to_response=movie_credits`. That parameter tells TMDB to include movie-credit information within the same main response.

```
GET /person/{id}
?language=es
&append_to_response=movie_credits

TMDB responds with:
main person data
+ movie_credits inside the same JSON
```

`person.service.ts` returns that structure to `usePersonDetail`. `PersonHero` consumes the main data, `PersonKnownFor` selects highlighted credits, and `PersonFilmography` sorts the filmography by year.

4.11 A CineVerse Account and a TMDB Account Are Not the Same Thing

The TMDB account is used only to manage developer API access and obtain the credential CineVerse needs. The account Carlos creates inside CineVerse is stored in the MySQL users table and authenticated with Laravel. Registering with CineVerse does not create a TMDB account, and CineVerse login does not send an email address or password to TMDB.

```
TMDB account
-> belongs to the developer / API configuration
```

```
-> provides access credentials

CineVerse account
-> users in MySQL
-> Laravel Auth + session
-> reviews and profile
```

4.12 Reference Resources Used When Working with TMDB

The official TMDB API v3 documentation is used to consult API endpoints and parameters. The public themoviedb.org website is used to explore the catalog as a user; api.themoviedb.org/3 is the base URL for data requests; image.tmdb.org serves image files. CineVerse combines these services, but each has a different responsibility.

5. General Architecture of CineVerse Laravel

The simplest way to understand CineVerse Laravel is to separate three flows: how the browser enters the application, how movie data is obtained, and how CineVerse-owned data is handled. Those flows can occur within the same screen without being confused with one another.

```
APPLICATION ENTRY
Browser /es/movies/550
-> routes/web.php
-> Inertia::render("CineVerse")
-> app.blade.php + resources/js/app.ts
-> Pages/CineVerse.vue
-> cineverse/App.vue
-> Vue Router
-> MovieDetailView

MOVIE CATALOG
MovieDetailView
-> useMovieDetail
-> movies.service.ts
-> tmdbApi.ts
-> TMDB
-> Vue Query
-> Vue components

APPLICATION-OWNED DATA
MovieReviews / LoginView / ProfileView
-> frontend service
-> webApi.ts
-> routes/auth.php or routes/api.php
-> Controller / Request
-> Eloquent
-> MySQL
```

5.1 Three Flows Coexisting in the Same Application

The architecture is easiest to understand by separating three flows. The first delivers the application to the browser; the second obtains the movie catalog directly from TMDB; the third handles identity and application-owned data through Laravel and MySQL.

```
ENTRY
Browser -> Laravel -> Inertia -> Vue -> Vue Router

CATALOG
View -> Composable -> Service -> tmdbApi -> TMDB -> Vue Query -> UI

APPLICATION-OWNED DATA
View -> Service -> webApi -> Laravel -> Eloquent -> MySQL -> JSON -> UI
```

A single screen can activate more than one flow. MovieDetailView gets the title, poster, cast, and trailer from TMDB; when Carlos opens the Reviews tab, MovieReviews queries Laravel and MySQL. Vue combines both sources in the same interface.

5.2 Laravel Does Not Render Every CineVerse Screen

In a traditional Inertia application, there could be a different Inertia page for each site route. CineVerse did not adopt that pattern. `routes/web.php` returns `Inertia::render("CineVerse")` for the localized route and the catch-all. The `Pages/CineVerse.vue` component is deliberately minimal and mounts `CineVerseApp`. As a result, Laravel does not need separate `MovieDetail`, `PersonDetail`, or `Favorites` Inertia pages.

The URL therefore serves two purposes at different moments. During a full load or reload, Laravel must recognize `/es/...` and deliver the app. Once Vue is mounted, Vue Router interprets that same URL to select the internal view. Section 3 develops this boundary in detail using the project's actual source code.

5.3 The Same Screen Can Use TMDB and Laravel

The movie detail screen is the clearest example. The synopsis, poster, cast, and trailers come from TMDB. The Reviews tab, by contrast, calls `/api/movies/{movieId}/reviews` and obtains the average rating, count, the user's own review, and community comments from MySQL. Vue presents both sources as a single experience without requiring the entire movie to be copied into the local database.

5.4 Where State Lives

Not all state is stored in the same place. The actual session belongs to Laravel and is linked to the browser through a cookie. Pinia maintains a reactive representation of the user and CSRF token so the interface can respond. Favorites are persisted locally in the browser with `pinia-plugin-persistedstate`. Remote TMDB results are cached through Vue Query. This separation avoids using MySQL as a universal store for data that does not require it.

5.5 Relationship Between the General Architecture and the Inertia Bridge

The general architecture shows where each responsibility belongs, but the exact mechanism connecting Laravel and Vue is not repeated here. Section 3 develops that bridge file by file and explains what happens both during a direct load and during internal SPA navigation.

6. Project Structure

CineVerse Laravel is a single project. There is no separate frontend folder and backend folder running as independent applications. Laravel occupies the root, and the CineVerse frontend is integrated under `resources/js`. Understanding this difference prevents old Apache configurations, Vite proxy setups, or the two HTTP servers used by the plain PHP project from being repeated unnecessarily.

```
cineverse-laravel/
├── app/
│   ├── Http/Controllers/
│   ├── Http/Middleware/
│   ├── Http/Requests/
│   ├── Models/
│   ├── Notifications/
│   ├── Logging/
│   └── Support/
├── bootstrap/
├── config/
├── database/
│   ├── migrations/
│   └── seeders/
├── public/
├── resources/
│   ├── js/app.ts
│   ├── js/Pages/CineVerse.vue
│   ├── js/cineverse/
│   └── views/
├── routes/
│   ├── web.php
│   ├── auth.php
│   └── api.php
```

```

├─ tests/
├─ composer.json
├─ package.json
├─ vite.config.js

```

6.1 app/: Laravel Server Code

app/Http/Controllers/Auth contains the account flows derived from Breeze and adapted to CineVerse. app/Http/Controllers/Api contains JSON endpoints for profile and reviews. app/Http/Requests contains reusable validations such as LoginRequest. app/Models contains User and Review. Notifications defines the project's custom emails, while Support/CineVerseLog and Logging/CineVerseLogFormatter preserve the project's diagnostic format.

6.2 resources/js/: Entry Point and Vue Application

resources/js/app.ts is the Vite and Inertia entry point. resources/js/Pages/CineVerse.vue is the page Inertia can resolve by name. Beneath it, resources/js/cineverse preserves the organization into core, features, layouts, shared, views, and assets. The frontend was not flattened into Pages because the goal was to preserve the original modular architecture.

6.3 database/: Structure and Reproducible Data Scenario

database/migrations describes the schema. The migrations include standard Laravel tables, personal_access_tokens, the transition of users from legacy columns to a shape compatible with Auth, the reviews table, and a migration that removes login_attempts because rate limiting no longer depends on that table. database/seeders loads 5,000 users and 24,322 reviews from NDJSON.

6.4 routes/: Three Different Boundaries

File	What It Receives	What It Returns
routes/web.php	Browser navigation	Redirects or the CineVerse Inertia page.
routes/auth.php	Account actions	Authentication, verification, and password JSON/redirects.
routes/api.php	Application-owned data used by Vue	Profile and review JSON.

6.5 bootstrap/: Framework Startup

bootstrap/app.php builds the Laravel 12 Application instance and registers routing, middleware, and exception handling. bootstrap/providers.php lists the project's own providers that are loaded. This folder does not contain CineVerse screens; it contains framework startup instructions.

6.6 config/: Configuration Translated from .env

The config files turn environment variables into PHP structures that the rest of the framework reads through config(...). database.php defines connections; session.php defines where sessions are stored and how long they live; mail.php defines the mailer; logging.php defines log channels; cache.php defines the store used by cache and RateLimiter.

6.7 public/: The Server's Only Public Root

public/index.php is Laravel's front controller. The web server must expose public/, not the entire repository root. Compiled assets end up under public/build; favicon.ico and robots.txt also live here. .env, app/, config/, and storage/ remain outside the public root.

6.8 resources/: Vue, CSS, and Blade Views

resources/js/app.ts starts Inertia and Vue; resources/js/Pages/CineVerse.vue is the adapter Inertia resolves; resources/js/cineverse contains the actual Vue application. resources/css/app.css is a Vite entry point. resources/views contains app.blade.php, email templates, and the database error view.

6.9 storage/: Sessions, Cache, Logs, and Runtime Files

storage does not contain business source code. Laravel uses it for information generated while the application is running. With SESSION_DRIVER=file, sessions go to storage/framework/sessions; file-based cache uses storage/framework/cache; logs are stored under storage/logs unless CINEVERSE_LOG_PATH redirects them to another folder.

storage contains runtime data, not public application files. It is not equivalent to public: logs and sessions must not be served as downloadable files from the browser.

6.10 tests/: Automated Checks

tests contains PHPUnit tests. Feature/Auth comes largely from Breeze scaffolding and still preserves expectations from the original project that no longer match every CineVerse adaptation. Section 34 separates what the tests are intended to check from what the current code actually represents.

6.11 Example: Which Folders a Review Passes Through

```
resources/js/cineverse/.../MovieReviews.vue
-> resources/js/cineverse/.../reviews.service.ts
-> routes/api.php
-> app/Http/Controllers/Api/ReviewController.php
-> app/Models/Review.php
-> MySQL

database/migrations defines the table
storage/logs participates only if a loggable technical failure occurs
```

7. Configuration Files and Environment Variables

Laravel manages environment-sensitive configuration in .env and translates it into values under config/*.php. The repository includes .env.example as a template, but .gitignore excludes .env because it can contain MySQL passwords, SMTP credentials, and the TMDB token.

7.1 .env → config → Application Flow

```
.env
DB_HOST=127.0.0.1
DB_PORT=3307
DB_DATABASE=cineverse_laravel
    |
    v
config/database.php
    |
    v
DB / Eloquent / migrations
```

The same principle applies to sessions, email, and logs. CineVerse code accesses these values through Laravel configuration. Laravel loads variables from .env, and config/*.php exposes them to the rest of the application through config(...).

7.2 Example of Minimum Configuration for the Local Environment

Reference: .env.example is located at the project root and shows which variables an installation can configure. The following block is not a literal copy of that file: it is a teaching example adapted to CineVerse, with

placeholders and some values from the environment used for these screenshots. The actual .env is local and is not published.

```
APP_NAME=CineVerse
APP_ENV=local
APP_KEY=base64:...
APP_DEBUG=true
APP_URL=http://127.0.0.1:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3307
DB_DATABASE=cineverse_laravel
DB_USERNAME=<local-user>
DB_PASSWORD=<local-password>

SESSION_DRIVER=database
CACHE_STORE=database
QUEUE_CONNECTION=database

VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"
VITE_TMDB_TOKEN="<TMDB-token>"
```

Port 3307 corresponds to the local environment used here and is not a universal Laravel rule. If MySQL is running on 3306 or another port, .env must reflect the actual installation. The same applies to the username, password, and SMTP configuration. In other words, .env.example is a template; each machine completes its own .env with the values it actually uses.

7.3 APP_KEY

APP_KEY is a cryptographic key for the Laravel application. It should not be copied from another machine or invented manually. `php artisan key:generate` creates a secure value and writes it to .env. Laravel uses it for framework cryptographic operations, so a newly cloned installation needs to generate it before it can operate correctly.

7.4 VITE_* Variables

Vite makes environment variables whose names begin with VITE_ available to browser code. CineVerse uses two of them to configure TMDB access: VITE_TMDB_BASE_URL and VITE_TMDB_TOKEN. The first contains the API base URL; the second contains the API Read Access Token used as the credential.

```
VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"
VITE_TMDB_TOKEN="<API Read Access Token>"
```

The base URL is the common portion that appears at the beginning of requests. For example, `movies.service.ts` can request the `/movie/top Rated` endpoint, and Axios combines it with `https://api.themoviedb.org/3` to obtain the complete address `https://api.themoviedb.org/3/movie/top Rated`.

The token serves a different purpose: it is the credential CineVerse presents to TMDB. `tmdbApi.ts` sends it in the HTTP Authorization header in the form `Bearer <token>`. Bearer is not a second credential; it only identifies the scheme used to present the token in that header.

Because VITE_* variables are embedded in the JavaScript Vite delivers to the browser, their values can be inspected from the client. They therefore must not be used for MySQL passwords, private server keys, or other secrets that must remain exclusively in the backend.

When a VITE_* variable is changed while `npm run dev` is running, Vite must be restarted so it can read the value again. APP_URL serves a different purpose: it identifies the main URL of the Laravel application and is not a TMDB credential.

7.5 Session Configuration

`config/session.php` uses `env("SESSION_DRIVER", "database")`, and `.env.example` also proposes `SESSION_DRIVER=database`. With that configuration, Laravel uses the `sessions` table. If `SESSION_DRIVER` is set

to file, sessions are stored as files under `storage/framework/sessions`. `SESSION_LIFETIME` is 120 minutes in `.env.example`. In both cases, the Laravel session is the real source of authentication; Pinia only reflects its state in Vue.

7.6 Email Configuration

The example uses SMTP and allows the host, port, username, password, encryption, and sender to be configured. Registration, verification resend, and password recovery depend on this configuration. Controllers catch delivery failures at sensitive points so they can log the error and return controlled messages.

7.7 Database Configuration

`config/database.php` reads `DB_CONNECTION`, `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, and `DB_PASSWORD`. Eloquent, Query Builder, migrations, and seeders use that configuration. Changing `DB_PORT` in `.env` does not require modifying `Review.php` or the Controllers: the connection is managed through configuration.

Explanatory diagram of the configuration flow; it is not a literal code block. Variables are defined in `.env`, and Laravel incorporates them through `config/database.php` before Eloquent, migrations, and seeders use the connection.

```
.env
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=cineverse_laravel

config/database.php
-> mysql connection
-> Eloquent / migrate / db:seed
```

7.8 Cache Configuration and Its Relationship with RateLimiter

`config/cache.php` uses `CACHE_STORE=database` by default in `.env.example`. The database store uses the `cache` and `cache_locks` tables created by migrations. `RateLimiter` relies on Laravel's cache system, so failed login attempts no longer require the `login_attempts` table used by the previous backend.

7.9 CineVerse Custom Log Configuration

`CINEVERSE_LOG_PATH` is interpreted inside `config/logging.php`. It can be an absolute path or a path relative to the project root. The `cineverse_unexpected` channel uses that folder and the base name `cineverse-backend.log`; the daily driver adds the date to the final file.

```
CINEVERSE_LOG_PATH=storage/logs
-> config/logging.php
-> storage/logs/cineverse-backend.log
-> daily driver
-> cineverse-backend-YYYY-MM-DD.log
```

7.10 Guided Example: Carlos Changes Computers and MySQL Uses a Different Port

Suppose the project was originally prepared with MySQL on port 3307, and Carlos installs MySQL on 3306. He does not modify Controllers or Models. He updates `DB_PORT` in `.env`, and Laravel rebuilds the connection from `config/database.php`.

```
Carlos edits .env
DB_PORT=3306
-> Laravel loads the variable
-> config/database.php builds the connection
-> Eloquent and php artisan migrate use the new port
```

8. MySQL Database, Migrations, and Eloquent

MySQL stores what belongs to CineVerse and the infrastructure Laravel requires. It does not store movies or people tables because those resources still come from TMDB. This decision is reflected in reviews: each row stores `movie_id` and `movie_title` to identify the reviewed movie, but there is no foreign key to a local movies table.

8.1 Main Tables

Table	Role
users	Authenticatable account: username, email, verification, password, remember token, and timestamps.
reviews	User reviews of TMDB movies.
sessions	Laravel sessions when <code>SESSION_DRIVER=database</code> .
password_reset_tokens	Password Broker tokens.
cache / cache_locks	Laravel cache; it can also back RateLimiter depending on configuration.
jobs / job_batches / failed_jobs	Queue infrastructure included by the standard migrations.
personal_access_tokens	Sanctum table present even though the main flow uses web sessions.
migrations	Record of migrations already applied.

8.2 users and the Transition from the Previous Structure

The first migration in this version creates users directly with `username`, `email`, `email_verified_at`, `password`, `remember_token`, and timestamps. The later `standardize_users_table_for_laravel_auth` migration preserves compatibility with a legacy database: it adds the standard columns required when they are missing and, if the old columns exist, copies `name` to `username` and `password_hash` to `password`. The following migration removes `password_hash`, `is_verified`, and legacy tokens after the information has been moved to Laravel conventions.

This design explains why the User model uses the casts `password => hashed` and `email_verified_at => datetime`. Laravel Auth and MustVerifyEmail can work with standard columns instead of maintaining parallel logic for `is_verified` and `verification_token`.

8.3 reviews and Its Relationship with users

```
reviews
  id          INT UNSIGNED PK
  user_id     FK -> users.id (cascade on delete)
  movie_id    INT UNSIGNED + index
  movie_title VARCHAR
  rating      TINYINT
  comment     TEXT
  created_at
  updated_at

UNIQUE(user_id, movie_id)
```

The foreign key guarantees that a review always belongs to an existing user. `cascadeOnDelete` means that if an account is permanently deleted, its reviews are deleted as well. The `movie_id` index speeds up listing reviews for a movie. The `unique_user_movie` constraint prevents a duplicate review for the same user/movie combination even if someone bypasses the interface.

8.4 Migration Versus Seeder

A migration defines structure; a seeder loads rows. `php artisan migrate` can run against an empty database and create the tables without loading the 5,000 test accounts. `php artisan db:seed` assumes that the structure already exists. To deliberately rebuild a local test environment, `php artisan migrate:fresh --seed` drops the current tables, reruns the migrations, and then loads the seeders.

8.5 Eloquent

Eloquent is Laravel's ORM. Instead of manually writing `SELECT * FROM reviews WHERE user_id = ?`, `ProfileController` can build `Review::query()->where("user_id", $userId)->count()`. The `Review` model defines fillable and casts and exposes `user()` as `belongsTo(User::class)`. Eloquent does not replace MySQL: it generates queries and transforms results into objects the controllers can use.

8.6 login_attempts No Longer Exists

The migration `2026_07_11_121000_drop_login_attempts_table.php` removes the legacy table. The comment in `down()` makes the decision explicit: Laravel uses `RateLimiter/cache` for login attempts and does not recreate the old table. This is a real architectural difference from the plain PHP backend.

8.7 How Laravel Knows Which Migrations Are Pending

The `migrations` table records which files have already been applied and in which batch. When `php artisan migrate` runs, Laravel orders the migrations by name/timestamp, compares that list with the `migrations` table, and executes `up()` only for pending migrations. It then records each applied migration.

```
php artisan migrate
-> read database/migrations
-> query the migrations table
-> detect pending files
-> execute up()
-> MySQL changes the schema
-> record the applied migration
```

`down()` describes how to reverse a migration when a rollback flow requires it. Not every decision is symmetrical: the migration that removes `login_attempts` deliberately leaves `down()` empty because the current system uses `RateLimiter/cache` and does not want to rebuild that legacy table.

8.8 users: Adapting the Previous Schema to Laravel Conventions

The project contains migrations capable of working with an installation that retained columns from the previous backend. `standardize_users_table_for_laravel_auth` adds `username`, `password`, `email_verified_at`, `remember_token`, and timestamps when they are missing and copies `name/password_hash` if those columns exist. The following migration transfers verification state and removes legacy columns such as `password_hash`, `is_verified`, and manual tokens.

The final result allows `User` to extend `Authenticatable`, implement `MustVerifyEmail`, and use the conventions expected by `Auth`, `Password Broker`, and `Notifications`.

8.9 Review as an Eloquent Model

`Review` defines fillable for fields that may be mass-assigned, casts to convert `ids/rating` to integers and timestamps to dates, and `user()` as a `belongsTo` relationship. That relationship expresses that many reviews can belong to one user.

Location in the project: `app/Models/Review.php` · Eloquent class `App\Models\Review`. `ReviewController` and `ProfileController` use it to query and persist reviews.

```
class Review extends Model {
    protected $fillable = [
        "user_id", "movie_id", "movie_title", "rating", "comment"
    ];
}
```

```
public function user() {
    return $this->belongsTo(User::class);
}
}
```

8.10 Example: Review::create Ends in MySQL

When `ReviewController` calls `Review::create()`, Eloquent takes the fields allowed by `fillable`, prepares the INSERT, and uses the configured MySQL connection. The `UNIQUE(user_id, movie_id)` constraint still exists in the database even though the Controller has already checked for a duplicate.

```
ReviewController::store()
-> $request->validate(...)
-> check exists()
-> Review::create([...])
-> Eloquent builds INSERT
-> MySQL validates FK + UNIQUE
-> row persisted
```

9. Test Data and Seeders

In Laravel, a seeder is a class that automatically loads records into tables that already exist. CineVerse uses seeders to prepare a large test dataset so reviews and profiles can be explored without manually creating thousands of records. Instead of hard-coding every record inside a PHP class, the seeders read NDJSON files located under `database/seeders/data`; each line contains an independent JSON object.

9.1 What a Seeder Does and How It Relates to Migrations

A migration and a seeder solve different problems. A migration defines or changes the database structure: it creates tables, columns, indexes, and constraints. A seeder works afterward on that existing structure and adds prepared records.

In CineVerse, `DatabaseSeeder` acts as the entry point and coordinates the project's specific seeders. Users are loaded first and reviews afterward because `reviews.user_id` depends on `users.id`.

9.2 DatabaseSeeder

```
DatabaseSeeder
-> UserSeeder
-> ReviewSeeder
```

The order matters because `reviews.user_id` has a foreign key. `ReviewSeeder` explicitly checks that 5,000 users exist before inserting reviews.

9.3 Expected Totals

Dataset	Expected Count	Batch Size
users.ndjson	5,000	500
reviews.ndjson	24,322	250

Each seeder counts the lines actually processed. If the total does not match `EXPECTED_COUNT`, it throws `RuntimeException` instead of presenting an incomplete dataset as valid.

9.4 Empty-Table Requirement

`UserSeeder` requires users to be empty, and `ReviewSeeder` requires reviews to be empty. This prevents inserting a second copy of the same scenario on top of existing data. When a test environment needs to be rebuilt, the error message points to `php artisan migrate:fresh --seed`.

9.5 Why NDJSON

NDJSON makes it possible to read the file line by line with `fgets` and process batches without loading the entire dataset into memory. `UserSeeder` accumulates 500 rows and `ReviewSeeder` 250 before each insert. For tens of thousands of reviews, this approach is more predictable than decoding one large JSON array at once.

9.6 Reproducible Test Data, Not a Production Migration

The seeders prepare a local test environment. They do not describe an operation that should be repeated on real data. `migrate:fresh --seed` destroys the existing structure, so it is appropriate only when that destruction is deliberate and the data does not need to be preserved.

9.7 Guided Example: Carlos Rebuilds a Local Test Environment

Carlos wants to return to a clean scenario without keeping previous data. He runs `migrate:fresh --seed`. Laravel drops the tables in that database, reruns all migrations, and then calls `DatabaseSeeder`. When the process finishes, the schema and dataset are in the state expected by the documentation.

`migrate:fresh --seed` is destructive. This document describes it for a test environment whose information can be deliberately deleted, not as a routine operation on real data.

10. How a User Action Flows Through CineVerse

Following complete actions avoids studying isolated files without understanding why they exist. The next three examples show the most representative flows: a TMDB-only query, a login that uses the Laravel session, and a review that combines a TMDB movie with application-owned data in MySQL.

10.1 Open a Movie

This flow uses `Promise.all()`. A Promise represents an asynchronous operation whose result will arrive later; `Promise.all()` makes it possible to start several of those operations and continue when all of them have finished. CineVerse uses it to wait for credits, videos, and similar movies together.

```
User clicks Fight Club
-> router.push({ name: "MovieDetail", id: 550 })
-> MovieDetailView.vue
-> useMovieDetail(550, language)
-> Promise.all(credits, videos, similar)
-> tmdbApi.ts
-> TMDB
-> Vue Query cache
-> MovieHero / MovieTabs / etc.
```

Laravel does not query the database to produce a synopsis or poster. Navigation is already occurring inside Vue Router, and the services communicate directly with TMDB. If only the language changes, the query key includes language and Vue Query can manage a different localized result.

10.2 Sign In

```
LoginView
-> loginUser({ email, password })
-> webApi.post("/login")
-> routes/auth.php
-> AuthenticatedSessionController::store
-> LoginRequest::authenticate
-> RateLimiter + User + Hash::check
-> Auth::login
-> session()->regenerate()
-> 204
-> GET /user
-> user + csrf_token
```

```
-> useAuthStore
-> header / guards / profile react
```

Pinia does not authenticate users by itself. The actual session exists in Laravel. The store keeps a reactive copy of the user so the interface knows what to display. If the page is reloaded, `initialize()` queries `/user` again and reconstructs that representation from the session cookie.

10.3 Create a Review

In `createReview(payload, csrfToken)`, `payload` is simply the object that groups the data to be sent to Laravel: `movie_id`, `movie_title`, `rating`, and `comment`. The CSRF token travels separately in a protective HTTP header.

```
MovieReviews
-> createReview(payload, csrfToken)
-> POST /api/reviews + X-CSRF-Token
-> web + auth middleware
-> ReviewController::store
-> validate movie_id/title/rating/comment
-> check duplicate user_id + movie_id
-> Review::create()
-> MySQL
-> REVIEW_CREATED
-> MovieReviews reloads statistics/list
```

The interface limits the comment and displays the appropriate form, but the backend validates the data again. The `unique_user_movie` constraint in MySQL adds a third layer for the “one review per user and movie” rule. After saving, the frontend does not invent the average locally: it requests the data again and displays the state confirmed by the server.

11. TypeScript in CineVerse

TypeScript is an extension of JavaScript that makes it possible to specify what type of data is expected in each part of the code. For example, CineVerse can declare that a movie id is a number, the title is text, and `poster_path` can be an image path or null when TMDB does not provide a poster. These checks help detect errors during development, before the application is opened in the browser.

In CineVerse, types also describe HTTP responses, users, reviews, filters, component props, and values returned by composables. The project enables `strict: true`, so TypeScript performs stricter checks than a basic configuration.

11.1 Strict Configuration

Location in the project: `tsconfig.json` · project TypeScript configuration. It is read by the compiler/`vue-tsc` during development and build.

```
{
  "compilerOptions": {
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true,
    "noUncheckedSideEffectImports": true
  }
}
```

With these options, the compiler can warn, for example, that code is trying to read a property that does not exist, that a value could be null, that a string was passed where a number was expected, or that a variable was declared but never used.

11.2 Domain Types

Location in the project: `resources/js/cineverse/features/movies/types/movie.ts` · Movie interface. It is used by `movies.service.ts`, `composables`, and `components` that work with TMDB movies.

```
export interface Movie {
  id: number;
  title: string;
  release_date: string;
  poster_path: string | null;
  backdrop_path: string | null;
  vote_average: number;
  popularity: number;
  genre_ids?: number[];
  overview: string;
}
```

interface is one of the forms TypeScript provides for describing an object's structure. In `Movie`, `poster_path: string | null` indicates that the value can be a text path or can be absent. The interface must account for both cases instead of assuming every movie has a poster.

11.3 Generics: `PaginatedResponse<T>`

Locations in the project: `resources/js/cineverse/shared/types/paginated-response.ts` defines `PaginatedResponse<T>`; `resources/js/cineverse/features/movies/services/movies.service.ts` uses it to type responses such as `/search/movie`.

```
export interface PaginatedResponse<T> {
  page: number;
  results: T[];
  total_pages: number;
  total_results: number;
}

const { data } = await api.get<PaginatedResponse<Movie>>(<
  "/search/movie",
  { params: { query, page, language } }
  >);
```

`PaginatedResponse<T>` is a generic type: `T` represents the specific type of elements contained in results. When `PaginatedResponse<Movie>` is used, TypeScript knows that `results` is `Movie[]` and that `page`, `total_pages`, and `total_results` are numbers. The same pagination structure can be reused with other types without copying it.

11.4 Reusing Types in Filters

Locations in the project: `resources/js/cineverse/features/movies/types/base-filters.ts` · `BaseFilters`; `discover-filters.ts` · `DiscoverFilters`; `search-with-text-filters.ts` · `SearchWithTextAndFilters`. They are used by `movies.service.ts` and `useMoviesQuery.ts`.

```
export interface BaseFilters {
  genres?: number[];
  yearFrom?: number;
  yearTo?: number;
  minRating?: number;
  maxRating?: number;
  sortBy?: string;
}

export interface DiscoverFilters extends BaseFilters {}

export interface SearchWithTextAndFilters extends BaseFilters {
  query: string;
}
```

BaseFilters groups the properties shared by several search types. DiscoverFilters reuses those properties, and SearchWithTextAndFilters adds query. This defines genres, years, ratings, and sort order once instead of duplicating them in every type.

11.5 Deriving a Type from a Composable

Location in the project: resources/js/cineverse/features/movies/composables/useMoviesQuery.ts ·

Composable · MovieFilters type and useMoviesQuery() function. The type is derived from useMoviesFilters.ts; the shown block is an abbreviated fragment.

```
export type MovieFilters = ReturnType<typeof useMoviesFilters>;

export function useMoviesQuery(
  filters: MovieFilters,
  language: Ref<string>
) { ... }
```

ReturnType is a TypeScript utility that automatically obtains the type returned by a function. MovieFilters is derived from useMoviesFilters; if that composable changes what it returns, the type updates with it and reduces the chance that the two definitions drift apart.

The same example includes Ref<string>. Ref is a Vue reactive container: it stores a value that can change and allows Vue to detect that change. Ref<string> indicates that the reactive value contains text; in useMoviesQuery, it represents the active language.

11.6 TypeScript Does Not Replace Backend Validation

Declaring rating: number in TypeScript protects the code during development, but it does not turn the browser into a security barrier. Someone can manually send rating = 999 without using the interface. ReviewController therefore validates rating again with integer|min:1|max:5 and comment with max:1000. The migration uses tinyInteger for rating but does not declare an SQL CHECK constraint; the 1–5 rule is currently enforced by Laravel and the interface.

Each layer has a different role
TypeScript helps detect errors while code is being written. Laravel validates the data that actually arrives over HTTP again through Request, Form Requests, or controller rules. Eloquent builds queries against MySQL, and UNIQUE and FOREIGN KEY constraints preserve structural database rules. For reviews, the 1–5 range is validated in Laravel; the migration uses tinyInteger but does not declare an SQL CHECK.

11.7 Areas with Less Strict Typing

The project uses a strict configuration, but some areas are still less strongly typed. There are Record<string, any> values, errors caught as any, and some credit arrays typed as any[]. any means TypeScript stops checking that value precisely. It is also important to remember that an Axios type describes what the code expects to receive, but it does not validate at runtime that TMDB or the backend actually sent exactly that structure.

11.8 Typed Props and Events Between Components

TypeScript also protects communication between Vue components. Props are values a parent component passes to a child. Events provide the reverse path: the child signals that something happened and the parent decides what to do.

Location in the project: `resources/js/cineverse/features/movies/components/movie-detail/MovieHero.vue` · Vue Component · `defineProps()/defineEmits()`. `MovieDetailView.vue` uses it as a child component through props and events.

```
// MovieHero receives data from the parent
defineProps<{
  movie: MovieDetail;
  cast: CastMember[];
  director: { name: string } | null;
  hasTrailer: boolean;
}>();

// MovieHero notifies the parent
const emit = defineEmits(["openTrailer", "ready"]);
```

With these definitions, TypeScript can warn if the parent attempts to pass the wrong type. In `MovieDetailView`, `openTrailer` is connected to the event emitted by `MovieHero`, and `ready` is used to remove the loading state when the hero has finished preparing its image.

11.9 Backend Response Types

The same principle is used with the Laravel API. `reviews.service.ts` declares that `getMovieReviews()` returns `ReviewsResponse` and that `getUserReviews()` returns `UserReviewsResponse`. This allows `MovieReviews` to know that data contains stats, pagination, `user_review`, and reviews without working with completely unknown objects.

The type helps during development, but it still does not validate the actual JSON at runtime. If a server returned a different structure, TypeScript could not stop that network response; error states and backend validation are therefore still necessary.

12. How the Vue Frontend Is Organized

`resources/js/app.ts` is the actual frontend entry point inside Laravel. It starts Inertia with `createInertiaApp()`, resolves `Pages/CineVerse.vue`, creates the Vue application, and registers Pinia, Vue Router, Vue Query, `vue-i18n`, and Ziggy. In the Laravel version there is no separate `main.ts`: Vite enters through `app.ts`, and the complete `CineVerse` application is mounted from there.

Main Mounting Flow

Location in the project: `resources/js/app.ts` · Vue/Inertia entry point · `createInertiaApp()`. The fragment is abbreviated to show page resolution and plugin registration; `Pages/CineVerse.vue` then mounts the `CineVerse` application.

```
createInertiaApp({
  resolve: name => resolvePageComponent(
    `./Pages/${name}.vue`,
    import.meta.glob("./Pages/**/*.vue")
  ),
  setup({ el, App, props, plugin }) {
    const pinia = createPinia();
    pinia.use(piniaPluginPersistedState);

    const app = createApp({ render: () => h(App, props) });

    app
      .use(plugin)
      .use(pinia)
      .use(router)
      .use(VueQueryPlugin)
      .use(i18n)
      .use(ZiggyVue);

    useAuthStore().initialize();

    router.isReady().then(() => app.mount(el));
```

```
}
});
```

12.1 Checking the Session When the Application Starts

At startup, `useAuthStore.initialize()` calls `GET /user` to determine whether the browser already has a valid session. `app.ts` starts that check without blocking public routes. Vue Router guards wait for the same Promise only when the route needs to know whether a session exists, such as `Profile` or `Login`.

Advantage

Home and Movies are not blocked by a call to `/user`. Routes that do need to know the session state wait for the result selectively.

As seen in the movie flow, a Promise represents an asynchronous operation whose result will arrive later. Here, the implementation reuses a single Promise to prevent `app.ts` and a Vue Router guard from sending two `GET /user` requests at the same time. `initializePromise` remains stored even after it resolves; therefore, if the first request failed because of a temporary backend outage, another call during the same SPA lifetime would reuse that already-settled promise instead of automatically issuing a new request.

Current integration detail. `app.ts` waits for `router.isReady()` before mounting Vue. This allows guest-only route guards to redirect an already-authenticated user before the first render and avoids briefly showing the login or registration form. The root document is no longer an `index.html` from an independent frontend: Laravel uses `resources/views/app.blade.php` as Inertia's base view.

12.2 Organizing Files by Features

The `features` folder groups files by functionality. `auth` contains registration, login, and recovery; `movies` contains exploration, filters, movie details, reviews, and favorites; `person` contains actor information; and `profile` contains profile screens. `core` is reserved for shared code used by several features, such as Axios, the router, and languages.

12.3 Components and Composables

A view represents a complete screen. Components split that screen into smaller parts and receive data or emit events. Composables are functions that group reusable Vue logic: `useMoviesFilters` manages filters, `useMoviesQuery` loads explorer pages, `useMovieDetail` combines movie data, and `usePersonDetail` obtains actor information.

12.3.1 How to Recognize a View, Component, Composable, and Service in the Code

The `resources/js/cineverse` folder is organized by features. The same flow usually passes through several file types, and each solves a different part of the problem.

Location in the Project	Type	Responsibility in the Example
<code>resources/js/cineverse/features/movies/views/MoviesView.vue</code>	View	Coordinates explorer filters, query, and grid.
<code>resources/js/cineverse/features/movies/components/MovieResults.vue</code>	Component	Receives pages and builds the visible <code>MovieCard</code> list.
<code>resources/js/cineverse/features/movies/composables/useMoviesQuery.ts</code>	Composable	Decides which query to run and configures <code>useInfiniteQuery</code> .
<code>resources/js/cineverse/features/movies/services/movies.service.ts</code>	Service	Builds the specific calls to TMDB endpoints.

Location in the Project	Type	Responsibility in the Example
resources/js/cineverse/features/movies/stores/useMoviesFavoritesStore.ts	Store	Maintains and persists the shared favorites list.

```

MoviesView
-> useMoviesFilters()
-> useMoviesQuery()
-> movies.service.ts
-> tmdbApi.ts
-> TMDB
-> Vue Query returns data
-> MovieResults
-> MovieCard

```

The arrow does not mean that every file imports every other file directly. It represents the conceptual data flow: the view coordinates, the composable decides, the service requests, and the components render.

12.4 How Vue Reactivity Works in CineVerse

Vue keeps values reactive so the interface updates when data changes without manually rebuilding the HTML. CineVerse mainly uses `ref`, `computed`, `reactive`, and `watch`.

- `ref(value)` stores a reactive value. Example: `MovieReviews` uses `formRating`, `formComment`, `isSubmitting`, and `reviews` as refs.
- `computed(...)` calculates a value from other reactive values. `favoritesCount` is recalculated from `favorites.length`, and `canSubmitForm` depends on the rating and comment length.
- `reactive({...})` groups several reactive properties in an object; `useMoviesFilters` uses it for `listType`, `text`, `genres`, `years`, `rating`, and `sort order`.
- `watch(...)` runs logic when a value changes. CineVerse uses it, for example, to wait briefly after the user stops typing before searching—a technique called `debounce`—to lock scrolling when a modal opens, or to reset the grid when filters change.

For example, after `favorites.value.push(movie)`, there is no need to find the counter in the DOM and modify it. `favoritesCount` depends on `favorites`, Vue detects the change, and template expressions that use that counter are reevaluated automatically.

12.5 Parent Components, Props, and Emits

Components do not directly share local variables. The parent passes data through props, and the child can emit events. `MovieTabs` receives `cast`, `similar`, `movieId`, and `movieTitle` from `MovieDetailView`; it then passes `movieId` and `movieTitle` to `MovieReviews`. `MovieHero`, by contrast, emits `openTrailer` so `MovieDetailView` is the component that opens the modal.

```

MovieDetailView
-> props -> MovieHero
<- emit "openTrailer" - MovieHero

MovieDetailView
-> props movieId/movieTitle -> MovieTabs
-> props movieId/movieTitle -> MovieReviews

```

12.6 Lifecycle: onMounted and onUnmounted

`onMounted` runs after the component has been created and inserted into the DOM. `MovieReviews` uses it to request the first page of reviews. `ProfileView` uses it to load the profile. `MovieDetailView` registers the `Escape`-key listener there to close the trailer.

`onUnmounted` runs when the component is destroyed. `MovieDetailView` removes the global keyboard listener, and `MovieResults` disconnects its `IntersectionObserver`. This cleanup prevents functions from remaining active after leaving a screen.

12.7 App.vue, RouterView, and When a View Is Recreated

App.vue keeps AppHeader and AppFooter outside RouterView so the overall frame remains while the central screen changes. RouterView receives the component associated with the current URL.

routeKey(route) builds a key from the route name and its parameters, but excludes lang. Therefore, moving from /es/movies/550 to /es/movies/272 changes the id and forces the view to be recreated, while changing only /es to /en preserves the instance and allows language reactivity and Vue Query to update the data.

12.8 Frontend Forms: This Project Does Not Use Inertia's useForm()

Although Inertia is part of the integration, CineVerse's visible forms are not built with useForm() from @inertiajs/vue3. Login, registration, recovery, and reviews keep their state with ref/computed/watch, validate in Vue, and send requests through TypeScript services and Axios.

```
LoginView.vue
-> refs: email / password / visual states
-> loginUser(payload)
-> auth.service.ts
-> webApi.post("/login", payload)
-> Laravel
```

For that reason, it would not make sense to document useForm() as if it were part of this version's actual behavior. Inertia delivers the application and shares context, but CineVerse forms retain the Vue + Axios approach used by the integrated frontend.

13. How Data Is Stored and Reused in the Frontend

Three different concepts appear in the frontend. State is information the application needs to remember while it is running, such as which user signed in. Cache is a temporary copy of data already obtained that can be reused to avoid unnecessary requests. Persistence means retaining certain data even if the page is reloaded or the user returns later.

CineVerse uses different tools for each need: Pinia maintains shared state, Vue Query manages data obtained through HTTP requests, and localStorage or sessionStorage preserves certain values within the browser.

13.1 Pinia: Authentication

The Pinia authentication store keeps the current user, the CSRF token, whether an authenticated session exists, and whether the initial check has completed. Because the store is available from different screens, the header, router, and profile can read the same state without duplicating it.

13.1.1 How Pinia Is Installed and Why a Store Is Shared

app.ts creates a single Pinia instance with createPinia() and registers pinia-plugin-persistedstate. That instance is installed inside the real setup() of createInertiaApp(), on the same Vue application that also receives Inertia, Vue Router, Vue Query, vue-i18n, and Ziggy. From that point on, any component can call useAuthStore() or useFavoritesStore() and work with the same shared state instead of maintaining independent copies.

Location in the project: resources/js/app.ts · Vue/Inertia entry point · createInertiaApp() setup(). This code creates the shared Pinia instance and installs pinia-plugin-persistedstate before the application is mounted.

```
const pinia = createPinia();
pinia.use(piniaPluginPersistedState);
```

Pinia does not mean permanent storage by itself. The authentication store lives in memory and is reconstructed by calling GET /user. The favorites store, by contrast, has persist: true, so pinia-plugin-persistedstate can keep it in localStorage.

13.1.2 Inside the Authentication Store

`useAuthStore` maintains `user`, `csrfToken`, and `isInitialized`. `isAuthenticated` is a computed value that is true when `user` is not null. After a successful login, `LoginView` calls `setUser()` and `setCsrfToken()`; when the user signs out or the session is found to be invalid, `clear()` sets both values back to null.

Summary diagram of the `useAuthStore` state; it is not a literal block from the file. The actual implementation is in `resources/js/cineverse/features/auth/stores/useAuthStore.ts` and is consumed by `app.ts`, guards, and authenticated views.

```
user: AuthUser | null
csrfToken: string | null
isAuthenticated = computed(() => user !== null)
isInitialized: boolean

successful login
-> setUser(response.data.user)
-> setCsrfToken(response.data.csrf_token)
```

`initialize()` is special: it calls `GET /user` to reconstruct state from the session cookie. It keeps a shared `initializePromise` so `app.ts` and a guard do not issue two requests at the same time. The current implementation retains that `Promise` even after it finishes; if the first request failed because of a temporary outage, a later call during the same SPA lifetime does not automatically force another attempt.

13.2 Pinia: Favorites

The favorites store keeps a list of `Movie` objects, prevents duplicates, and limits the collection to 20. `pinia-plugin-persistedstate` stores that state in `localStorage`, allowing favorites to survive a browser reload. They are not sent to the backend or associated with the account.

13.3 Vue Query

Home, Movies, movie detail, and actor screens use `TanStack Vue Query` to request data and control states such as loading, error, and available data. Each query has a `queryKey`, a key that identifies what information it represents. When important values such as the `id`, `language`, or filters change, the key changes and `Vue Query` knows it must treat the result as a different query.

Current implementation detail. In `useMoviesQuery.ts`, the `enabled` option ends with `|| true`. As a result, the expression is always true and the query remains enabled in every case. The conditions written before that operator do not determine the final value of `enabled` in this version.

Area	Observed Strategy
Home	Three independent queries. Data is considered fresh for 5 minutes, and previous data is temporarily retained when the language changes.
Explorer	Paginated query with <code>useInfiniteQuery</code> . A fresh load is forced when the explorer is mounted again.
Movie Details	Query by <code>id</code> and <code>language</code> . It does not retry automatically and fetches credits, videos, and similar movies after the main detail request.
Actor	Query by <code>id</code> and <code>language</code> . It does not retry automatically.

13.3.1 What Vue Query Actually Stores

`Vue Query` does not replace `Pinia`. It is used for data obtained through requests: `TMDB` movies, actor details, and Home results. A `queryKey` identifies the result. If an important part of that key changes—for example the `id`, `language`, or a filter—`Vue Query` treats it as a different query.

On Home, data is considered fresh for five minutes. In the explorer, `stateTime` and `gcTime` are 0, so the behavior favors fresher data and does not retain the cache after it becomes unused. On movie and actor details, the key includes the id and language.

13.3.2 queryKey and queryFn: Query Identity and Work

A Vue Query query has two central ideas. `queryKey` identifies what information it represents; `queryFn` is the function that obtains that information when needed.

Teaching pseudocode; this is not a literal block. The actual implementation is in `resources/js/cineverse/features/movies/composables/useMoviesQuery.ts`, where `useInfiniteQuery` builds `queryKey` and `queryFn` from `useMoviesFilters`.

```
queryKey: ["movies", listType, debouncedQuery, genres, yearFrom, yearTo, minRating, maxRating, sortBy, language]

queryFn: ({ pageParam }) => {
  // decides between search/movie, discover/movie, or movie/{listType}
}
```

If an important part of `queryKey` changes—for example the language, Batman to Matrix, or a genre—Vue Query stops treating the result as the same query. This allows the explorer to react to filters without `MovieResults` needing to know how the request was built.

13.3.3 useQuery and useInfiniteQuery Do Not Solve the Same Problem

Tool	Where It Is Used	What It Provides
<code>useQuery</code>	Home, movie details, person details	One query identified by a key. Exposes data, loading/error states, and cache options.
<code>useInfiniteQuery</code>	Movie explorer	Keeps multiple pages under a single query and exposes <code>fetchNextPage</code> / <code>hasNextPage</code> .

13.3.4 States a View Receives from Vue Query

Value	Practical Meaning
<code>data</code>	Confirmed data returned by the query.
<code>isLoading</code> / <code>isPending</code>	The initial load has not yet produced usable data.
<code>isError</code> / <code>error</code>	The operation ended with an error, and the view can display a controlled state.
<code>fetchNextPage()</code>	Requests the next page in an infinite query.
<code>hasNextPage</code>	Indicates whether <code>getNextPageParam</code> found another available page.
<code>isFetchingNextPage</code>	Prevents multiple page loads from being shown or triggered simultaneously.

13.3.5 Pinia, Vue Query, and Axios: Three Different Responsibilities

Tool	Question It Answers	CineVerse Example
Pinia	What application-owned shared state do I need to remember?	Authenticated user and favorites.
Vue Query	What remote data did I retrieve, and what state is the query in?	Movies, people, and result pages.
Axios	How do I send the HTTP request?	<code>tmdbApi.ts</code> and <code>webApi.ts</code> .
<code>localStorage</code> / <code>sessionStorage</code>	What data do I want to keep in the browser?	Favorites, visual preferences, and the latest filter state.

Axios does not replace Vue Query, and Vue Query does not replace Pinia. Axios transports; Vue Query manages remote data; Pinia maintains shared application state.

13.4 sessionStorage and localStorage

Data	Where It Is Stored	How Long It Remains
Latest filter query	sessionStorage	Isolated per tab; allows returning to Movies with a clean URL.
Favorites	Pinia persistence / localStorage	Survives reloads and new browser sessions.
Dismissed Firefox banner	localStorage	Does not appear again after being closed.
Panel order/visual state	localStorage	Restored between visits according to the filter component.

sessionStorage and localStorage are storage mechanisms provided by the browser. sessionStorage lasts for the current tab; localStorage can remain after the site is closed and reopened. CineVerse uses sessionStorage to remember filters and avoid repeating Home's full skeleton within the same tab, and localStorage for favorites and visual preferences.

14. Navigation Without Page Reloads: SPA and Vue Router

Because CineVerse is an SPA, moving between sections does not require downloading a complete HTML page from the server every time. Vue Router decides which view should be displayed based on the URL, and Vue replaces the corresponding content inside the already-loaded application.

A route is the relationship between a URL and a screen. For example, /es/movies displays the explorer and /es/movies/550 displays the details of the movie whose id is 550. CineVerse uses createWebHistory, so URLs look like normal addresses and do not use the # symbol.

The language is also part of the route: functional sections begin with /es or /en. This makes it possible to share a URL that preserves both the screen and the selected language.

Route	View / Purpose
/:lang	HomeView
/:lang/movies	Movie explorer
/:lang/movies/:id	Movie details; numeric id only
/:lang/person/:id	Actor details; numeric id only
/:lang/favorites	Local favorites
/:lang/profile	Profile; requiresAuth
/:lang/profile/reviews	My reviews; requiresAuth
/:lang/login	Login; requiresGuest
/:lang/register	Registration; requiresGuest
/:lang/forgot-password	Recovery; requiresGuest
/:lang/reset-password	New-password form

14.1 Initial Language Detection

On a direct site load, the root / reaches Laravel first. routes/web.php redirects / to /es, so the observable server behavior is to start in Spanish. Once the SPA is mounted, Vue Router controls /:lang/... routes and the global

guard synchronizes the lang parameter with vue-i18n. The root route defined inside Vue Router retains browser-language detection logic, but a normal HTTP entry at / has already been resolved by Laravel before that point.

```
Direct load at /
-> Laravel routes/web.php
-> redirect("/es")
-> Inertia delivers CineVerse
-> Vue Router resolves /es
-> vue-i18n locale = "es"
```

14.2 Navigation Guards

A guard is a function that runs before a navigation is completed. `requiresAuth` prevents access to the profile or the user's own reviews without a valid session. `requiresGuest` does the opposite: it prevents Login, Register, and ForgotPassword from being shown to someone who is already signed in. Both wait for `initialize()` only when they need to know the authentication state.

14.3 What Happens When a Route Does Not Exist: 404 Error

A 404 route is displayed when the URL does not match any valid screen. CineVerse has catch-all routes that preserve /es or /en whenever possible. Detail routes accept only numeric ids, so /es/movies/abc ends in `NotFoundView` without sending an unnecessary request to TMDB.

14.4 createWebHistory, RouterView, and routeKey

`createWebHistory(import.meta.env.BASE_URL)` makes CineVerse use normal URLs without #. Vue Router observes the URL and `RouterView` displays the associated view. In the Laravel integration, directly reloading /es/movies/550 does not depend on a fallback to index.html: the localized catch-all in routes/web.php returns `Inertia::render("CineVerse")`, and once the app is mounted, Vue Router resolves `MovieDetailView`.

`App.vue` adds a second decision through `routeKey()`: changing the id recreates the view; changing only the language does not. This avoids retaining data from another movie when moving from /movies/550 to /movies/272, while allowing ES/EN changes without destroying the component's entire visual state.

14.5 Complete Example of Navigating to Another Movie

```
User clicks a MovieCard with id 550
-> MovieCard.goToMovie(550)
-> router.push({ name: "MovieDetail", lang, id: 550 })
-> beforeEach synchronizes the language
-> Router resolves /es/movies/550
-> RouterView receives MovieDetailView
-> routeKey includes id=550
-> Vue mounts the view
-> useMovieDetail(550, locale)
-> TMDB
```

`scrollBehavior()` returns `{ top: 0 }`, so navigation begins at the top of the new screen instead of preserving the vertical position of the previous view.

15. Languages: vue-i18n and Language Switching

i18n is an abbreviation used in development for internationalization. CineVerse uses vue-i18n to maintain interface text in Spanish and English without duplicating each component's logic. Translations are stored in `resources/js/cineverse/core/i18n/es.json` and `resources/js/cineverse/core/i18n/en.json`.

Location in the project: `resources/js/cineverse/core/i18n/index.ts` · global `vue-i18n` configuration module. The exported instance is installed from `resources/js/app.ts` and used by components through `useI18n()`.

```
const i18n = createI18n({
  legacy: false,
  locale: "es",
  fallbackLocale: "en",
  messages: { en, es },
});
```

`legacy: false` makes `vue-i18n` use its modern Vue 3 API. `locale` indicates the active language, and `fallbackLocale` defines a fallback language. Components do not write the same phrase twice: they request a key such as `t("nav.movies")`, and `vue-i18n` looks up the corresponding text in the active dictionary.

15.1 What Happens When Switching from Spanish to English

The language is part of the URL. If the user is at `/es/movies/550` and clicks EN, `AppHeader` runs `switchLang("en")` and `Vue Router` navigates to the same route with `lang="en"`. Before navigation completes, the global guard reads `to.params.lang` and assigns that value to `i18n.global.locale.value`. From then on, `t(...)` keys resolve against `en.json`.

Location in the project: `resources/js/cineverse/core/router/index.ts` · global router · `beforeEach()`. `Vue Router` uses it on every navigation to synchronize `:lang` with `vue-i18n` and apply guards.

```
router.beforeEach(async (to) => {
  const lang = to.params.lang as string;

  if (lang === "en" || lang === "es") {
    i18n.global.locale.value = lang;
  }

  // ... authentication guards
});
```

Changing the language affects more than interface buttons and headings. Home, Movies, movie detail, and actor detail include the language in their queries, so `TMDB` receives the corresponding language value. `useLocalizedDate` uses `es-AR` or `en-US` to determine the visual day/month order through `Intl.DateTimeFormat`.

Two implementation details are worth noting. `useLocalizedDate` first converts the received text with `Date(dateStr.replace(" ", "T"))`; when the source is a date without a time, `Date` interpretation and the local time zone can shift the displayed day. In addition, `PersonHero.vue` keeps the strings "Leer más" and "Leer menos" hard-coded in Spanish, so those two labels do not change when EN is selected.

The language is also sent to `Laravel` during registration, verification resend, and password-recovery flows. The `CineVerseVerifyEmail` and `CineVerseResetPassword` notifications read that value and select the Blade views `resources/views/emails/verify-account-es.blade.php`, `verify-account-en.blade.php`, `reset-password-es.blade.php`, or `reset-password-en.blade.php`.

Current edge case: `switchLang` preserves route parameters such as a movie id, but does not copy `route.query`. `route.query` contains the parameters that appear after `?` in a URL. Therefore, if the user changes language from an explorer URL containing search text, genres, years, or other filters, those parameters are lost during the current navigation.

15.1.1 How the Language Reaches TMDB and Emails

The language is not decided independently in every part of the system. `Vue Router` and `vue-i18n` maintain a single active language, and different functions reuse that value. It is therefore useful to follow the complete path from the URL to queries and emails.

```

URL /es or /en
  |
  v
Vue Router synchronizes lang
  |
  v
vue-i18n keeps locale = "es" or "en"
  |
  +--> MoviesView uses locale as apiLanguage
  |
  |   v
  |   useMoviesQuery -> TMDB receives language
  |
  +--> RegisterView / LoginView / ForgotPasswordView
  |
  |   v
  |   language payload
  |
  |   v
  |   Laravel/Breeze routes
  |
  |   +--> CineVerseVerifyEmail
  |   |       -> verify-account-{language}.blade.php
  |   |       -> signed URL with lang
  |   |
  |   +--> CineVerseResetPassword
  |   |       -> reset-password-{language}.blade.php

```

In MoviesView, `const { locale } = useI18n()` obtains the current language and `apiLanguage` points to that same reactive value. `useMoviesQuery` includes `language.value` in its `queryKey`. Therefore, when the user switches from `/es` to `/en`, Vue Query treats the query as changed and requests the data again in the new language.

Something similar happens in forms that can generate an email. `RegisterView` takes `locale.value` and sends it as `language` in the object posted to `/register`. `ForgotPasswordView` sends it to `/forgot-password`, and `LoginView` uses it when requesting that the verification email be resent.

Location in the project: `resources/js/cineverse/features/auth/views/RegisterView.vue` · Vue View · registration submit. The view reads `locale` and calls `registerUser()` from `auth.service.ts` with `language` in the payload.

```

const lang = locale.value as "es" | "en";

await registerUser({
  username,
  email,
  password,
  password_confirmation,
  language: lang
});

```

Laravel does not have a custom `AuthService` or `MailService` for this task. `RegisteredUserController` and `PasswordResetLinkController` retain language in the request; when User triggers `CineVerseVerifyEmail` or `CineVerseResetPassword`, each notification reads `request()->input("language", app()->getLocale())`, accepts only `es` or `en`, and uses `en` as a fallback. Verification also includes `lang` in the signed URL so returning to the SPA preserves the language. The complete registration, verification, and Password Broker flow is covered in section 26.

15.1.2 Guided Example: Carlos Switches the Menu from Spanish to English

Carlos is using CineVerse in Spanish. AppHeader displays Inicio, Películas, and Favoritos. When he clicks EN, the button does not manually replace words: it runs `switchLang("en")` and asks Vue Router to navigate to the same route with a different `lang` value.



Figure. CineVerse header before the language change, with ES active.

Location in the project: resources/js/cineverse/layouts/components/AppHeader.vue · Component · language selector and switchLang() function.

```
function switchLang(lang: string) {
  router.push({
    name: route.name as string,
    params: { ...route.params, lang },
  });
}
```

Carlos clicks EN

```
-> AppHeader runs switchLang("en")
-> /es/... becomes /en/...
-> router.beforeEach reads lang="en"
-> i18n.global.locale.value = "en"
-> t(...) resolves again
-> Vue updates the visible text
```

*Figure. The same header after switching to EN.*

15.1.3 How t("nav.movies") Ends Up Displaying Películas or Movies

Components request a key, not a fixed phrase. AppHeader uses t("nav.movies"); vue-i18n looks up that key in the active dictionary. Location in the project: resources/js/cineverse/core/i18n/es.json and resources/js/cineverse/core/i18n/en.json · translation dictionaries. The following block shows only a partial nav fragment to illustrate the same key in both languages; the real objects contain more keys.

```
// es.json
"nav": {
  "home": "Inicio",
  "movies": "Películas",
  "favorites": "Favoritos"
}

// en.json
"nav": {
  "home": "Home",
  "movies": "Movies",
  "favorites": "Favorites"
}

t("nav.movies")
-> locale="es" -> es.json -> "Películas"
-> locale="en" -> en.json -> "Movies"
```

15.1.4 Guided Example: Carlos Changes the Language of Matt Damon's Profile

A person's biography does not come from en.json. Carlos opens the profile with ES active; PersonDetailView passes the id and locale to usePersonDetail(). When he switches to EN, the language is part of the query identity and TMDB receives another localized request.

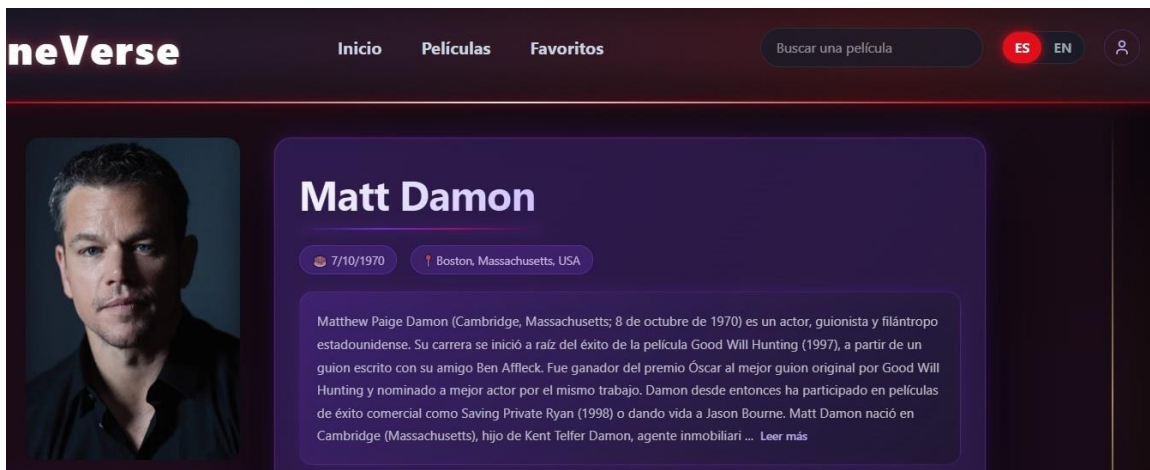


Figure. Matt Damon profile with the interface and biography in Spanish.

Location in the project: resources/js/cineverse/features/person/composables/usePersonDetail.ts · Composable · query identified by person and language.

```
return useQuery<PersonDetail>({
  queryKey: ["person-detail", id, language],
  enabled: Number.isFinite(id) && id > 0,
  retry: false,
  queryFn: async () => getFullPersonDetail(id, language.value),
});
```

Location in the project: resources/js/cineverse/features/person/services/person.service.ts · Frontend Service · TMDB call.

```
api.get(`/person/${personId}`, {
  params: {
    language,
    append_to_response: "movie_credits",
  },
});
```

Carlos clicks EN
 -> locale changes
 -> queryKey changes
 -> Vue Query runs queryFn again
 -> person.service.ts sends language=en
 -> TMDB returns localized data
 -> PersonHero displays person.biography



Figure. The same profile after requesting TMDB data in English.

15.1.5 Translating the Interface, Localizing Data, Formatting Dates, and Choosing an Email

What Changes	Where It Comes From	Mechanism
Inicio → Home	es.json / en.json	vue-i18n and t(...)
Person biography	TMDB	language inside the query
Date format	JavaScript	Intl.DateTimeFormat with es-AR / en-US
Verification / reset email	Laravel	Notification selects an ES/EN Blade view

Changing a translation in en.json does not change the biography returned by TMDB. They are different data sources that share the same active language.

15.2 Dictionaries, Keys, and Fallback Language

The two language files do not contain exactly the same keys. For example, t("nav.movies") looks for nav.movies in the active dictionary; for a key to have its own translation in both languages, it must exist in both es.json and en.json.

In this version, es.json contains 210 terminal keys and en.json contains 208. English does not define auth.success.resendSuccessTitle and auth.success.resendSuccessSubtitle, even though LoginView uses them in the successful verification-resend state. fallbackLocale: "en" cannot supply a key that is missing precisely from the English dictionary.

fallbackLocale: "en" defines English as the fallback language: if a key is missing from the active language, vue-i18n tries to find it there. In this version, when a key is missing specifically from en.json, the fallback cannot provide another English translation and the absence is exposed in that flow.

16. Communication Between Vue, TMDB, and Laravel

CineVerse keeps two data communications and one application bridge separate. Vue queries TMDB directly for catalog and people data; Vue queries Laravel for account, session, profile, and reviews; and Laravel/Inertia delivers the Vue application when the browser enters through a web route. Laravel uses Eloquent to read and write MySQL.

The browser chooses the path based on the action. A movie profile can obtain synopsis, cast, or images from TMDB without querying MySQL. Signing in, recovering a password, opening the profile, or publishing a review, by contrast, requires the Laravel backend and the application's own database.

16.1 From Vue to TMDB: Summary Flow

When a screen needs movies or actors, a frontend service uses tmdbApi.ts to query TMDB directly. That client provides the base URL and the Authorization header containing Bearer <token>; the response returns to the frontend for Vue to render. Section 4 develops the concrete Top Rated case step by step.

```
Vue screen
-> movies or person service
-> tmdbApi.ts
-> TMDB
-> JSON response
-> Vue Query / view
```

PHP and MySQL do not participate in this path: the Bearer token identifies CineVerse to TMDB, and the response returns directly to the frontend.

16.2 From Vue to Laravel: Login Example

When the user submits the login form, LoginView calls auth.service.ts. webApi sends POST /login with JSON, X-Requested-With, and withCredentials:true. routes/auth.php passes the request to AuthenticatedSessionController, which uses LoginRequest. That Form Request validates the data, applies RateLimiter, finds User with Eloquent, checks Hash::check, requires a verified email, and opens a session with Auth::login(). Vue then calls GET /user to reconstruct user and csrf_token in Pinia.

```
User clicks "Sign in"
-> LoginView.vue
-> auth.service.ts
-> webApi.ts
-> POST /login
-> routes/auth.php
-> AuthenticatedSessionController::store()
-> LoginRequest::authenticate()
-> User / Eloquent / MySQL
-> Laravel session
-> GET /user
-> useAuthStore / Pinia
```

Here the application's own API, MySQL, and the Laravel session do participate. Once an authenticated session exists, review operations and logout combine the session cookie with an X-CSRF-Token explicitly sent by CineVerse.

16.3 HTTP, JSON, and the Two Axios Clients

Both paths use HTTP and JSON. Authorization: Bearer identifies the application to TMDB. Requests to the application's own backend remain on the same Laravel origin and use its web cookies. CSRF protection does not always depend on GET /user: before login, routes such as /login, /register, /forgot-password, and /reset-password follow Laravel and Axios's normal mechanism, which uses the XSRF-TOKEN cookie and the X-XSRF-TOKEN header. After authentication, GET /user also returns csrf_token; CineVerse keeps it in Pinia and explicitly sends it as X-CSRF-Token when creating, editing, or deleting reviews and when logging out.

Axios is the library used to send these requests. tmdbApi.ts communicates with TMDB, and webApi.ts communicates with Laravel. Keeping two instances prevents the TMDB Bearer token from being mixed with the application's own backend session rules.

tmdbApi.ts creates the client that communicates with TMDB, and webApi.ts creates the client that communicates with Laravel. webApi uses baseURL "/", Accept application/json, X-Requested-With XMLHttpRequest, and withCredentials:true.

Feature	TMDB	Laravel / CineVerse
File	resources/js/cineverse/core/api/tmdbApi.ts	resources/js/cineverse/core/api/webApi.ts
baseURL	VITE_TMDB_BASE_URL	"/" (same Laravel origin)
Authentication	Authorization: Bearer <TMDB token>	web guard session cookie
Cookie sending (withCredentials)	Not enabled	true
CSRF	Not applicable	X-CSRF-Token on protected write operations
Data	Movies, actors, videos, genres	Account, session, profile, and reviews

Locations in the project: `resources/js/cineverse/core/api/tmdbApi.ts` creates the TMDB Axios client; `resources/js/cineverse/core/api/webApi.ts` creates the Laravel backend client. Services import one instance or the other depending on the destination.

```
// TMDB: external client
const api = axios.create({
  baseURL: import.meta.env.VITE_TMDB_BASE_URL,
  headers: {
    Authorization: `Bearer ${token}`,
    "Content-Type": "application/json"
  }
});

// Laravel: same origin as the application
const webApi = axios.create({
  baseURL: "/",
  headers: {
    "Content-Type": "application/json",
    Accept: "application/json",
    "X-Requested-With": "XMLHttpRequest"
  },
  withCredentials: true
});
```

16.4 Laravel and Vite During Development

In the integrated Laravel version, the old independent frontend proxy to `cineverse.local` no longer exists. Laravel serves the application from its own origin—for example, `http://127.0.0.1:8000`—and Vite acts as an asset server during development. `webApi` uses relative routes, so `/login`, `/user`, and `/api/reviews` reach the same Laravel instance that delivered the page.

Current state

The two Axios clients have no global timeout and no global error interceptor. An interceptor is a shared function that can inspect every request or response before it reaches the rest of the code. Currently, each service or composable receives the error and decides how to handle it within its own flow.

16.5 What Parts a Request Has in CineVerse

A request can carry information in different places. CineVerse uses URL parameters for pagination and filters, a JSON body for form data—the payload—headers for Authorization/CSRF, and cookies for the Laravel session.

- URL parameters: in `GET /api/movies/550/reviews?page=2`, `page=2` travels in the query string and Laravel reads it with `$request->query('page', 1)`.
- JSON body: `POST /api/reviews` sends `movie_id`, `movie_title`, `rating`, and `comment`. Laravel decodes the JSON and `$request->validate()` applies the controller rules.
- Headers: Authorization carries the Bearer token to TMDB. On Laravel web routes, Axios can send X-XSRF-TOKEN from the XSRF-TOKEN cookie. CineVerse also explicitly sends X-CSRF-Token in authenticated review and logout operations using the token returned by `GET /user`.

The session cookie is different: it is not part of the JSON or the CSRF token. The browser manages it as an HTTP cookie, and `webApi` uses `withCredentials:true` to preserve session credential behavior.

16.6 Cookie, CSRF, and Bearer Are Not the Same Thing

```

TMDB
  Authorization: Bearer <TMDB token>
  -> identifies the application to TMDB

CineVerse / Laravel Backend
  Session cookie
  -> allows the web guard to retrieve the user
  XSRF-TOKEN / X-XSRF-TOKEN
  -> normal Laravel/Axios web CSRF protection

Authenticated reviews and logout
  Session cookie + X-CSRF-Token
  -> session identity + explicit token obtained from GET /user

```

For example, GET /api/profile requires a session because it is inside middleware(['web', 'auth']). GET /api/movies/{movieId}/reviews is public, although it uses the web group so that, if a session exists, it can return the user's own review separately from the community list.

16.7 How an Axios Error Reaches a View

When Laravel responds with an HTTP error status, the Axios Promise is rejected and the code moves into error handling. Views can inspect `error.response?.data?.code` or validation errors to convert them into localized messages. The backend, meanwhile, reserves the technical log for unexpected failures and does not need to show a stack trace to the user.

17. Movie Explorer and Search System

The explorer does not use a single search method. `useMoviesQuery` chooses one of three paths depending on what the user selected: a predefined TMDB list, Discover with filters sent to TMDB, or a text search in which some filters are applied afterward in the browser.

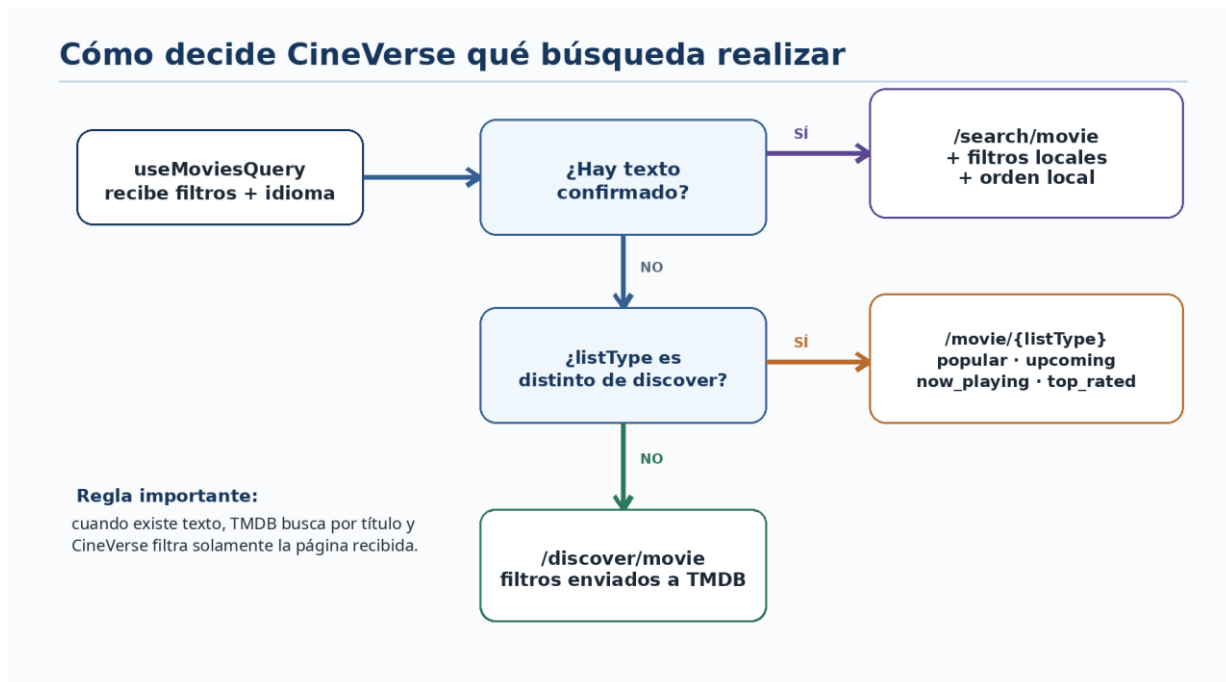


Figure. `useMoviesQuery` decision between text, predefined lists, and Discover.

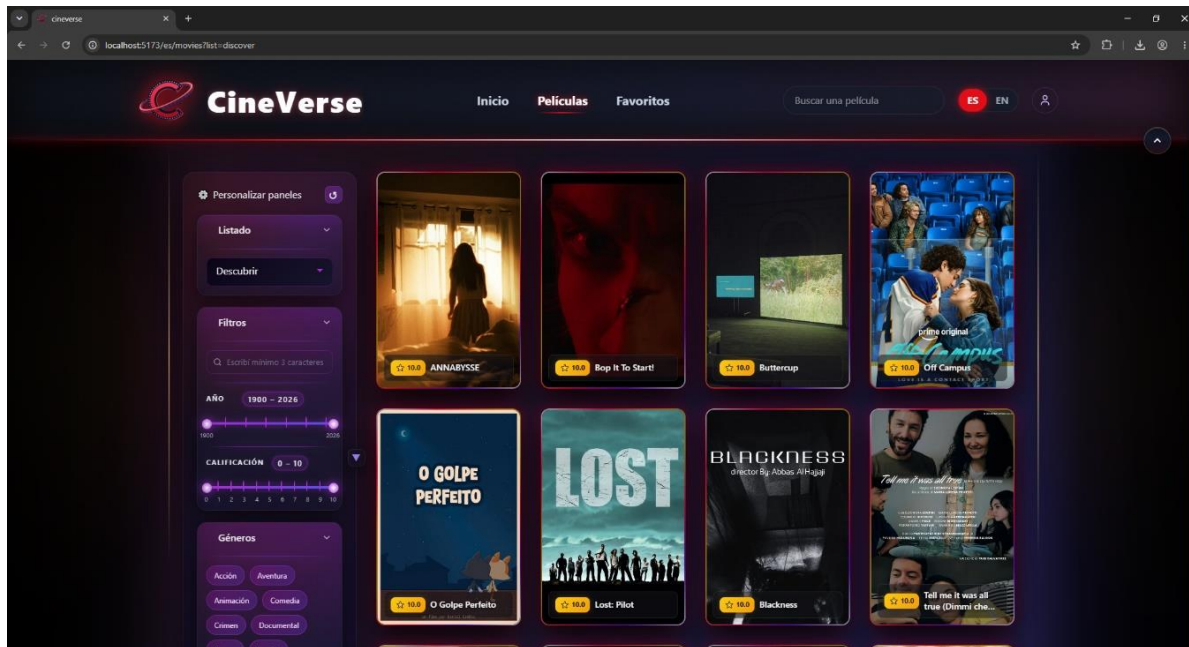


Figure. Movie explorer with side panel and results grid.

17.1 Guided Example: Carlos Opens Movies and the First Grid Is Built

Carlos enters `/es/movies`. `MoviesView` mounts and calls `useMoviesFilters()`. If the URL has no parameters and there is no previous state in `sessionStorage`, `listType` takes the default value `popular`. `useMoviesQuery` then creates an infinite query whose first page is 1.

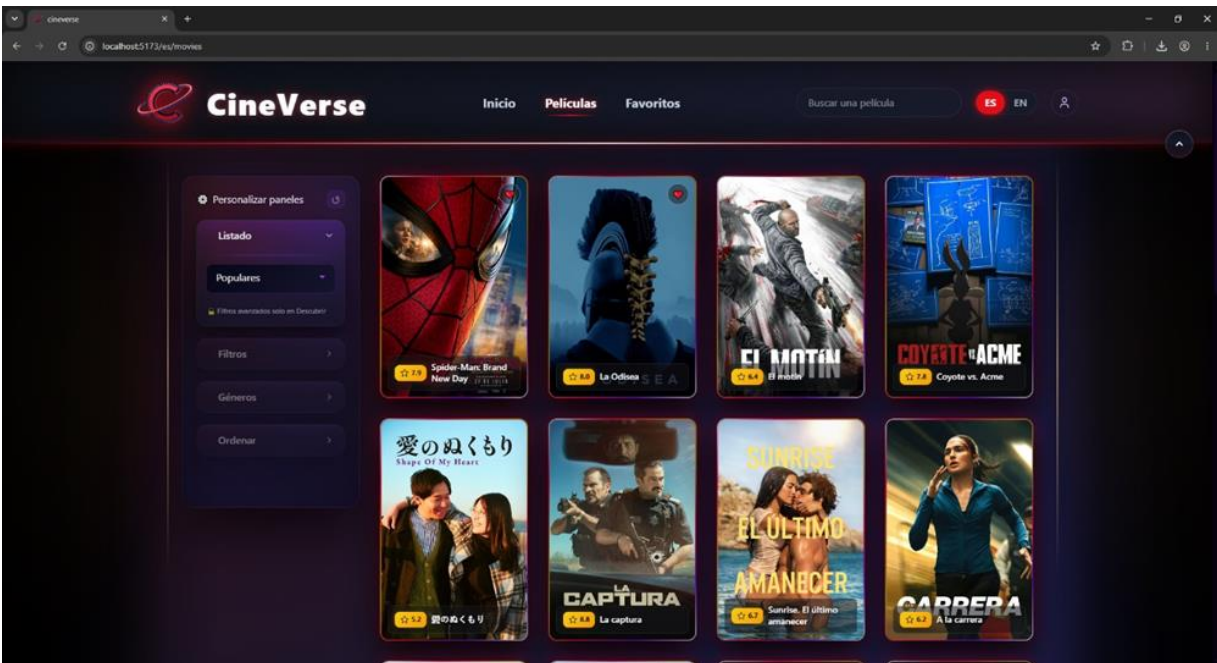


Figure. Movies explorer with the Popular catalog and initial grid.

```

Carlos enters /es/movies
-> MoviesView.vue
-> useMoviesFilters()
-> no URL or previous state: listType="popular"
-> useMoviesQuery(filters, apiLanguage)
-> useInfiniteQuery(initialPageParam=1)
-> getMovieList("popular", 1, "es")
-> movies.service.ts
-> GET /movie/popular?page=1&language=es

```

```
-> TMDB returns PaginatedResponse<Movie>
-> Vue Query stores the page in data.pages
-> MovieResults processes the results
```

Location in the project: resources/js/cineverse/features/movies/components/MovieResults.vue · Component · transforms the received pages into a usable movie list.

```
const pages = data?.pages;
const allMovies = pages.flatMap(p => p.results);

validMovies.value = allMovies.filter(
  movie => movie.poster_path && !movie.poster_path.includes("null")
);
```

The template then iterates over validMovies. Each v-for iteration creates a different MovieCard and passes it the corresponding movie object. :delay="index * 30" staggers the cards' entrance animation.

Location in the project: resources/js/cineverse/features/movies/components/MovieResults.vue · **Vue Component** · MovieCard v-for template

Used by: MovieResults.vue. Responsibility: render one card per valid movie.

```
<MovieCard
  v-for="(movie, index) in validMovies"
  :key="movie.id"
  :movie="movie"
  :delay="index * 30"
/>
```

The number of cards and the number of columns are different problems. validMovies determines how many MovieCard components exist; CSS Grid decides how they are distributed on the screen.

Location in the project: resources/js/cineverse/features/movies/components/MovieResults.vue · **component scoped CSS** · .movies-grid

Used by: MovieResults.vue. Responsibility: define responsive grid columns.

```
.movies-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 32px;
}

> 1024 px: 4 columns
<= 1024 px: 3 columns
<= 768 px: 2 columns
<= 480 px: 1 column
```

MIN_MOVIES is 20. If fewer than 20 movies remain after entries without posters are discarded and another page exists, MovieResults can emit load-more automatically before Carlos clicks Show more.

17.1.1 The JSON May Have Arrived While the Poster Is Still Loading

Each MovieCard has a second visual process. The Movie object may already exist in validMovies while the browser is still downloading the image from image.tmdb.org. The card therefore displays a skeleton until it receives the load event.

Location in the project: resources/js/cineverse/features/movies/components/MovieCard.vue · **Vue Component** · template + onPosterLoad()

Used by: MovieResults.vue. Responsibility: keep the skeleton visible until the browser finishes loading the poster.

```
<div v-if="!imageLoaded" class="poster-skeleton"></div>



function onPosterLoad() {
  imageLoaded.value = true;
}

```

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

TMDB data already available
-> MovieCard mounts
-> poster-skeleton is visible
-> browser requests the poster with loading="lazy"
-> @load
-> imageLoaded=true
-> skeleton disappears
-> poster appears

```

17.2 Guided Example: Carlos Hovers Over a Movie

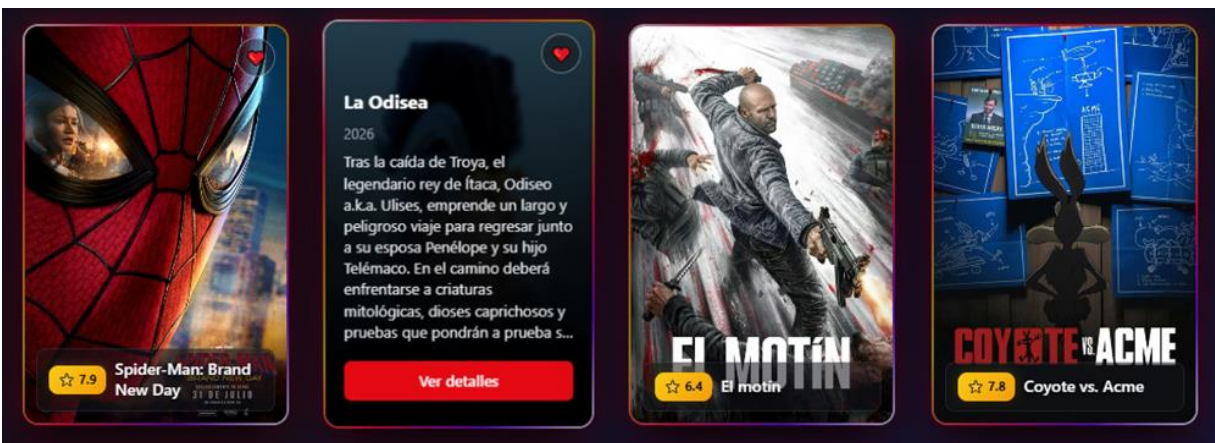


Figure. MovieCard during hover: title, year, synopsis, and View details button.

The description does not trigger another TMDB call. MovieCard already received the movie object from the grid, and that object contains title, release_date, overview, and vote_average. Hover only reveals information that is already in memory.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

Carlos places the cursor over The Odyssey
-> MovieCard already has movie.overview
-> CSS activates .movie-card:hover .poster-overlay
-> overlay changes from opacity 0 to visible
-> Vue displays title / year / overview
-> no new HTTP request is sent

```

MovieResults also listens for mousemove on the grid. handleGridMouseMove() identifies the .movie-card under the cursor, calculates the relative position inside the card, and applies rotateX/rotateY and glare. requestAnimationFrame groups the update with the browser's painting cycle.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

mousemove on .movies-grid
-> handleGridMouseMove(event)
-> closest(".movie-card")
-> getBoundingClientRect()
-> cursor position relative to center
-> rotateX / rotateY / glare
-> requestAnimationFrame
-> visible 3D effect

```

When View details is clicked, there is no specific @click on that button. The click bubbles to the .movie-card container, whose @click runs goToMovie(movie.id). The heart uses @click.stop precisely to prevent that navigation when Carlos only wants to save a favorite.

17.3 Guided Example: Carlos Searches for Batman from the Header

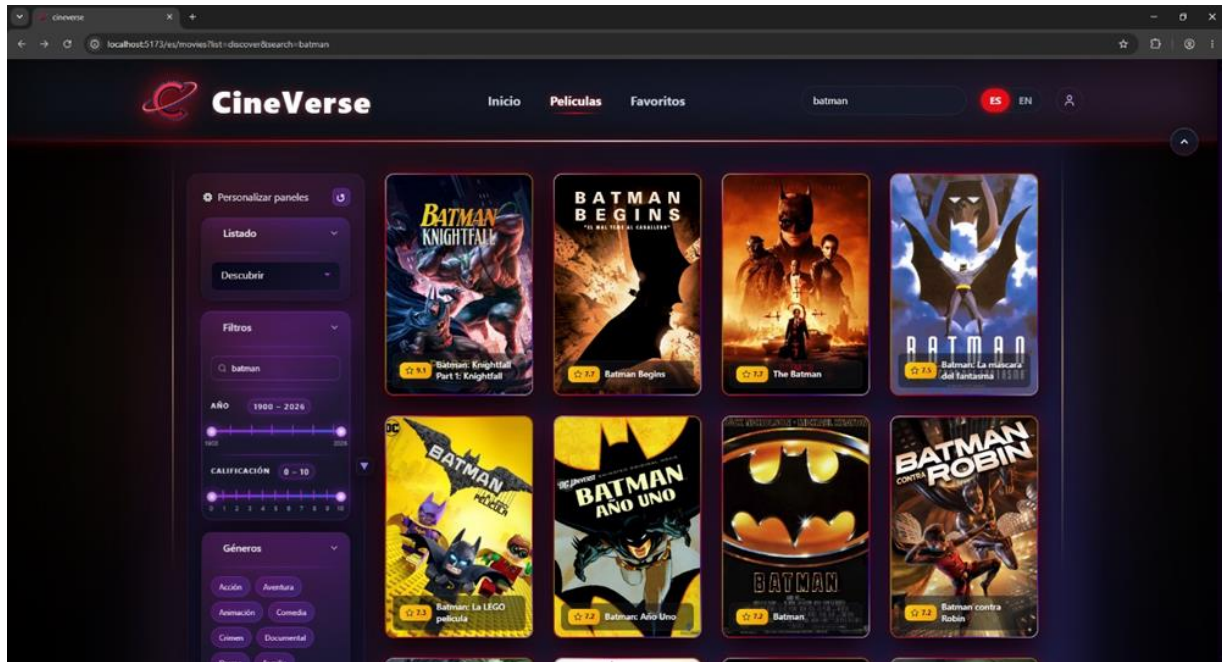


Figure. Batman search from the header and results in the grid.

The top search box and the side-panel search box represent the same criterion, but reach it through different paths. In AppHeader, Carlos types Batman and presses Enter; the form runs `handleSearch()`, which enforces a minimum of three characters and navigates to Movies with query parameters.

Location in the project: `resources/js/cineverse/layouts/components/AppHeader.vue` · **Vue Component** · `handleSearch()`

Used by: top search box in the header. *Responsibility:* validate the text and navigate to the explorer with query params.

```
function handleSearch() {
  const query = searchQuery.value.trim();
  if (!query || query.length < 3) return;

  router.push({
    name: "Movies",
    params: { lang: route.params.lang },
    query: {
      search: query,
      list: "discover",
      sort: "vote_average.desc"
    },
  });
}
```

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Carlos types Batman in the header
-> AppHeader.handleSearch()
-> router.push(...)
-> URL contains search=Batman
-> useMoviesFilters reads route.query
-> txtMovie / debouncedQuery take that value
-> useMoviesQuery detects text
```

```
-> searchByTextAndFilters()
-> GET /search/movie
-> TMDB
-> MovieResults replaces the grid
```

In this flow, the URL updates the side-panel state. The header input behaves differently: `searchQuery` takes `route.query.search` only when `AppHeader` is created through `getInitialSearchQuery()`, and the component has no watch on `route.query` that writes back to that ref. Therefore, if Carlos later changes the text from the side panel, the top field can retain the previous value during the same SPA session.

When the term arrives confirmed from the URL, the side panel can reconstruct the same state. They are not two independent queries: `Vue Router` and `useMoviesFilters` allow both interfaces to represent the same search criterion.

17.4 Path 1: Predefined Lists

If no text is entered and the selected type is `popular`, `now_playing`, `upcoming`, or `top_rated`, CineVerse directly queries `/movie/{listType}`. TMDB decides which movies make up the page and the order in which they arrive. For `upcoming`, CineVerse also removes results whose release date has already passed.

17.5 Path 2: Discover

If the user selects `Discover` and enters no text, CineVerse converts the interface filters into parameters for `/discover/movie`. On this path, TMDB receives genre, dates, rating, and sort order, so it filters the data before returning the page to the browser.

17.6 Path 3: Text Search Plus Filters

If there is confirmed search text, CineVerse uses `/search/movie`. TMDB receives the text, page, and language. The browser then applies any additional genre, year, rating, and sorting filters to the received results. This must be distinguished from `Discover`: on this path, those filters are not executed by TMDB.

17.7 Guided Example: Carlos Searches for Batman from the Side Panel

```
Confirmed text: "Batman"
↓
GET /search/movie?query=Batman&page=1&language=es
↓
TMDB returns page 1
↓
CineVerse applies, if present:
- genre
- year
- rating
- sorting
↓
MovieResults displays the already-filtered page
```

Essential limitation

Text + filters are applied to each received page, not to the complete universe of TMDB matches. `total_results` is replaced with `results.length` for that page, while `total_pages` preserves TMDB's original pagination.

17.7.1 Complete Flow of the “Batman” Search

The previous example shows the final URL. The actual frontend flow is longer and makes it possible to see which file participates at each step:

1. The user types Batman in the search field. `useMoviesFilters` stores the text in `txtMovie`.
2. The search watcher waits 500 milliseconds. If the trimmed text has at least three characters, it copies Batman to `debouncedQuery`.
3. The `debouncedQuery` change modifies `queryKey`. `useMoviesQuery`, built with Vue Query's `useInfiniteQuery`, runs its `queryFn` again.
4. Because confirmed text exists, `queryFn` chooses `searchByTextAndFilters()` and passes it the page, active language, and filters.
5. `searchByTextAndFilters()` first calls `searchByText()`, which runs `api.get("/search/movie", { params: { query, page, language } })`.
6. `tmdbApi.ts` completes the address with `VITE_TMDB_BASE_URL` and adds the Bearer token. TMDB receives the HTTP request.
7. TMDB responds with a page of results. Axios returns data in the `PaginatedResponse<Movie>` shape.
8. If genre, year, rating, or sorting filters exist, `searchByTextAndFilters()` applies them in the browser to that page's results. It then replaces `total_results` with `results.length` but preserves `total_pages` originally reported by TMDB.
9. Vue Query keeps the retrieved pages and exposes data, loading state, error state, and `fetchNextPage`.
10. `MoviesView` passes those values to `MovieResults`. `MovieResults` combines the loaded pages and displays the movies in the grid.

```
Batman
-> useMoviesFilters
-> debounce 500 ms
-> debouncedQuery
-> useMoviesQuery
-> searchByTextAndFilters
-> searchByText
-> tmdbApi / Axios
-> TMDB
-> JSON PaginatedResponse<Movie>
-> local filters
-> Vue Query
-> MoviesView
-> MovieResults
```

The language also participates in this flow. `MoviesView` passes `locale` as `apiLanguage`, and `useMoviesQuery` adds `language.value` to `queryKey`. Switching from ES to EN triggers a new request with `language=en`; it does not merely translate the screen buttons.

17.8 How Multiple Selected Genres Are Interpreted

The same genre selection is not interpreted exactly the same way on both paths. In Discover, with `_genres=28,12` is sent to TMDB. In text search, the code checks `genre_ids` with `some(...)`, so a movie is kept if it matches at least one selected genre. This difference is part of the project's current behavior.

18. How Filters Are Stored and Reflected in the URL

Filters do not begin in a URL or in `sessionStorage`: they begin when Carlos changes a visible option in the explorer. `MoviesView` creates a single filter object with `useMoviesFilters()`, passes it to `MovieFilters` so the interface can modify it, and at the same time passes it to `useMoviesQuery` so those changes ultimately affect the movie query.

18.1 Which Components Are Involved and Where They Are

Main location: `resources/js/cineverse/features/movies/composables/useMoviesFilters.ts` · **Composable** · exported `useMoviesFilters()` function

`MoviesView.vue` creates this composable once and passes the resulting object as a prop to `MovieFilters.vue`. The same object is also passed to `useMoviesQuery()`. Therefore, the visual panel, URL, and query do not maintain three independent copies: they share the same reactive state.

18.2 Guided Example: Carlos Selects Filters and the URL Changes

Carlos enters Movies, chooses Discover, selects Action and Adventure, sets the starting year to 2000, and a minimum rating of 7. `MovieFilters` modifies refs that belong to the object created by `useMoviesFilters()`; a watcher waits 300 ms, writes only the required parameters, and uses `router.replace()` to update the URL without filling the browser history with every intermediate slider movement.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Carlos changes MovieFilters.vue
-> props.movieFilters changes refs from useMoviesFilters()
-> watcher in useMoviesFilters.ts waits 300 ms
-> writeStoredQuery(query) stores movieFilters:lastQuery in sessionStorage
-> router.replace({ query }) updates the URL
-> filters.queryKey changes
-> useMoviesQuery.ts runs the query again
-> MovieResults receives the new data
-> Carlos sees the grid matching his filters
```

18.3 Data Controlled by the Filter System

- Typed text and confirmed debouncedQuery.
- Catalog type.
- Selected genre IDs.
- Year from / to.
- Rating range [0,10].
- Sorting.

18.4 Delay Before Launching a Search (Debounce)

To avoid sending a request for every keystroke, search waits 500 milliseconds after the last change. This technique is known as debounce. The text must also contain at least three characters: typing “ba” does not query TMDB; typing “bat” and then stopping for half a second does.

18.5 How Filters Are Written to the URL

Filter changes also wait 300 milliseconds before updating the URL. `router.replace` is used instead of `router.push` so that a new browser-history entry is not created for every intermediate movement of a slider.

The example uses rating order as the default value. Because `vote_average.desc` already represents that order, the code that converts filters into URL parameters does not need to write `sort` until the user changes that option.

```
/es/movies
?list=discover
&genres=28,12
&yearFrom=2000
&yearTo=2020
&ratingMin=7
```

18.6 Which Information Has Priority When Restoring Filters

URL with query parameters
→ the URL is the source of truth

Clean URL when returning to Movies
→ try to restore `sessionStorage["moviesFilters:lastQuery"]`

No URL or saved data
→ use default values

If the URL already contains filters, those values take priority. If the URL is clean, CineVerse tries to restore `sessionStorage["moviesFilters:lastQuery"]`. If that data does not exist either, it uses default values. When filters are restored from `sessionStorage`, they are written back into the URL so the address can be copied, shared, or saved.

18.7 Customizable Panels

The filter panel allows sections to be opened or closed and reordered through drag-and-drop. Part of that layout is stored in the browser so it can be restored on later visits. During customization mode, the panels temporarily close to make movement easier.

Current	edge	cases
The resize listener is registered with an anonymous function and removal is attempted with a different reference, so it is not actually removed when the component is unmounted. A scroll listener also remains attached, and a <code>scrollInterval</code> can remain active after leaving the view. If the view is left during customization mode, the temporary state of closed panels may end up being persisted.		

18.8 Complete Example: Filters -> URL -> Query -> Return

Filter state is synchronized in both directions. The composable does not only write the URL: it can also rebuild its state from a shared URL, from Back/Forward buttons, or from `sessionStorage` when returning to Movies with a clean address.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
User chooses:
  Discover
  genres 28 and 12
  year from 2000
  minimum rating 7

useMoviesFilters
  -> waits 300 ms
  -> router.replace(...)

URL:
/es/movies?list=discover&genres=28,12&yearFrom=2000&ratingMin=7

queryKey changes
  -> useMoviesQuery
  -> searchByFilters(...)
  -> GET /discover/movie with the filters
  -> TMDB responds
  -> MovieResults displays the page
```

If that URL is then copied and opened directly, `useMoviesFilters` reads `route.query` at startup, converts genres to numbers, restores `yearFrom/ratingMin`, and creates the same reactive state. If the URL contains no parameters, it tries to restore `moviesFilters:lastQuery` from `sessionStorage`.

`syncingFromRoute` and array comparison prevent a read -> write -> read loop and avoid resetting the grid when a new array contains exactly the same values as the previous one.

19. How More Movies Are Loaded

Incremental loading is easiest to understand from the interface: Carlos receives an initial grid, clicks “Show more,” and from that point `CineVerse` can continue requesting pages as the user approaches the end.

Pagination is shared among `useMoviesQuery()`, `MoviesView`, and `MovieResults`; none of those components performs the entire job alone.

`CineVerse` uses `useInfiniteQuery<PaginatedResponse<Movie>>` to preserve and manage the result pages already obtained from TMDB. The `getNextPageParam` function determines which page should be requested next by comparing the current page with the total number of pages reported by the API. When no more pages are available, it returns undefined and incremental loading ends.

19.1 Which Components Are Involved

Location: `resources/js/cineverse/features/movies/composables/useMoviesQuery.ts` · Composable · `useMoviesQuery()`. It decides which TMDB endpoint to use and, through `useInfiniteQuery()`, keeps the received pages and calculates the next one with `getNextPageParam()`.

Location: `resources/js/cineverse/features/movies/views/MoviesView.vue` · View. It receives `fetchNextPage` and `hasNextPage` from `useMoviesQuery()`, passes those values to `MovieResults`, and connects `@load-more="fetchNextPage"`.

Location: `resources/js/cineverse/features/movies/components/MovieResults.vue` · Component. It combines data pages, discards results without a usable poster, displays the “Show more” button, manages `autoScrollEnabled`, and installs the `IntersectionObserver`.

19.2 Guided Example: Carlos Clicks “Show More”

Carlos can already see the first page. When he clicks the button, `MovieResults.enableScroll()` enables automatic loading mode and emits `load-more`. `MoviesView` listens for that event and runs `fetchNextPage()`, a TanStack Vue Query function. `useMoviesQuery` calculates `page=2` from the latest response and follows the same search path used for page 1.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Carlos clicks "Show more"
-> MovieResults.vue :: enableScroll()
-> autoScrollEnabled = true
-> emit("load-more")
-> MoviesView.vue :: @load-more="fetchNextPage"
-> useInfiniteQuery runs getNextPageParam(lastPage)
-> pageParam = 2
-> queryFn uses the same active path: list / text / Discover
-> TMDB returns page 2
-> data.pages keeps page 1 + page 2
-> MovieResults uses flatMap and updates validMovies
-> Carlos sees new cards below the previous ones
```

19.3 Where the Next Page Comes From

The decision is not made in `MovieResults`. It is in `useMoviesQuery.ts`: `getNextPageParam` compares `lastPage.page` with `lastPage.total_pages`. If pages remain, it returns `page + 1`; when the last page is reached, it returns undefined and TanStack Vue Query stops offering another page.

Location in the project: resources/js/cineverse/features/movies/composables/useMoviesQuery.ts ·

Composable · useMoviesQuery() · getNextPageParam option

Used by: MoviesView.vue through useMoviesQuery(). Responsibility: decide the number of the next TMDB page.

```
getNextPageParam: (lastPage) => {
  if (lastPage.page < lastPage.total_pages) {
    return lastPage.page + 1;
  }
  return undefined;
}
```

19.4 How Result Pages Are Combined

MovieResults combines the movies from all pages already loaded and presents them as one continuous list. The user does not manually navigate to /page/2: new movies appear below the previous ones.

19.5 What Happens When Posters Are Missing

Some cards are not displayed when a movie has no poster. To avoid an initial grid that is too short, the flow can request another page if too few usable items remain after those movies are discarded.

19.6 “Show More” Button and Automatic Loading on Scroll

The view initially offers the “Show more” button. It then uses IntersectionObserver, a browser feature that reports when an element approaches the visible area. As the user approaches the end of the list, CineVerse can automatically request another page.

Technical detail of pagination with text and filters

Because total_pages preserves the original value from /search/movie, Vue Query's paginated query can continue moving through TMDB pages even if a specific page ends up with very few results after local filtering.

19.7 What MovieResults Does with Received Pages

MovieResults receives data.pages from useInfiniteQuery. Each pages item is a paginated TMDB response. The component combines them with flatMap and then discards movies whose poster_path is missing or contains an invalid value. The final result is stored in validMovies, the list actually iterated by v-for.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
pages[0].results + pages[1].results + ...
-> flatMap
-> all loaded movies
-> filter invalid poster_path
-> validMovies
-> MovieCard v-for
```

MIN_MOVIES is 20. Because TMDB paginates before CineVerse discards movies without posters, a page could leave fewer than 20 usable cards. If hasNextPage still exists and no other load is in progress, MovieResults emits load-more to automatically complete a reasonable initial amount.

19.8 Show More Button and IntersectionObserver

Infinite scrolling is not enabled from the start. autoScrollEnabled begins as false. The user clicks Show more, enableScroll() changes it to true and also requests a page. From that point on, an IntersectionObserver watches an invisible element placed at the end of the grid.

The observer uses rootMargin: "2500px" to begin loading before the user physically reaches the end. loadingMore and isFetchingNextPage prevent several simultaneous requests. There is also a 200 ms setTimeout that can request an additional page as prefetch if the user moves very quickly.

19.9 What Happens When the Search Changes

MoviesView increments resetSignal when the query identity changes. MovieResults watches that number, temporarily blocks automatic loads, disables infinite scrolling, clears validMovies, and returns to the top of the page. New Vue Query data then repopulates the grid.

20. Home Page

The home page displays three movie groups: trending, now playing, and top rated. Each group performs its own TMDB query, so one row can finish loading and appear even while the others are still waiting for their responses.

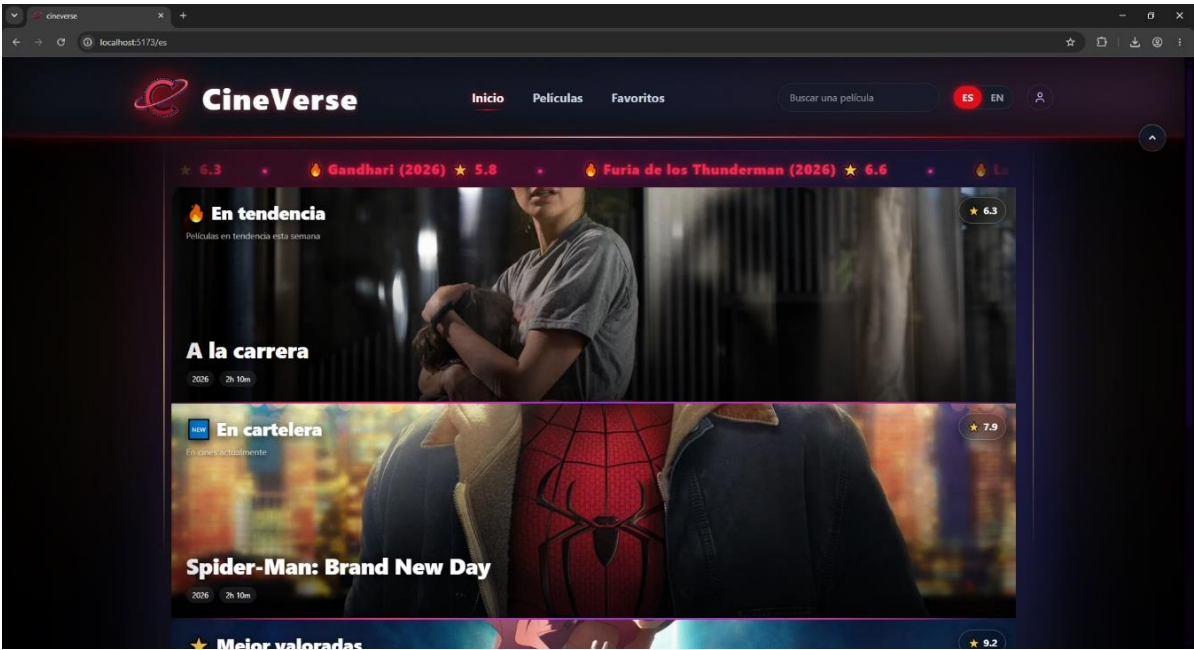


Figure. Home page with trending, now playing, and top-rated movies.

Block	Endpoint	How It Loads
Trending	/trending/movie/day	Independent query.
Now playing	/movie/now_playing	Independent query.
Top rated	/movie/top Rated	Independent query.

20.1 How the Home Page Reuses Data

Home results are considered fresh for five minutes. During that time, Vue Query can reuse them without treating them as stale. When the language changes, keepPreviousData temporarily preserves the previous version while the new response arrives, preventing the row from becoming empty.

20.2 Runtime Displayed on Cards

Current state
MovieCardCarousel displays “2h 10m” as a fixed runtime. The actual runtime is obtained only on each movie's detail page through /movie/{id}; Home cards do not perform that additional query.

20.3 Visual Loading State

HomeRow receives a loading prop but does not currently use it directly. The loading experience depends on `sessionStorage["home-loaded"]`, `showOverlay`, `isReady`, and the carousel's ready event. As a result, an empty response or a TMDB error does not always have a specific error message for each row.

20.4 Complete Home Flow

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

HomeView mounts
-> useHomeMovies(locale)
-> useQuery trending
-> useQuery top_rated
-> useQuery now_playing
-> all 3 requests are sent independently
-> movies.service.ts
-> tmdbApi.ts / Axios
-> TMDB
-> each query updates its own data
-> HomeView takes up to 10 movies per group
-> HomeRow -> HomeCarousel -> MovieCardCarousel
  
```

None of the three rows waits for the others. If trending arrives first, that information can already feed the ticker and its row while the others continue loading. Each group's `queryKey` includes the language; changing ES/EN requests the corresponding localized version.

HomeRow does not use the loading prop directly to decide the entire visual state. It also checks `sessionStorage["home-loaded"]`, waits for the carousel's ready event, and hides the overlay after a short delay. HomeCarousel emits ready only when Swiper has been initialized and the first visible image has finished loading and painting.

The ticker is built from the first five trending movies: title, year, and `vote_average`. It then adds three translated phrases and duplicates the array to create a continuous ribbon.

20.5 Guided Example: Carlos Opens Home

Carlos enters `/es`. HomeView runs `useHomeMovies(locale)` and creates three independent queries: trending, top rated, and now playing. None needs to wait for the others before starting.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

Carlos opens Home
-> HomeView
-> useHomeMovies(locale)
-> useQuery trending
-> useQuery top_rated
-> useQuery now_playing
-> movies.service.ts
-> tmdbApi.ts
-> TMDB
-> each query updates its own data
-> HomeRow
-> HomeCarousel
-> MovieCardCarousel
  
```

The visual loader also does not depend solely on JSON having arrived. HomeCarousel emits ready when Swiper is initialized and the first visible image has finished loading and painting. HomeRow uses that event together with `showOverlay` and `sessionStorage["home-loaded"]` to remove the initial state.

Practical result: one row can be ready while another is still loading because the three queries are independent.

21. Movie Details

The detail screen needs data from several endpoints. It first queries `/movie/{id}` to confirm that the movie exists. Only then does it request credits, videos, and similar movies. Those three requests are launched together with `Promise.all`. `Promise.all` waits for several asynchronous operations at the same time, so CineVerse does not need to finish one request before starting the next.

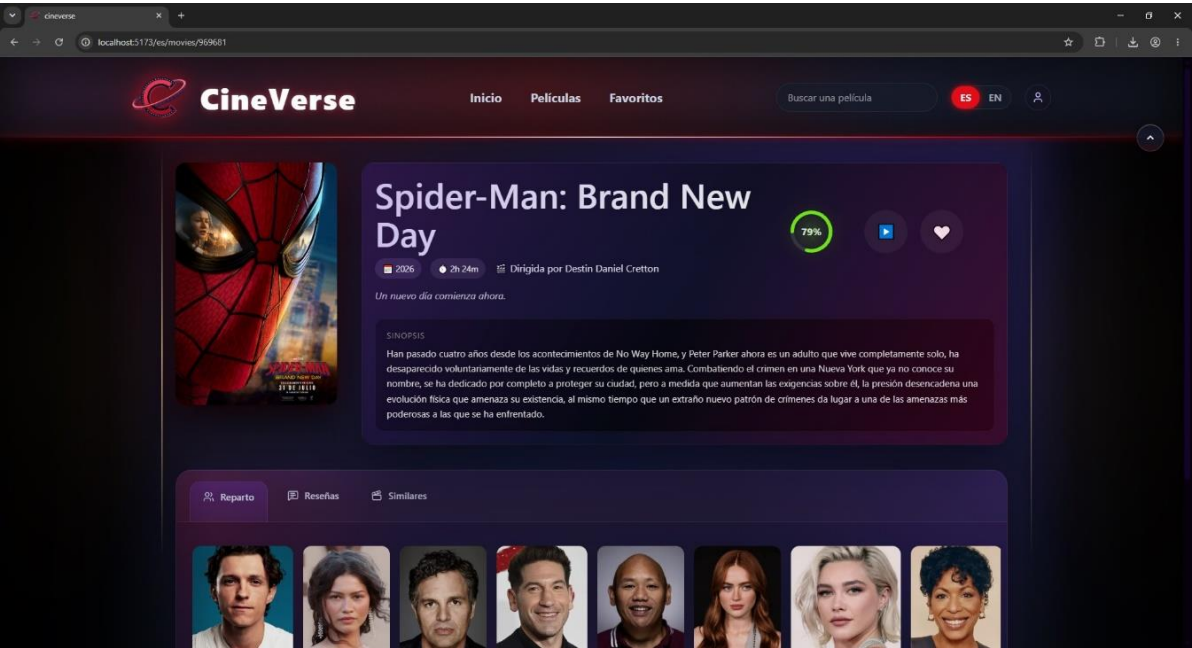


Figure. Movie detail page with main information, trailer, and cast.

Request Coordination

Location in the project: `resources/js/cineverse/features/movies/composables/movie-detail/useMovieDetail.ts` · Composable · `useMovieDetail()` `queryFn`. `MovieDetailView.vue` uses it to coordinate details, credits, videos, and similar movies.

```
const movieData = await getMovieDetail(id, language.value);

const [creditsData, videosData, similarData] = await Promise.all([
  getMovieCredits(id, language.value),
  getMovieVideos(id, language.value),
  getSimilarMovies(id, 1)
]);
```

21.1 How Data Is Prepared for Display

`useMovieDetail` transforms TMDB responses into a structure that is more convenient for the screen. It limits the cast to ten members, searches the crew for the entry whose job field is "Director", selects a YouTube trailer, and returns similar movies. This preparation avoids repeating the same logic inside several visual components.

Final Property	Source
movie	<code>/movie/{id}</code>
cast	<code>credits.cast.slice(0,10)</code>
director	First crew entry whose job field is Director

FinalProperty	Source
trailer	First video whose type field is Trailer and whose site field is YouTube
similar	results from /similar

21.2 When the Query Runs and How Errors Are Handled

The query runs only when the id is a valid number greater than zero. `retry=false` prevents automatic retries. The current limitation is visual: different query failures can end up showing a “not found” experience, so a nonexistent id and a temporary TMDB outage are not always clearly distinguished.

Code **detail**
 useMovieDetail contains a standalone expression ``trailer: videosData.results[0]?.key ?? null`` before building the final object. That expression does not assign the value to any variable or modify the returned object; the effective trailer is obtained later through `find(...)`.

21.3 Complete Flow When Opening /es/movies/550

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Vue Router
-> MovieDetailView obtains id=550
-> useMovieDetail(550, locale)
-> GET /movie/550
-> if it exists:
  Promise.all(
    GET /movie/550/credits,
    GET /movie/550/videos,
    GET /movie/550/similar
  )
-> limit cast to 10
-> find crew.job === "Director"
-> find first Trailer + YouTube video
-> MovieDetailResult
-> MovieHero + MovieTabs
```

The main query runs first because it confirms that the movie exists before launching the three secondary requests. `Promise.all` makes credits, videos, and similar movies wait together instead of chaining them one after another.

MovieDetailView keeps heroReady separate from isLoading. The loader does not disappear simply when JSON arrives: MovieHero emits ready when the poster has loaded and its visual entrance can be shown without an abrupt jump.

21.3.1 Guided Example: Carlos Opens Fight Club and Waits for the Hero

Carlos clicks a MovieCard with id 550. Vue Router changes the URL, and MovieDetailView obtains the id. useMovieDetail first queries /movie/550. Only if that detail exists does it launch credits, videos, and similar movies in parallel through `Promise.all`.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Carlos clicks Fight Club
-> MovieCard.goToMovie(550)
-> /es/movies/550
-> MovieDetailView
-> useMovieDetail(550, locale)
-> GET /movie/550
-> Promise.all(credits, videos, similar)
```

```
-> prepare cast / director / trailer / similar movies
-> MovieHero + MovieTabs
```

MovieDetailView keeps heroReady separate from request state. The JSON may have arrived, but the main loader also waits for MovieHero to finish loading its image and emit ready. This prevents the profile from appearing with an abrupt jump while the poster is still coming in.

21.4 When Cast, Reviews, and Similar Movies Are Loaded

MovieTabs starts with activeTab="cast". Cast is shown first. SimilarMovies receives the results that useMovieDetail already obtained from TMDB, but MovieReviews works differently: it is behind a v-if, and its onMounted(loadInitial) runs when the user enters the Reviews tab for the first time.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
open details
-> active tab: Cast
-> MovieReviews.onMounted() has not run yet

user clicks Reviews
-> Vue creates MovieReviews
-> onMounted(loadInitial)
-> GET /movies/550/reviews?page=1
-> Laravel / ReviewController / MySQL
```

MovieTabs wraps all three components in KeepAlive. After a tab has been created, Vue can preserve its instance while switching among Cast, Reviews, and Similar instead of destroying it immediately. This preserves, for example, MovieReviews' already-loaded state during those changes.

22. Trailers, Similar Movies, and Images

The detail page does not display only the movie's main data. It also uses additional TMDB information to play trailers, suggest similar titles, and show fallback images when resources are missing. This section follows that data from the TMDB response to what the user ultimately sees.

22.1 Trailer Selection

CineVerse scans the videos returned by TMDB and takes the first item whose type is Trailer and whose site is YouTube. It does not check the official property, so the accurate description is “first YouTube trailer found,” not “official trailer.”

Location in the project: resources/js/cineverse/features/movies/composables/movie-detail/useMovieDetail.ts · Composable · trailer selection inside queryFn. The result is passed to MovieDetailView.vue and then to MovieHero.vue.

```
const trailerData = videosData.results.find(
  (v) => v.type === "Trailer" && v.site === "YouTube"
);

trailer: trailerData ? trailerData.key : null
```

The key property identifies the video within YouTube. CineVerse uses it to build the embedded player inside a modal, allowing the trailer to be watched without leaving the detail page.

22.1.1 Guided Example: Carlos Clicks Watch Trailer

useMovieDetail has already examined videosData.results and stored the key of the first video whose type is Trailer and site is YouTube. When Carlos clicks the hero button, MovieHero does not create the iframe itself: it emits openTrailer, and MovieDetailView controls the modal.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Carlos clicks Watch trailer
-> MovieHero emits openTrailer
```

```
-> MovieDetailView sets isTrailerOpen=true
-> Vue mounts the modal
-> iframe src = https://www.youtube.com/embed/{key}?autoplay=1
-> watch locks body scroll
```

Close / Escape

```
-> isTrailerOpen=false
-> iframe unmounts
-> playback stops
```

The key comes from TMDB; the video plays from YouTube. CineVerse does not download or store the video file.

22.2 Similar Movies

getSimilarMovies queries /movie/{id}/similar. The current code fixes language="en-US" without using the interface's active language, so some similar movies may display an English title or synopsis while the rest of CineVerse is in Spanish.

22.3 Fallback Images When TMDB Does Not Provide One

CineVerse includes no-image-poster.jpg and no-image-avatar.jpg to display an alternative image when TMDB does not provide a poster or photo. Handling is not completely uniform: some components use these files explicitly, while others directly build the image.tmdb.org URL.

22.4 Image Sizes Used

Context	Observed Size / Intent
Movie cards	w500
Home backdrop	w1280
Cast	w185
Similar	w300
Detail hero	original
Actor	w500

22.5 From the TMDB key to the YouTube player

TMDB does not return the full video. It returns a key in results. useMovieDetail keeps the key of the first video whose type is Trailer and whose site is YouTube. When the user clicks the trailer button, MovieHero emits openTrailer and MovieDetailView opens the modal.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
trailer key = "abc123"
-> MovieDetailView isTrailerOpen = true
-> iframe src =
  https://www.youtube.com/embed/abc123?autoplay=1
```

Closing the modal sets isTrailerOpen to false; when the iframe is unmounted, playback stops. It can also be closed with Escape. While it is open, a watch sets document.body.style.overflow to hidden so the page in the background cannot keep scrolling.

22.6 How an image URL is built

TMDB returns paths such as `/abc123.jpg`. Components prepend `https://image.tmdb.org/t/p/` and the size they need. For example, a card can use `w500` while the hero uses `original`. If `poster_path` or `profile_path` is missing, certain components display local files such as `no-image-poster.jpg` or `no-image-avatar.jpg`.

Not all components apply exactly the same fallback mechanism. `MovieResults` discards movies without a poster before building the grid, while `MovieHero` can display `no-image-poster.jpg`. This difference is part of the current behavior.

23. Actors and Filmography

When the user selects an actor from the cast, `PersonDetailView` uses `usePersonDetail` and `person.service.ts`. TMDB calls this type of resource a person in its API. CineVerse requests the actor's data and filmography in a single request by using `append_to_response=movie_credits`, a TMDB parameter that allows movie credits to be included in the same response.

Location in the project: `resources/js/cineverse/features/person/services/person.service.ts` · Frontend Service · `getFullPersonDetail()`. Used by `usePersonDetail.ts` to retrieve the person and `movie_credits` from TMDB.

```
const { data } = await api.get<PersonDetail>(`
  /person/${personId}`,
  {
    params: {
      language,
      append_to_response: "movie_credits"
    }
  }
);
```

23.1 Components

Component	Responsibility
PersonHero	Displays the actor's main details, photo, and biography.
PersonKnownFor	Selects featured movies from the actor's credits.
PersonFilmography	Organizes the filmography for display in the view.

The Vue Query key includes the actor id and language so that each combination has its own result. The query is not retried automatically when it fails, nor is it rerun merely because the user returns to the tab. If TMDB does not provide `profile_path`, the interface displays the local fallback avatar.

23.2 Complete actor flow

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
User clicks an actor in the cast
-> router.push(name="Person", id)
-> PersonDetailView gets the id
-> usePersonDetail(id, locale)
-> person.service.ts
-> GET /person/{id}
    language=<language>
    append_to_response=movie_credits
-> TMDB returns PersonDetail + movie_credits
-> Vue Query stores the result
-> PersonHero
    -> PersonKnownFor
    -> PersonFilmography
```

`append_to_response=movie_credits` avoids a second separate request for the filmography: TMDB includes `movie_credits` in the same actor response.

23.3 How "Known For" is built

`PersonKnownFor` clones the credits so it does not modify the original prop, sorts them by popularity from highest to lowest, and takes the first eight. When a movie is clicked, it uses Vue Router to open `MovieDetail` while preserving the current language.

23.4 How the filmography is organized

`PersonFilmography` discards credits without `release_date`, sorts the rest by release date from newest to oldest, and then groups them by year. The result is transformed into a year-based timeline. The main photo uses `w500` and, if `profile_path` is missing or invalid, `PersonHero` uses the local avatar.

23.5 Guided example: Carlos opens an actor and changes the language

Carlos clicks a person in the cast. Vue Router preserves `lang` and navigates to `/:lang/person/:id`. `PersonDetailView` gets the `id`, `usePersonDetail` creates the query, and `person.service.ts` uses `append_to_response=movie_credits` to request the main data and filmography in the same response.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Carlos clicks an actor
-> router.push(name="Person", id)
-> PersonDetailView
-> usePersonDetail(id, locale)
-> person.service.ts
-> GET /person/{id}?language=<locale>&append_to_response=movie_credits
-> TMDB
-> Vue Query
-> PersonHero + PersonKnownFor + PersonFilmography
```

If Carlos switches from ES to EN, `language` is part of the `queryKey`. Vue Query treats the new language as a different result and `person.service.ts` queries TMDB again; the buttons change through `vue-i18n` and the external biography changes because TMDB received a different language.

24. Favorites

Favorites are stored only in the browser. They do not require an account, are not sent to the backend, and do not create records in MySQL. The Pinia store keeps Movie objects and `pinia-plugin-persistedstate` saves them in `localStorage`.

Rule	Implementation
No duplicates	<code>isFavorite()</code> and <code>addFavorite()</code> compare <code>movie.id</code> .
Maximum	<code>MAX_FAVORITES = 20</code> .
Remove	<code>removeFavorite(id)</code> .
Clear	<code>clearFavorites()</code> .
Persistence	<code>persist: true</code> in the store.

Practical implication

Favorites belong to the current browser, not to the CineVerse account. If the user switches browsers, uses another profile, or clears local storage, the list may disappear. Reviews are not affected because they are stored in MySQL and associated with the account.

24.1 What Pinia is in this flow

Pinia is the frontend's shared-state system. Local state inside MovieCard would exist only in that card; a Pinia store can be accessed by MovieCard, MovieHero, AppHeader, or MoviesFavoritesView, and all of them see the same favorites list.

app.ts creates Pinia only once, registers pinia-plugin-persistedstate, and then installs Pinia in the Vue application. Therefore, useFavoritesStore() does not create an independent list for each component: it returns access to the store identified as "favorites".

24.2 How useFavoritesStore is built

Location in the project: resources/js/cineverse/features/movies/stores/useMoviesFavoritesStore.ts · **Pinia Store**
· useMoviesFavoritesStore

Used by: MovieCard.vue, MovieHero.vue, AppHeader.vue, and MoviesFavoritesView.vue. Responsibility: maintain and persist local favorites.

```
const favorites = ref<Movie[]>([]);

const favoritesCount = computed(() =>
  favorites.value.length
);

isFavorite(id)
addFavorite(movie)
removeFavorite(id)
clearFavorites()

persist: true
```

favorites is the main state and contains complete Movie objects, not just ids. favoritesCount is derived: it is recalculated from favorites.length. isFavorite() uses some() to find a movie by id. addFavorite() returns true if it could add the movie and false if it already existed or the maximum of 20 had been reached.

removeFavorite(id) builds a new array excluding that id. clearFavorites() replaces the array with an empty one. Because favorites is reactive, each change is reflected immediately in every component that consumes the store.

24.3 Complete example: saving Fight Club as a favorite

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
MovieCard receives a Movie object:
{
  id: 550,
  title: "Fight Club",
  poster_path: "/...",
  release_date: "1999-10-15",
  vote_average: ...,
  ...
}

user clicks the heart
-> onFavoriteClick(movie)
-> isFavorite(550) == false
-> addFavorite(movie)
-> favorites.value.push(movie)
-> returns true
-> Vue updates the heart and favoritesCount
-> the persistence plugin saves the new state in localStorage
```

No POST is sent to the backend and no row is created in MySQL. The movie detail is not requested again in order to save it either: the Movie object that the component already had available is persisted.

24.4 What happens when the browser is reloaded

persist: true enables pinia-plugin-persistedstate for this store. When the state changes, the plugin serializes it in browser storage. When the application is created again after F5, the plugin restores the persisted state and favorites once again contains the saved movies.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
F5 / new load
-> app.ts creates Pinia
-> pinia-plugin-persistedstate is registered
-> useFavoritesStore is instantiated
-> the persisted state is restored
-> favorites contains the movies again
-> favoritesCount and hearts are recalculated
```

Persistence belongs to the browser, not to the authenticated user. Signing in with another account in the same browser profile does not create another independent list. Using another browser, another profile, or clearing localStorage can make those favorites disappear.

24.5 Limit of 20 and button behavior

MAX_FAVORITES is 20. addFavorite() first checks for duplicates and then favorites.length >= MAX_FAVORITES. MovieCard uses the returned boolean: if addFavorite() returns false because of the limit, the card temporarily displays a tooltip for 1500 ms.

MovieHero also toggles favorites, but it currently does not use the false value returned by addFavorite(). The limit is therefore still enforced, although the hero does not show the same visual notice that the limit has been reached. This is a current difference between the two interfaces.

24.6 Removing a favorite

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
user clicks the heart of a movie that is already saved
-> isFavorite(id) == true
-> removeFavorite(id)
-> filter(m => m.id !== id)
-> favorites changes
-> Vue updates the interface
-> the persisted state is updated as well
```

24.7 Favorites page and "Clear All"

MoviesFavoritesView gets favorites and favoritesCount directly from the store. It displays the current count out of 20 and creates a MovieCard for each saved object. If the list is empty, it displays an empty state with a link back to the explorer instead of the grid.

The Clear All button does not delete immediately. openConfirm() opens a modal, a watch blocks body scrolling, and confirmClear() calls clearFavorites(). When the store is emptied, the grid disappears, favoritesCount becomes 0, and the plugin persists the empty state.

24.8 Consequence of storing the complete Movie object

MoviesFavoritesView reuses the stored Movie objects to build the cards. The view does not iterate through favorites and make a new TMDB request for each movie. This makes opening Favorites immediate and allows it to work with persisted data, but it also means that the data is a local snapshot from the moment each movie was saved.

25. How the Laravel Backend Is Organized

The current backend no longer uses a custom Router, manual services, and direct PDO as its backbone. Laravel receives requests, determines middleware, runs Controllers or Form Requests, works with Eloquent, and builds

an HTTP response. The frontend still consumes JSON for account, profile, and reviews, but those contracts now live within the framework.

25.1 routes/web.php, auth.php, and api.php

routes/web.php handles navigation and delivers pages through Inertia. routes/auth.php contains routes related to user authentication, such as registration, login and logout, password recovery, and email verification. routes/api.php contains the application's own JSON endpoints. Keeping them separate makes it possible to recognize the purpose of a URL by its role: /es/movies/550 is SPA navigation, /login belongs to the authentication process, and /api/reviews provides data access through the API.

Route	Middleware	Controller/action	Use
GET /user	auth	closure in auth.php	Rebuild the session and CSRF state in Pinia.
POST /login	guest	AuthenticatedSessionController@store	Sign in.
POST /register	guest	RegisteredUserController@store	Create an account and send verification.
GET /verify-email/{id}/{hash}	signed + throttle	VerifyEmailController	Confirm email from the link.
POST /forgot-password	guest	PasswordResetLinkController@store	Request password recovery.
POST /reset-password	guest	NewPasswordController@store	Save a new password.
GET /api/profile	web + auth	Api\ProfileController@show	Profile summary.
GET /api/reviews	web + auth	ReviewController@index	My reviews.
POST /api/reviews	web + auth	ReviewController@store	Create a review.
PUT /api/reviews/{id}	web + auth	ReviewController@update	Edit own review.
DELETE /api/reviews/{id}	web + auth	ReviewController@destroy	Delete own review.
GET /api/movies/{movieId}/reviews	web	ReviewController@movieReviews	Public reviews + user context when available.

25.2 bootstrap/app.php

Laravel 12 centralizes part of its startup configuration in bootstrap/app.php. withRouting registers web.php, api.php, console.php, and the /up route, which is used as a basic application health check. withMiddleware adds HandleInertiaRequests and AddLinkHeadersForPreloadedAssets to the web group. withExceptions customizes reporting and the response used when MySQL is unavailable.

25.3 Controllers versus Requests

A Controller coordinates an HTTP action; a Form Request can encapsulate authorization and validation rules. LoginRequest is the most important example because it does more than validate email/password: it also normalizes the email, applies RateLimiter, checks the hash, requires a verified email address, and runs Auth::login. ProfileUpdateRequest centralizes profile rules. ReviewController uses \$request->validate() directly because its rules are limited to each specific operation.

25.4 Eloquent and JSON responses

The API Controllers work with Review::query() and User. ReviewController combines select, join, where, orderByDesc, forPage, count, and selectRaw to build community reviews, averages, and pagination. Laravel serializes Eloquent results to JSON when they are included in response()->json().

25.5 Traditional web routes inherited from Breeze

web.php retains GET/PATCH/DELETE /profile because they are part of the structure expected by Breeze. The visible CineVerse experience uses /es/profile through Vue Router and /api/profile for the summary. Keeping the traditional routes preserves part of the original infrastructure and tests, although not all inherited tests remain aligned with the current adaptation; section 34 explains this in detail.

25.6 Summary map: a frontend JSON request reaches Laravel

CineVerse's own endpoints do not follow the same path as TMDB. webApi.ts uses the same Laravel origin, sends Accept: application/json, and keeps session cookies with withCredentials: true. Laravel routes apply middleware before running the Controller.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Vue
-> TypeScript service
-> webApi.ts / Axios
-> Laravel route
-> middleware
-> Controller / Form Request
-> Eloquent
-> MySQL
-> response()->json(...)
-> Axios response.data
-> Vue updates the interface
```

Axios client	Destination	Credential / context
tmdbApi.ts	TMDB	Authorization: Bearer <VITE_TMDB_TOKEN>
webApi.ts	CineVerse Laravel	Session cookie and, for protected mutations, X-CSRF-Token

25.7 public/index.php: Laravel's front controller

When the web server receives a navigation request or another request that Laravel must process, the public entry point is public/index.php. This file does not contain CineVerse rules: it loads Composer's autoloader, obtains the Application instance from bootstrap/app.php, and hands the Request to the framework.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Browser / Axios
-> public/index.php
-> require vendor/autoload.php
-> require bootstrap/app.php
-> Application receives Request
-> routing + middleware + controller
```

This pattern is known as a front controller: many different URLs pass through a single entry point, and the framework decides which code must respond.

25.8 bootstrap/app.php: building the Application

Laravel 12 centralizes in bootstrap/app.php an important part of startup that, in earlier versions, could have been distributed differently. CineVerse uses Application::configure() and chains withRouting(), withMiddleware(), and withExceptions().

- withRouting registers routes/web.php, routes/api.php, routes/console.php, and the /up health check.
- withMiddleware adds HandleInertiaRequests and AddLinkHeadersForPreloadedAssets to the web group.
- withExceptions registers diagnostic-channel reporting and the special response used when MySQL is unavailable.

This file runs before a specific route reaches a Controller. It can therefore define cross-cutting behavior for many requests.

25.9 Laravel Router: HTTP method + URL + action

A route associates an HTTP method and a URL with an action. The same `/reviews` text can mean different things depending on the method: GET lists, POST creates. In `routes/api.php` the routes are declared as `/reviews`, but `bootstrap/app.php` registers that file through `withRouting(api: ...)`; Laravel applies the `/api` prefix, which is why the frontend calls `/api/reviews`. `routes/api.php` also groups protected routes and applies `web + auth` middleware before their Controllers.

Location in the project: `routes/api.php` · Laravel API routes. This group uses `web + auth` middleware and connects CRUD `/reviews` to `App\Http\Controllers\Api\ReviewController` (`index`, `store`, `update`, and `destroy`).

```
Route::middleware(["web", "auth"]->group(function () {
    Route::get("/reviews", [ReviewController::class, "index"]);
    Route::post("/reviews", [ReviewController::class, "store"]);
    Route::put("/reviews/{id}", [ReviewController::class, "update"]);
    Route::delete("/reviews/{id}", [ReviewController::class, "destroy"]);
});
```

The `{id}` parameter is extracted from the URL and Laravel passes it to the Controller method. Something similar happens in `movieReviews` with `{movieId}`.

25.10 Middleware: stopping or allowing the request before the Controller

Middleware runs around a route. `auth` requires an authenticated user; `guest` prevents certain authentication actions when a session already exists; `signed` validates a URL signature; `throttle` limits how often a route may be called; `web` enables the set of middleware for sessions, cookies, CSRF, and other web behavior.

```
POST /api/reviews
-> api.php group with web + auth
-> web restores session/cookies and applies CSRF
-> auth requires request()->user()
-> only then ReviewController::store()
```

This explains why security does not depend on Vue hiding a button. A manually constructed request still passes through the server middleware.

25.11 Request and Form Request

`Illuminate\Http\Request` represents the current request. It can read query parameters, JSON, the user, the session, and headers. A Form Request extends this class and adds authorization/validation rules in its own class.

`LoginRequest` is the most complete example in the project: `prepareForValidation()` trims the email; `rules()` validates shape and presence; `authenticate()` queries User, uses `Hash::check`, requires a verified email, calls `Auth::login`, and clears the `RateLimiter`; `ensureIsNotRateLimited()` applies the limit of three attempts per email + IP.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
POST /login
-> Laravel creates LoginRequest
-> prepareForValidation()
-> rules()
-> AuthenticatedSessionController::store(LoginRequest $request)
-> $request->authenticate()
-> session()->regenerate()
```

25.12 Controller: coordinating an HTTP action

The Controller sits at the boundary between HTTP and application logic. It decides which data it needs to read from the Request, which models or facades it uses, and which Response it returns. In CineVerse, some Controllers are very small because the framework has already handled part of the work; others, such as `ReviewController`, contain Eloquent queries and rules specific to the use case.

A Controller is not a Vue page. Vue runs in the browser; the Controller handles an HTTP request on the server.

25.13 Models and Eloquent

User and Review are Eloquent Models. User also extends Authenticatable because it participates in the authentication system. Models describe attribute assignment, casts, and relationships; queries are built with Review::query(), User::query(), where(), join(), count(), first(), get(), and other Query Builder/Eloquent methods.

25.14 How Laravel builds JSON responses

CineVerse's own endpoints return JsonResponse through response()->json(). ReviewController maintains a contract with success, code, and, when needed, data. Laravel serializes arrays, models, and collections to JSON and sets the specified HTTP status.

Location in the project: app/Http/Controllers/Api/ReviewController.php · ReviewController class · store(Request \$request) method. This block responds with 409 when a review by the same user for the movie already exists.

```
return response()->json([
    "success" => false,
    "code" => "REVIEW_ALREADY_EXISTS",
], 409);
```

The frontend receives status 409 and that JSON. Axios rejects the Promise because of the error status; MovieReviews can inspect response.data.code and decide which message to display. This version does not have its own Response.php class like the previous PHP backend.

25.15 Notifications and Blade in the current backend

User uses the Notifiable trait and replaces the standard notifications with CineVerseVerifyEmail and CineVerseResetPassword. A Notification decides which channel sends the message; in this project via() returns mail. MailMessage selects a different Blade view for Spanish or English.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
User
-> notify(new CineVerseVerifyEmail())
-> Notification::via() = mail
-> toMail()
-> MailMessage
-> emails.verify-account-es.blade.php / -en.blade.php
-> mailer configured in config/mail.php
```

25.16 Guided example: Carlos edits a review end to end

Carlos already has a review and changes its rating. This example brings the previous components together without skipping the boundaries between browser, framework, and database.

Location in the project: resources/js/cineverse/features/movies/components/movie-detail/MovieReviews.vue · **Vue Component** · submitForm()

Used by: MovieTabs.vue when the Reviews tab is active. Responsibility: collect rating/comment and call createReview() or updateReview().

```
Carlos clicks Save Changes
-> MovieReviews.vue
-> reviews.service.ts
-> webApi.put("/api/reviews/15", { rating, comment }, { X-CSRF-Token })
-> public/index.php
-> Laravel Router finds PUT /reviews/{id}
-> web + auth middleware
-> ReviewController::update($request, 15)
-> normalizeReviewComment()
-> $request->validate()
-> Review::query()->find(15)
```

```

-> check review.user_id === request()->user()->id
-> $review->update(...)
-> Eloquent -> MySQL
-> JSON { success:true, code:"REVIEW_UPDATED" }
-> Axios
-> MovieReviews reloads data
-> Carlos sees the confirmed state

```

26. Authentication and Account: Breeze, Sessions, and Recovery

CineVerse authentication is a good example of how Breeze was adapted without abandoning Laravel's standard mechanisms. The screens belong to the SPA, but the server uses Auth, the web guard, sessions, hashing, RateLimiter, MustVerifyEmail, Password Broker, and middleware. Simply saying "it uses Breeze" would hide what actually happens.

26.1 What Breeze provides and what Laravel actually executes

Breeze is not a service that remains running behind CineVerse. In this project it appears in `require-dev` and provided the initial code structure—the scaffolding—for authentication routes, Controllers, LoginRequest, and Feature/Auth tests. CineVerse then adapted that foundation and connected its own Vue screens. When a person signs in, the components that actually execute at runtime belong to the Laravel framework: the web guard, Auth, Hash, RateLimiter, the session, cookies, middleware, MustVerifyEmail, and the Password Broker.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

Breeze (initial code structure / scaffolding)
-> routes/auth.php + Auth controllers + LoginRequest + tests

Actual execution
CineVerse's own Vue
-> Laravel routes
-> Auth / Hash / RateLimiter / sessions / middleware
-> User / Eloquent / MySQL
-> JSON responses back to Vue

```

For that reason, "CineVerse uses Breeze" describes the origin of the structure, but it is not enough to explain authentication. It also explains why the visible screens are not Breeze's default views: `LoginView.vue`, `RegisterView.vue`, `VerifyEmailView.vue`, and the others are part of the CineVerse SPA and consume Laravel routes through `auth.service.ts` and `webApi.ts`.

26.1.1 Guided example: Carlos creates an account

Carlos completes `RegisterView` with username, email, password, and confirmation. Vue validates the basic user experience, but the account is actually created in Laravel. `RegisterView` also adds language so that the verification notification uses the template for the active language.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

Carlos clicks Create Account
-> RegisterView
-> registerUser(payload)
-> auth.service.ts
-> webApi.post("/register")
-> routes/auth.php
-> RegisteredUserController::store
-> normalize username / email / language
-> validate username / email / password
-> User::create()
-> Eloquent -> MySQL
-> sendEmailVerificationNotification()
-> CineVerseVerifyEmail
-> ES/EN Blade
-> SMTP

```

Unlike Breeze's initial behavior, this Controller does not run `Auth::login` after registration. Carlos must verify the email and then sign in.

26.2 Login: from the form to the session

`LoginView` validates the basic format and calls `loginUser()`. `auth.service.ts` sends `POST /login` through `webApi`. Because the request declares `Accept: application/json`, the Controller responds with 204 when access succeeds. Before that, `LoginRequest` runs the server-side checks.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

LoginRequest::authenticate()
  1. ensureIsNotRateLimited()
  2. find User by email
  3. Hash::check(password, user.password)
  4. require user->hasVerifiedEmail()
  5. Auth::login(user, remember)
  6. RateLimiter::clear(throttleKey)

AuthenticatedSessionController
  7. session()->regenerate()
  8. respond 204 to Vue

```

Regenerating the session identifier after authentication reduces the risk of session fixation. The browser keeps the cookie issued by Laravel; that cookie does not contain the password or replace the database, but allows future requests to be associated with the session stored by the configured driver.

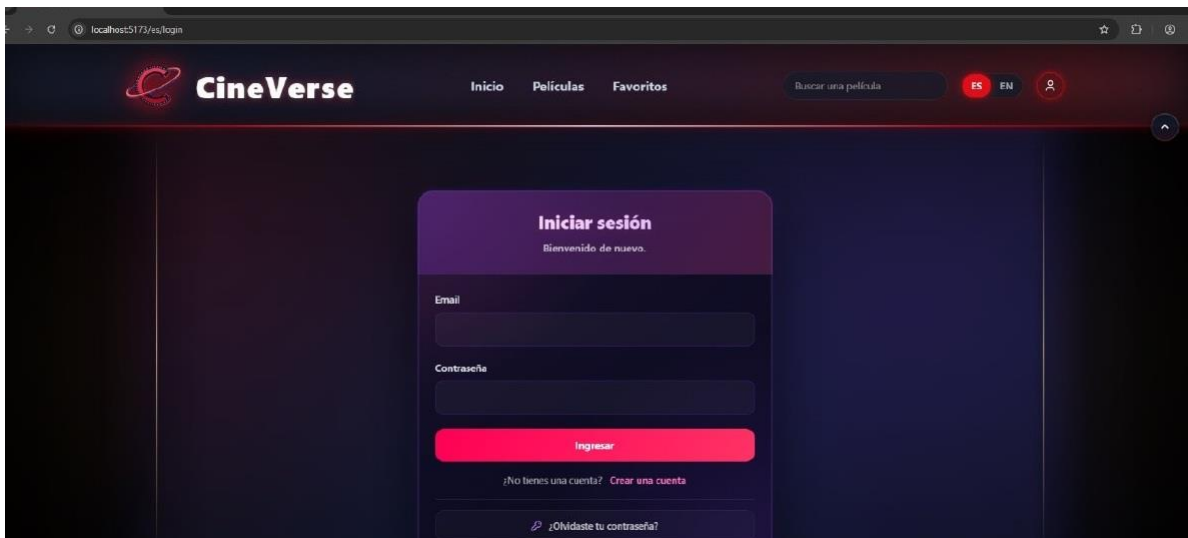


Figure. CineVerse login screen.

26.2.1 Guided example: Carlos signs in

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

Carlos enters email and password
-> LoginView validates format
-> loginUser()
-> POST /login
-> LoginRequest::authenticate()
-> ensureIsNotRateLimited()
-> User by email
-> Hash::check()
-> hasVerifiedEmail()
-> Auth::login()
-> session()->regenerate()
-> 204
-> frontend requests GET /user
-> Laravel returns user + csrf_token
-> useAuthStore updates Pinia

```

The 204 confirms the login but does not contain the full profile. CineVerse then requests `/user` to rebuild the reactive object used by the header, guards, and profile.

26.3 Rebuilding the session after F5

Once the user is authenticated, reloading the page destroys the in-memory JavaScript state. For that reason `app.ts` calls `authStore.initialize()`. The store requests `GET /user`. Laravel obtains `request()->user()` from the session and returns `id`, `username`, `email`, `created_at`, and `csrf_token`. Pinia repopulates `user` and `csrfToken` without asking again for email/password.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
F5
-> app.ts
-> useAuthStore.initialize()
-> GET /user
-> session cookie travels with the request
-> auth middleware
-> request()->user()
-> { user, csrf_token }
-> Pinia rebuilds the state
```

26.4 Registration and email verification

`RegisteredUserController` normalizes username, email, and language before validating. username must be 3–20 characters and use letters, numbers, or underscores; email must be valid, at most 80 characters, and unique. password uses `Rules\Password::defaults()` and confirmation. In `app/Providers/AppServiceProvider.php`, `Password::defaults()` is configured for a minimum of 8 and maximum of 32 characters, a mix of uppercase and lowercase letters, at least one number, and at least one symbol. `User::create()` then stores the hashed password and `sendEmailVerificationNotification()` is called.

Unlike the initial Breeze flow, registration does not run `Auth::login`. The account is created but must verify its email before login. User implements `MustVerifyEmail` and replaces the standard notification with `CineVerseVerifyEmail`, which uses separate HTML templates in Spanish and English.

26.4.1 Signed link

`CineVerseVerifyEmail` creates a temporary signed URL with `verification.verify`, `id`, the email hash, and `lang`. `routes/auth.php` applies signed and `throttle:6,1` middleware. `VerifyEmailController` reloads `User`, compares the hash with `sha1(getEmailForVerification())`, sets `email_verified_at`, and redirects to `/es/verify-email?verified=1` or its English equivalent.

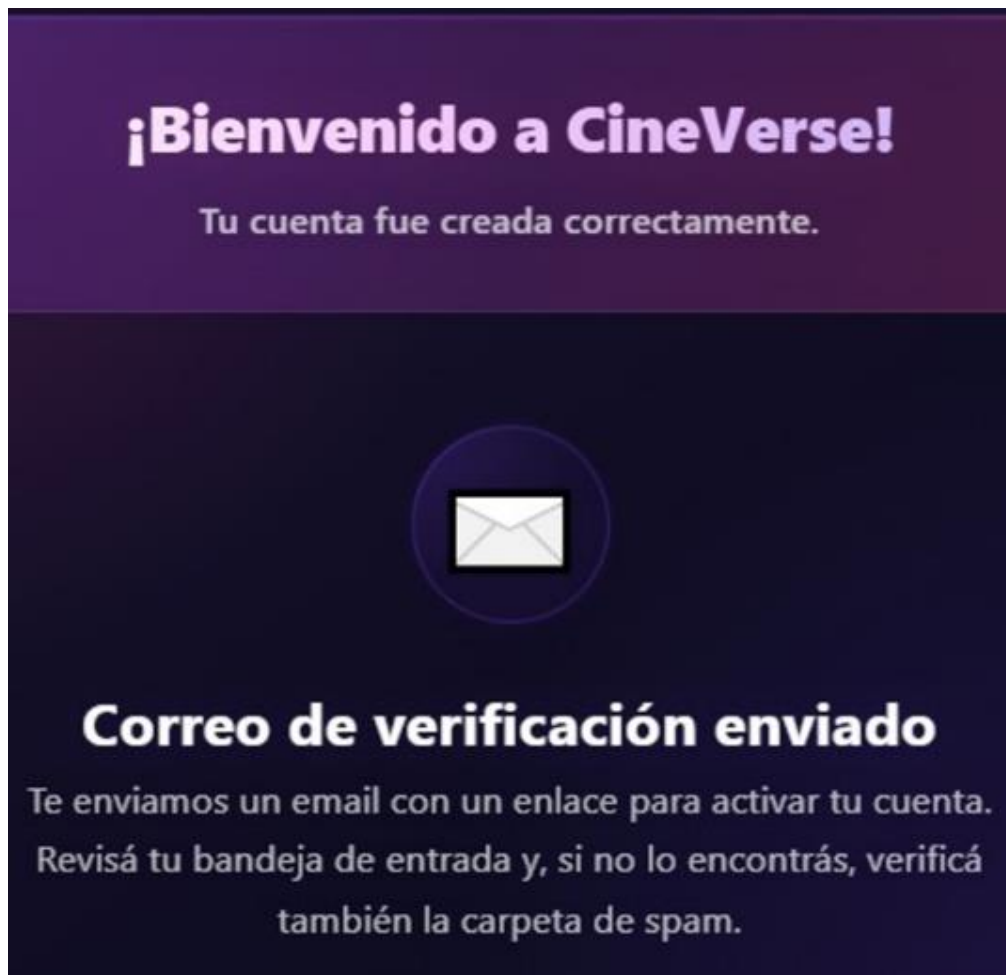


Figure. CineVerse email verification screen.

26.4.2 Actual verification-link lifetime in this version

CineVerseVerifyEmail generates the URL with `now()->addMinutes(config("auth.verification.expire", 60))`. `config/auth.php` does not define an `auth.verification.expire` key, so this call uses its fallback value of 60 minutes. The `verify-account-es.blade.php` and `verify-account-en.blade.php` templates now also display a validity period of 60 minutes, so the email text matches the effective expiration time of the verification link.

26.5 Resending verification

The `POST /email/verification-notification` route is inside `auth` middleware and uses `throttle:6,1`. At the same time, `LoginRequest` prevents `Auth::login()` from completing while the email is still unverified, while `LoginView` offers a resend button after that rejection. Reading the complete flow shows that the frontend can attempt the resend without having the session required by the protected endpoint. This is an inconsistency in the current behavior between the interface and the route protection.

26.6 Password recovery and Password Broker

`ForgotPasswordView` sends `POST /forgot-password`. `PasswordResetLinkController` delegates to `Password::sendResetLink()`, which uses the Password Broker configured as users in `config/auth.php` and the `password_reset_tokens` table. Password Broker is Laravel's mechanism for coordinating reset tokens: it creates and stores them, controls their expiration—60 minutes—and applies a 60-second throttle. User customizes the notification through `CineVerseResetPassword` to preserve CineVerse's design and language.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
request
ForgotPasswordView -> POST /forgot-password
```

```

-> Password::sendResetLink()
-> password_reset_tokens
-> CineVerseResetPassword -> email

final change
ResetPasswordView -> POST /reset-password
-> Password::reset()
-> new password hash
-> new remember_token

```

26.6.1 What ResetPasswordView actually validates when the link opens

The internal comments in ResetPasswordView state that the token is validated against the backend when the view mounts. The code currently executed does not make that request: onMounted only checks that token and email are present in the URL and, if they are, changes tokenStatus to valid. The token's actual validity and expiration are checked when Carlos sends POST /reset-password and Laravel runs Password::reset().

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

open /reset-password?token=...&email=...
-> ResetPasswordView.onMounted()
-> do token and email exist?
  no -> tokenStatus="invalid"
  yes -> tokenStatus="valid" (without validation request)

Carlos submits new password
-> POST /reset-password
-> Password::reset()
-> Laravel validates token / expiration
-> success or error

```

26.7 Logout

logoutUser() sends POST /logout with X-CSRF-Token. AuthenticatedSessionController calls Auth::guard("web")->logout(), invalidates the session, and regenerates the CSRF token. Pinia then clears its local representation. Logging out therefore does not merely hide the name in the header: it destroys the authenticated identity on the Laravel side.

26.8 Web guard, session, and cookie: three related concepts

config/auth.php defines the web guard with the session driver. The guard is the strategy Laravel uses to determine which user belongs to the request. The session stores state on the server or in the configured driver; the browser cookie carries the identifier needed to associate later requests with that session.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

successful login
-> Auth::login($user)
-> web guard associates user with the session
-> session()->regenerate() rotates the id
-> browser receives/updates session cookie

later request
-> cookie is sent automatically
-> web middleware restores the session
-> auth can resolve request()->user()

```

Pinia does not replace this mechanism. useAuthStore keeps a reactive representation for the interface, but the real identity is rebuilt from Laravel through GET /user.

26.9 Hash and the stored password

User has a password => hashed cast, and the Controllers also use Hash::make when creating or resetting credentials. During login, LoginRequest uses Hash::check to compare the entered password with the stored hash. The plaintext password is not stored in the users table.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

Carlos enters password
-> LoginRequest
-> User::where("email", ...)->first()
-> Hash::check(entered password, user.password hash)
-> true: can continue
-> false: ValidationException + RateLimiter::hit()

```

26.10 Login RateLimiter by email + IP

`LoginRequest::throttleKey()` combines the lowercase email with the IP. `ensureIsNotRateLimited()` allows up to three attempts before blocking. Each credential failure runs `RateLimiter::hit()`; a successful login runs `RateLimiter::clear()`. When the limit is exceeded, Lockout is emitted and a `ValidationException` with status 429 is thrown.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

key = strtolower(email) + "|" + ip

if tooManyAttempts(key, 3)
-> Lockout
-> 429
if credentials are incorrect
-> RateLimiter::hit(key)
if login succeeds
-> RateLimiter::clear(key)

```

26.11 Password Broker: password recovery in Laravel

Password Broker is Laravel's mechanism for coordinating reset tokens. `Password::sendResetLink()` locates the user, creates or replaces the corresponding token, and calls `sendPasswordResetNotification()` on the model. User redirects that notification to `CineVerseResetPassword`, which builds the link to the Vue screen.

When saving a new password, `Password::reset()` validates the token, email, and expiration according to the broker configuration. If the token is valid, `NewPasswordController` applies `Hash::make`, rotates `remember_token`, saves User, and emits the `PasswordReset` event.

26.11.1 Guided example: Carlos forgot his password

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

Step 1: request email
Carlos opens /es/forgot-password
-> ForgotPasswordView
-> POST /forgot-password { email, language }
-> PasswordResetLinkController::store()
-> Password::sendResetLink()
-> User::sendPasswordResetNotification($token)
-> CineVerseResetPassword
-> ES/EN Blade
-> SMTP

Step 2: open link
Carlos clicks the link
-> /es/reset-password?token=...&email=...
-> ResetPasswordView checks that token and email exist
-> displays the form

Step 3: save new password
-> POST /reset-password
-> NewPasswordController::store()
-> validate token/email/password
-> Password::reset()
-> Hash::make() + new remember_token
-> 204
-> Carlos returns to Login

```

In the current implementation `ResetPasswordView` does not remotely validate the token when it mounts. Although its internal comments say otherwise, it only checks that token and email are present; actual validity is decided when `POST /reset-password` is submitted.

26.12 remember_token

The users table includes `remember_token` because it is part of Laravel's authentication conventions. During password reset CineVerse rotates it with `Str::random(60)`. This token is not the password-reset token or the CSRF token; they are values with different purposes.

26.13 Email verification: signature, hash, and user state

`CineVerseVerifyEmail` generates a temporary signed URL with `id`, the `sha1` of the email, and `lang`. The signed middleware checks that protected parameters have not been altered. `VerifyEmailController` reloads the user, compares the expected hash, and sets `email_verified_at`. Only then will `LoginRequest` allow access.

The signed URL uses `config("auth.verification.expire", 60)`, and `config/auth.php` does not define that key, so the fallback value of 60 minutes is used. The email templates now also display a validity period of 60 minutes, so the email text matches the effective expiration time of the verification link.

27. Reviews and Profile

Reviews are CineVerse's most complete domain data because they pass through the interface, session, CSRF, Laravel routes, Controller, Eloquent, and MySQL. To avoid assuming those components are already understood, this section always follows the flow from what Carlos does to the query that runs and the response returned to Vue. The movie still belongs to TMDB; MySQL stores only CineVerse's own activity.

27.1 Review rules

Field	Laravel validation
<code>movie_id</code>	<code>required integer min:1</code>
<code>movie_title</code>	<code>nullable string max:255</code>
<code>rating</code>	<code>required integer min:1 max:5</code>
<code>comment</code>	<code>required string min:20 max:1000</code>

The comment must be between 20 and 1000 characters both when creating and updating. The interface can prevent invalid submissions, but `ReviewController` validates again because the browser is not a security boundary. `movie_id` must be positive, `movie_title` is optional in the request, and `rating` is limited to the 1–5 range. There is an important difference between HTTP validation and persistence: the migration defines `movie_title` as a non-nullable string column and `ReviewController::store()` saves `$validated["movie_title"] ?? ""`; therefore, if the client does not send a title, the row is persisted with an empty string rather than `NULL`.

27.1.1 Guided example: Carlos creates a review

Carlos opens the Reviews tab for a movie, chooses a rating, and writes a comment. `MovieReviews` validates the form for the user experience; when saving, `reviews.service.ts` sends the mutation to Laravel together with the session CSRF token.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

Carlos clicks Save
-> MovieReviews
-> createReview(payload, csrfToken)
-> reviews.service.ts
-> POST /api/reviews
    session cookie
    X-CSRF-Token
  
```

```

-> web + auth middleware
-> ReviewController::store
-> validate movie_id / movie_title / rating / comment
-> check duplicate user_id + movie_id
-> Review::create()
-> MySQL
-> JSON REVIEW_CREATED
-> MovieReviews reloads stats and list

```

The frontend can prevent an invalid submission, but Laravel validates again because a request can also be built outside the interface.

27.1.2 Map of files involved in creating a review

Frontend: MovieReviews.vue contains the form and submitForm(); reviews.service.ts defines createReview(); webApi.ts creates the Axios client that talks to Laravel and keeps session cookies.

Backend: routes/api.php registers POST /api/reviews inside web + auth middleware; app/Http/Controllers/Api/ReviewController.php, ReviewController class, store() method, normalizes and validates the payload; app/Models/Review.php is the Eloquent model used to create the row.

Database: database/migrations/2026_07_11_120000_create_reviews_table.php defines reviews, the foreign key to users, and the UNIQUE(user_id, movie_id) constraint.

27.2 One review per user and movie

Before inserting, ReviewController looks for a row whose user_id equals the session user and whose movie_id equals the payload. If one exists, it responds with REVIEW_ALREADY_EXISTS and 409. In addition, the migration defines UNIQUE(user_id, movie_id). The rule is protected both in application logic and in the database.

27.3 Editing and deleting only your own review

update() and destroy() load the review by id. If it does not exist, they return 404. If review.user_id does not match request()->user()->id, they return 403 NOT_REVIEW_OWNER. Vue hiding other users' buttons improves the experience, but actual authorization is enforced on the server.

27.4 Reviews for a movie

Reading reviews is public, but it can take advantage of an existing session. When Carlos opens the Reviews tab, MovieTabs mounts MovieReviews; MovieReviews runs loadInitial(), which calls fetchReviews(1), and reviews.service.ts sends GET /api/movies/{movieId}/reviews?page=1. The route only requires web middleware: a guest can read, while a valid cookie allows Laravel to recognize the user optionally.

The response separates four concepts: global movie statistics, pagination data, the user's own review if one exists, and the community list. This separation explains why Carlos's review can be highlighted without appearing twice among the community cards.

27.4.1 Guided example: Carlos opens the Reviews tab for Fight Club

Fight Club has movieId=550. Cast is the initial tab, so this request does not occur when the detail page first opens. Only when Carlos clicks "Reviews" does Vue create MovieReviews and run onMounted(loadInitial).

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

Carlos opens Fight Club (id 550)
-> MovieTabs.vue changes activeTab to "reviews"
-> Vue mounts MovieReviews.vue
-> onMounted(loadInitial)
-> fetchReviews(1)
-> reviews.service.ts :: getMovieReviews(550, 1)
-> webApi GET /api/movies/550/reviews?page=1
-> routes/api.php :: web middleware
-> ReviewController::movieReviews($request, 550)
-> Eloquent queries reviews + users

```

```
-> JSON stats/pagination/user_review/reviews
-> MovieReviews updates refs
-> Carlos sees average, total, his review, and the community
```

27.4.2 Where each step is located

Component that activates the tab: `resources/js/cineverse/features/movies/components/movie-detail/MovieTabs.vue` · Vue Component · `activeTab` and conditional rendering of `MovieReviews`.

Component that requests and renders data: `resources/js/cineverse/features/movies/components/movie-detail/MovieReviews.vue` · Vue Component · `loadInitial()`, `fetchReviews()`, and `loadMore()`.

HTTP Service: `resources/js/cineverse/features/movies/services/reviews.service.ts` · `getMovieReviews(movieId, page)`.

Route: `routes/api.php` · `GET /api/movies/{movieId}/reviews` with web middleware.

Backend: `app/Http/Controllers/Api/ReviewController.php` · `ReviewController` class · `movieReviews(Request $request, int $movieId)` method.

Model queried: `app/Models/Review.php` · `Review` class · Eloquent on the `reviews` table.

27.5 Profile and My Reviews

Profile and My Reviews use the same session identity, but answer different questions. `GET /api/profile` builds a short summary: total number of reviews and the six most recent. `GET /api/reviews?page=N` walks through the complete history in pages of 12. The first feeds `ProfileView`; the second feeds `MyReviewsView`.

27.5.1 Guided example: Carlos opens Profile

Frontend location: `resources/js/cineverse/features/profile/views/ProfileView.vue` · View · `onMounted(loadProfile)`.

Service location: `resources/js/cineverse/features/profile/services/profile.service.ts` · `getProfile()` · `GET /api/profile`.

Laravel location: `app/Http/Controllers/Api/ProfileController.php` · `ProfileController` class · `show(Request $request)` method. The route is in `routes/api.php` inside `web + auth` middleware.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Carlos enters /es/profile
-> ProfileView.vue :: onMounted(loadProfile)
-> profile.service.ts :: getProfile()
-> GET /api/profile
-> routes/api.php :: web + auth
-> Api\ProfileController::show()
-> $request->user()->id identifies Carlos
-> Review::where(user_id)->count()
-> Review::where(user_id)->orderByDesc(created_at)->limit(6)->get()
-> JSON reviews_count + latest_reviews
-> ProfileView updates reviewsCount and latestReviews
-> Carlos sees the total and his six most recent reviews
```

Location in the project: `app/Http/Controllers/Api/ProfileController.php` · **Laravel API Controller** · `App\Http\Controllers\Api\ProfileController::show()`

Used by: `GET /api/profile` from `routes/api.php`; the response is consumed by `profile.service.ts`. Responsibility: count all of the user's reviews and retrieve only the six most recent.

```
$reviewsCount = Review::query()
    ->where('user_id', $userId)
    ->count();

$latestReviews = Review::query()
    ->where('user_id', $userId)
    ->orderByDesc('created_at')
```

```
->limit(6)
->get();
```

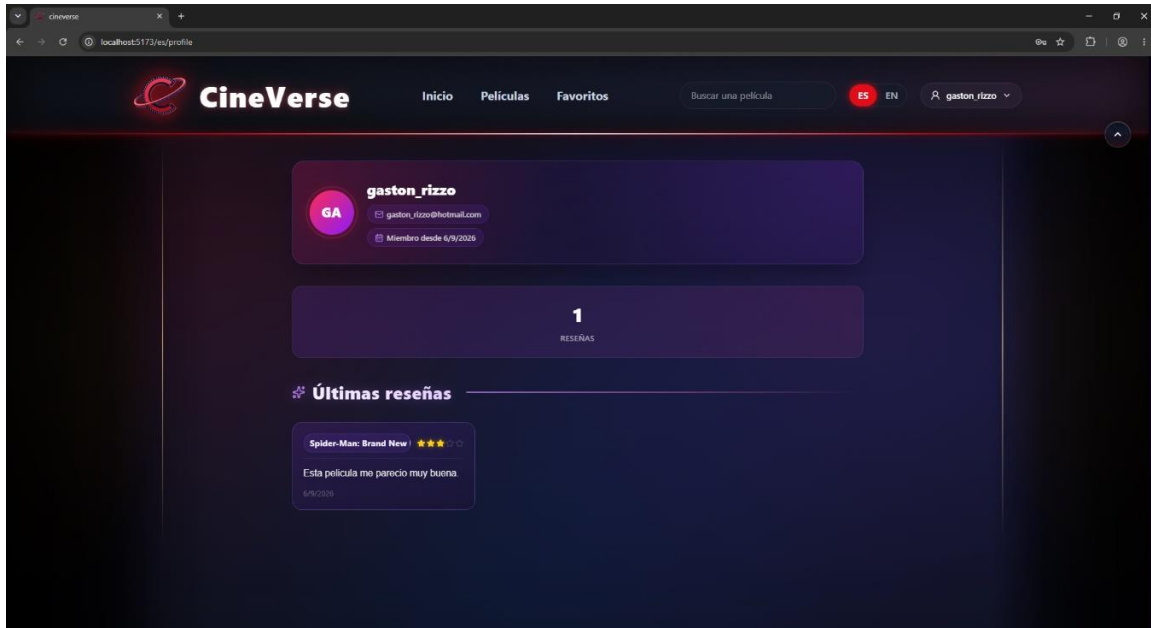


Figure. User profile with one review.

27.5.2 Guided example: Carlos opens "My Reviews"

Frontend location: resources/js/cineverse/features/profile/views/MyReviewsView.vue · View · loadReviews(), fetchReviews(page), and loadMore().

Service location: resources/js/cineverse/features/movies/services/reviews.service.ts · getUserReviews(page) · GET /api/reviews?page=N.

Laravel location: app/Http/Controllers/Api/ReviewController.php · ReviewController class · index(Request \$request) method. The route uses web + auth middleware.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Carlos opens My Reviews
-> MyReviewsView.vue :: onMounted(loadReviews)
-> fetchReviews(1)
-> reviews.service.ts :: getUserReviews(1)
-> GET /api/reviews?page=1
-> ReviewController::index()
-> page = max(1, query page)
-> perPage = 12
-> count total own reviews
-> forPage(1, 12)
-> JSON pagination + reviews
-> MyReviewsView replaces reviews[] with page 1
```

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Carlos clicks "Load More"
-> MyReviewsView.loadMore()
-> fetchReviews(currentPage + 1)
-> GET /api/reviews?page=2
-> Laravel returns up to 12 more
-> reviews.value = [...reviews.value, ...data.reviews]
-> the new cards appear below the previous ones
```

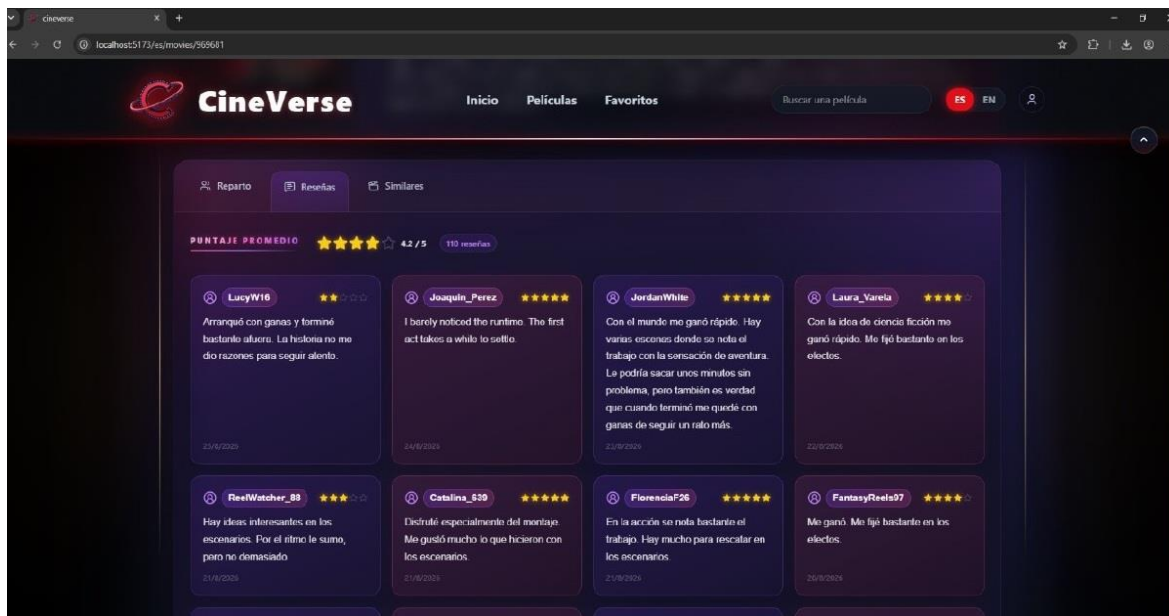


Figure. Reviews tab for a movie.

27.6 After creating, editing, or deleting

MovieReviews does not try to guess locally how the average, total, or list changed after a mutation. `submitForm()` and `confirmDelete()` call `fetchReviews(1)` again. The interface therefore rereads the state confirmed by Laravel/MySQL and updates stats, `userReview`, reviews, `currentPage`, and `hasNext` from one consistent response.

27.7 What happens in MovieReviews before sending a review

MovieReviews.vue keeps local state for `formMode`, `formRating`, `formComment`, `isSubmitting`, and `isFormOpen`. The creation button appears only when `authStore.isAuthenticated` is true, `userReview` does not exist, and the form is closed. This check improves the experience, but it does not replace auth or server validation.

Backend location: `app/Http/Controllers/Api/ReviewController.php · normalizeReviewComment()` method. Before validation, Laravel converts CRLF/CR to LF, removes spaces around line breaks, collapses three or more consecutive line breaks to two, and applies trim. Only then are `min:20` and `max:1000` evaluated.

27.8 Editing your own review

Editing does not resend `movie_id` or `movie_title`. `ReviewController::update` validates rating and comment, finds the review by id, and checks that `review.user_id` matches `request()->user()->id`. Only then does it run `update`.

Location in the project: `resources/js/cineverse/features/movies/components/movie-detail/MovieReviews.vue · Vue Component · openEditForm() + submitForm()`

Used by: MovieTabs.vue. Responsibility: preload rating/comment, send PUT, and reload the first page when finished.

```
Carlos clicks Edit
-> form loads current rating/comment
-> PUT /api/reviews/{id}
-> auth + CSRF
-> validate rating 1..5 / comment 20..1000
-> Review::find(id)
-> if it does not exist: REVIEW_NOT_FOUND 404
-> if it belongs to another user: NOT_REVIEW_OWNER 403
-> $review->update(...)
-> REVIEW_UPDATED
```

27.9 Deleting your own review

Deletion repeats the existence and ownership check. Hiding the delete button in Vue is not enough: the backend compares `user_id` before calling `delete()`.

Location in the project: `app/Http/Controllers/Api/ReviewController.php` · **Laravel API Controller** · `ReviewController::destroy()`

Used by: DELETE /api/reviews/{id} from routes/api.php. Responsibility: check existence/ownership and delete the review.

```
DELETE /api/reviews/{id}
-> Review::find(id)
-> check owner
-> $review->delete()
-> REVIEW_DELETED
-> frontend requests the first page again
```

27.10 Inside ReviewController::movieReviews()

Section 27.4 followed the visible flow from `MovieTabs` and `MovieReviews`. Here we look only at the server implementation that supports that flow: the public route in `routes/api.php` and the `ReviewController::movieReviews()` method, responsible for separating the user's own review, the community, pagination, and statistics.

The community list and the statistics are not the same query. `communityQuery` joins users to add username, excludes the authenticated user, and paginates in groups of 10. A separate `Review` query calculates `COUNT(*)` and `ROUND(AVG(rating), 1)`, including all reviews for the movie, including the user's own.

Location in the project: `app/Http/Controllers/Api/ReviewController.php` · **Laravel API Controller** · `ReviewController::movieReviews()`

Used by: GET /api/movies/{movieId}/reviews. Responsibility: build and paginate the community list with username, without duplicating the user's own review.

```
$communityQuery = Review::query()
->join('users', 'users.id', '=', 'reviews.user_id')
->where('reviews.movie_id', $movieId)
->orderByDesc('reviews.created_at');

if ($userId !== null) {
    $communityQuery->where('reviews.user_id', '<', $userId);
}

$reviews = $communityQuery
->forPage($page, $perPage)
->get();
```

Location in the project: `app/Http/Controllers/Api/ReviewController.php` · **Laravel API Controller** · `ReviewController::movieReviews()`

Used by: MovieReviews.vue through getMovieReviews(). Responsibility: calculate the movie's global total and average.

```
$stats = Review::query()
->where('movie_id', $movieId)
->selectRaw('COUNT(*) as total_reviews, ROUND(AVG(rating), 1) as average_rating')
->first();
```

27.11 Technical comparison: Profile versus My Reviews

The difference was already demonstrated in examples 27.5.1 and 27.5.2: `Profile` needs a fixed summary and `My Reviews` needs a paginated history. In backend terms, `ProfileController::show()` applies `limit(6)`, while `ReviewController::index()` sets `perPage = 12` and calculates `has_next`. This section keeps only the technical comparison so the Carlos flows are not repeated.

Location in the project: app/Http/Controllers/Api/ReviewController.php · **Laravel API Controller** · ReviewController::index()

Used by: GET /api/reviews?page=N from MyReviewsView.vue. Responsibility: paginate the user's review history 12 at a time and calculate has_next.

```
$page = max(1, (int) $request->query('page', 1));
$perPage = 12;

$totalReviews = (clone $query)->count();

$reviews = Review::query()
    ->where('user_id', $request->user()->id)
    ->orderByDesc('created_at')
    ->forPage($page, $perPage)
    ->get();
```

Result: ProfileController returns at most six recent items for a summary view; ReviewController::index() returns up to 12 per page and allows further pages while has_next is true. They are two different queries for the same session user.

Figure. The six most recent reviews.

28. Validation, Security, CSRF, and RateLimiter

CineVerse security does not depend on a single function. Validation, hashing, sessions, CSRF, rate limiting, middleware, database constraints, and review authorization cover different risks. The frontend also validates for user experience, but Laravel checks again what arrives over HTTP.

28.1 Frontend versus server validation

TypeScript and validation functions help detect errors before submitting a form, but they can be bypassed with external tools. Laravel validates registration, login, reviews, and profile data. MySQL adds structural integrity through foreign keys and the unique reviews constraint. This defense in depth lets the UI remain convenient without making it the only guarantee.

28.2 Passwords

User defines the password => hashed cast. RegisteredUserController also uses Hash::make when creating the account. LoginRequest never compares plaintext with plaintext: Hash::check verifies the entered password against the stored hash. Invalid-credential responses do not reveal whether the email or password failed, reducing information useful for account enumeration.

28.3 Login RateLimiter

LoginRequest creates throttleKey by combining the normalized email and IP. ensureIsNotRateLimited() allows at most three attempts before returning 429. RateLimiter::availableIn reports the remaining time and the Lockout event is fired. After a successful login, RateLimiter::clear removes accumulated failures.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
throttleKey = lower(email) + "|" + ip

if RateLimiter::tooManyAttempts(key, 3)
    -> Lockout event
    -> ValidationException status 429
else
    -> check credentials
    -> failure: RateLimiter::hit(key)
    -> success: RateLimiter::clear(key)
```

28.4 CSRF

A session cookie can travel automatically with a browser request. CSRF protects against a scenario in which another site tries to exploit that cookie to perform an unwanted action. On normal web routes, Laravel and Axios use the XSRF-TOKEN / X-XSRF-TOKEN mechanism. Once the user is authenticated, GET /user also returns csrf_token and useAuthStore keeps it in memory; createReview, updateReview, deleteReview, and logoutUser send that value explicitly in X-CSRF-Token.

CSRF does not replace auth. The cookie lets Laravel know who the user is; the token proves that the state-changing request knows a value generated inside the session. A sensitive operation can therefore require both.

28.5 auth and guest middleware

Login/registration/reset routes are grouped under guest. Routes for /user, resending verification, password confirmation, password changes, and logout are under auth. In routes/api.php, profile and review CRUD use web + auth. The frontend adds requiresAuth/requiresGuest guards to avoid inconsistent screens, but server middleware remains the real barrier.

28.6 Signed URLs and verification throttling

Email verification combines a temporary signed URL with throttle:6,1. signed prevents protected parameters from being modified without invalidating the signature. The Controller also compares the email hash with hash_equals before marking the account. A manipulated URL should not become a valid verification.

28.7 What the web group actually provides

routes/api.php explicitly adds web middleware to endpoints that need to share the browser session. Laravel's web group bundles middleware related to cookies, sessions, and CSRF protection. This allows /api/profile and review CRUD to work with the same session as /login and the web routes.

The public movie-reviews route also uses web. It does not require auth, but if a valid cookie exists Laravel can resolve request()->user() and return user_review separately.

28.8 Vue Router guard versus Laravel middleware

Vue Router's requiresAuth and requiresGuest improve navigation behavior so the SPA does not display inconsistent screens. They are not a security boundary because they exist only in the browser. Laravel's auth and guest middleware do run on the server for every protected request.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Vue guard requiresAuth
-> navigation experience

Laravel auth middleware
-> real endpoint protection
-> a manual request without a session is rejected too
```

28.9 Password hashing

Hash::make creates the hash used when registering or resetting a password; Hash::check verifies login. The Password::defaults rules defined in AppServiceProvider require 8–32 characters, uppercase and lowercase letters, a number, and a symbol. Validating composition and storing a hash are different tasks.

28.10 RateLimiter and cache

RateLimiter does not use the login_attempts table. Its state relies on Laravel's configured cache system. With CACHE_STORE=database, counting and expiration keys are stored through the cache tables. LoginRequest builds a specific key from email + IP so the limit applies to that user/source.

28.11 Guided example: why a review needs both a session and CSRF

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

Carlos is already signed in
-> GET /user returns csrf_token
-> Pinia stores csrfToken

Carlos clicks Save Review
-> browser sends session cookie
-> reviews.service.ts adds X-CSRF-Token
-> web middleware validates session/CSRF
-> auth resolves user
-> ReviewController validates payload
-> Eloquent saves

If the session is missing: auth stops the flow
If CSRF is missing/expired: web middleware rejects the write

```

The cookie answers "which session is this?"; the CSRF token proves that the write request knows a value generated inside that session. One does not replace the other.

28.12 Defense in depth for POST /api/reviews

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

1. Vue validates user experience and length
2. web/auth protect the request
3. CSRF protects the mutation
4. $request->validate checks types/ranges
5. Controller checks duplicate/ownership
6. Eloquent uses SQL parameters
7. MySQL applies FOREIGN KEY + UNIQUE

```

Each layer covers a different problem. The fact that MovieReviews does not allow six stars does not remove the need for max:5 in Laravel; a request can be built without using the Vue interface.

29. Interface States and Error Handling

CineVerse represents more states than just "success" and "error". A screen may be loading, may have completed successfully with no results, may receive an expected validation error, or may suffer a technical failure in a dependency. Distinguishing these cases makes it possible to follow the correct code instead of attributing every negative message to the backend.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

Loading
-> the operation has not finished yet

Empty state
-> completed successfully, but there are no useful items

Expected error
-> Laravel or the interface knows the cause and can express it in a controlled way

Technical failure
-> unexpected exception or unavailable dependency

Unknown navigation
-> Vue Router did not find a valid View

```

Reading method used in this section: cause -> file that detects it -> state/request -> response -> consuming component -> template condition -> visible message.

CineVerse distinguishes expected errors—invalid credentials, duplicate review, validation, 404—from infrastructure failures or internal errors. The interface tries to show a useful message to the user while technical details are retained in logs when they provide diagnostic value.

29.1 API errors

Review Controllers return semantic codes such as `REVIEW_NOT_FOUND`, `NOT_REVIEW_OWNER`, `REVIEW_ALREADY_EXISTS`, or `REVIEW_CREATED` together with appropriate HTTP statuses. The frontend can map those codes to vue-i18n messages without exposing exceptions or SQL.

29.2 Database unavailable

`bootstrap/app.php` detects `QueryException` whose message contains `SQLSTATE[HY000] [2002]`. For JSON requests it returns 503 with code `DATABASE_UNAVAILABLE`. For web navigation it returns `resources/views/errors/database.blade.php`. A MySQL outage therefore does not necessarily end in an incomprehensible stack-trace page.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
API
{
  "success": false,
  "code": "DATABASE_UNAVAILABLE",
  "message": "The database is unavailable."
}
HTTP 503

Web
-> errors.database Blade
-> HTTP 503
```

29.3 Frontend 404

Vue Router contains a localized catch-all and a global catch-all to render `NotFoundView`. Laravel, in turn, delivers CineVerse for localized deep routes. This allows a URL such as `/es/whatever` to reach the SPA and display CineVerse's visual 404 instead of failing before Vue mounts.

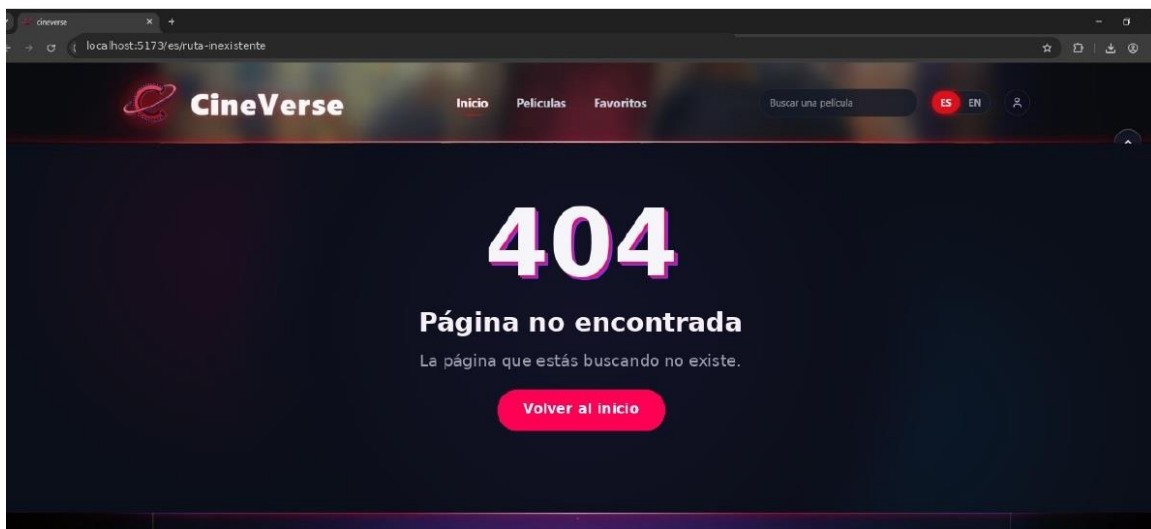


Figure. 404 view displayed by Vue Router.

29.4 Expected errors do not flood the log

The report handler in `bootstrap/app.php` avoids logging `ValidationException` and expected HTTP errors such as 403, 404, 419, or 422. The custom channel is reserved mainly for 500 failures and database-unavailable events. This decision improves the signal-to-noise ratio of the log file.

29.5 How to read a state from cause to screen

In Vue there is not necessarily a different page for each state. A ref, a Vue Query result, a store, or an HTTP response changes; Vue reevaluates the template and v-if/v-else determines which block remains visible. If the text is translated, vue-i18n resolves the final key.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
CAUSE
-> file that produces/detects it
-> ref / store / Vue Query / HTTP response
-> View or Component that consumes it
-> v-if / catch / computed
-> t("...")
-> MESSAGE CARLOS SEES
```

29.6 Example: why "Loading movie" appears

MovieDetailView separates two conditions: the useMovieDetail query may still be loading, and MovieHero may not have finished loading the poster. That is why the arrival of the JSON alone does not remove the overlay.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
MovieDetailView
-> useMovieDetail(id, locale)
-> isLoading = true while Vue Query waits for TMDB
-> heroReady = false

TMDB responds
-> isLoading = false
-> MovieHero loads poster
-> onPosterLoad()
-> waits 150 ms
-> emit("ready")
-> onHeroReady() -> heroReady = true
-> isLoading || !heroReady = false
-> "Loading movie" disappears
```

If poster_path does not exist, MovieHero considers that part ready so a missing image cannot leave the loader blocked indefinitely.

29.7 Example: "We couldn't find any movies" does not mean TMDB is down

MovieResults receives data.pages from useInfiniteQuery, merges results with flatMap, and discards movies without a usable poster_path. hasProcessedData is enabled when that processing finishes. The empty message appears only when there is no error, processing has completed, and validMovies is empty.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
TMDB responds correctly
-> data.pages
-> flatMap(results)
-> filter invalid poster_path
-> hasProcessedData = true
-> validMovies.length === 0
-> empty state: "We couldn't find any movies"
```

A real query failure follows the error/isError path. Visually, both paths can look negative, but technically they do not mean the same thing.

29.8 Example: empty Favorites is resolved without HTTP

Favorites uses Pinia persisted in localStorage. If favorites.length is 0, MoviesFavoritesView displays the empty state. Axios, Laravel, MySQL, and TMDB are not involved.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
localStorage
-> pinia-plugin-persistedstate
-> useFavoritesStore
-> favorites = []
-> MoviesFavoritesView
-> empty state
```

29.9 Example: valid profile with zero reviews

This case does pass through Laravel. GET /api/profile can respond successfully with reviews_count = 0 and latest_reviews = []. ProfileView interprets the empty array and displays the corresponding message. It is not a 404 or a 500.

```
ProfileView
-> profile.service.ts
-> GET /api/profile
-> web + auth middleware
-> Api\ProfileController::show()
-> Review::count() = 0
-> latestReviews = []
-> JSON success
-> Vue: latestReviews.length === 0
-> profile-with-no-reviews message
```

29.10 Guided example: Carlos enters an incorrect password

LoginView first runs form validation. If email and password have a valid shape, it sends POST /login. LoginRequest is what can determine whether the password matches the user's hash in MySQL.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Carlos clicks Sign In
-> LoginView.validateLoginForm()
-> loginUser()
-> POST /login
-> routes/auth.php
-> LoginRequest::rules()
-> LoginRequest::authenticate()
-> User::where(email)->first()
-> Hash::check() = false
-> RateLimiter::hit()
-> ValidationException errors.email
-> Laravel responds 422
-> Axios rejects Promise
-> LoginView catch detects errors.email
-> formError = "invalidCredentials"
-> vue-i18n
-> Carlos sees "Incorrect username or password."
```

The same form treats 429 as a temporary lockout, 403 as an unverified email, and status >= 500 as an internal error. In other words, the view does not depend on a single server string; it classifies the result by status and validation structure.

29.11 Example: invalid recovery link

In the current Laravel version, ResetPasswordView considers the URL invalid when token or email is missing. When both values exist, it displays the form without performing remote validation in onMounted. The actual verification occurs when POST /reset-password is submitted: Password::reset validates the token and Laravel returns a validation error if it cannot be used.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
CASE A
token or email missing
-> ResetPasswordView
-> tokenStatus = "invalid"
-> no HTTP
```

```

CASE B
token/email present, Carlos submits new password
-> POST /reset-password
-> NewPasswordController
-> Password::reset()
-> invalid/expired token
-> ValidationException
-> Axios catch
-> interface displays a controlled error

```

29.12 HTTP states and Laravel backend error structure

Laravel combines HTTP status with JSON or with the standard validation-error structure. ReviewController returns custom codes for business rules; ValidationException returns per-field errors; global database-outage handling returns a specific DATABASE_UNAVAILABLE contract.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

Real examples
401 / auth middleware      -> user not authenticated
403 NOT_REVIEW_OWNER      -> attempts to modify another user's review
409 REVIEW_ALREADY_EXISTS -> controlled duplicate
422 ValidationException   -> invalid data
429 LoginRequest          -> RateLimiter blocked
500                       -> unexpected internal failure
503 DATABASE_UNAVAILABLE  -> MySQL unavailable

```

29.13 Technical failures and external dependencies

TMDB can fail even when Laravel and MySQL work; SMTP can fail during registration or recovery; MySQL can be down before a Controller gets to execute its query. Each dependency enters through a different layer, and not every failure ends on the same screen.

bootstrap/app.php specifically catches QueryException whose message contains SQLSTATE[HY000] [2002]. For JSON requests it returns 503 with DATABASE_UNAVAILABLE; for web navigation it returns the errors.database Blade view. In both cases, the technical detail is logged separately.

29.14 Final state map

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```

SPA 404          -> Vue Router catch-all -> NotFoundView
Loading movie    -> Vue Query + MovieHero -> overlay
No movies        -> MovieResults -> empty state
No favorites     -> Pinia/localStorage -> local empty state
Profile with no reviews -> Laravel/MySQL returns valid empty data
Invalid credentials -> LoginRequest -> ValidationException -> LoginView
Database down    -> bootstrap/app.php -> 503 + technical log
SMTP failure     -> Controller -> CineVerseLog + ValidationException

```

30. Logs: Error Recording and Diagnostics

The project retains its own logging layer so important errors are recorded in a recognizable format. CineVerseLog manages logging and CineVerseLogFormatter controls the representation. The base path is configured through CINEVERSE_LOG_PATH, whose example points to storage/logs.

30.1 What context is logged

The global exception handler—the handler configured in bootstrap/app.php—tries to identify controller and method from the route action. It can also include status, user_id, email, username, or movie_id when available. This makes it possible to investigate an error without exposing those details in the response sent to the browser.

30.2 Email errors

RegisteredUserController catches Throwable around verification-email delivery. If SMTP fails, it records a mail-failure message with context and deletes the newly created user so it does not leave an account that the flow expects to verify but whose email was never sent. Other email Controllers apply the same idea of logging the failure and returning a controlled message.

30.3 Do not confuse a log with a user-facing message

The interface needs short, actionable text; the log needs technical context. A response can say that the service is unavailable while the file records QueryException, Controller, and status. Separating the two responsibilities avoids leaking internal details while preserving diagnostic information.

30.4 Laravel Logging, Monolog, and CineVerseLog are not the same component

CineVerseLog.php belongs to CineVerse code and provides a simple entry point for recording technical failures. It does not open files directly. Internally, it uses Laravel's Log facade and selects the cineverse_unexpected channel. Laravel configures that channel and uses Monolog underneath to write the file.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
CineVerse code / bootstrap
-> CineVerseLog::error(...)
-> Illuminate\Support\Facades\Log
-> channel("cineverse_unexpected")
-> config/logging.php
-> daily driver
-> Monolog handler
-> CineVerseLogFormatter
-> storage/logs/cineverse-backend-YYYY-MM-DD.log
```

This difference matters compared with the previous PHP backend: there is no longer a class that manually constructs Logger and RotatingFileHandler. Laravel creates the channel from config/logging.php and still uses Monolog internally.

30.5 The cineverse_unexpected channel in config/logging.php

Location in the project: config/logging.php · **Laravel Configuration** · cineverse_unexpected channel

Used by: Log facade / CineVerseLog. Responsibility: define the daily file, level, and formatter for the custom channel.

```
'cineverse_unexpected' => [
    'driver' => 'daily',
    'path' => $cineverseLogPath,
    'level' => 'error',
    'days' => 30,
    'tap' => [App\Logging\CineVerseLogFormatter::class],
    'replace_placeholders' => true,
],
```

driver=daily indicates daily rotation. level=error means this channel does not act as a general INFO action history. days=30 retains up to 30 rotated files. tap allows the Logger already created by Laravel to be modified and the custom formatter applied.

30.6 How the file path is calculated

config/logging.php reads CINEVERSE_LOG_PATH. If it contains an absolute Windows or Linux path, that path is used directly. If it is relative, Laravel combines it with base_path(). If the variable is empty, storage_path("logs") is used. The base filename cineverse-backend.log is then appended.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
CINEVERSE_LOG_PATH=storage/logs
-> base_path("storage/logs")
```

```
-> ../storage/logs/cineverse-backend.log
-> daily driver
-> cineverse-backend-2026-09-19.log
```

The final dated name is produced by the daily driver. The date does not need to be constructed manually inside CineVerseLog.

30.7 CineVerseLogFormatter: format of each entry

app/Logging/CineVerseLogFormatter.php receives Laravel's Logger and iterates over its handlers. It applies a Monolog LineFormatter with a short structure to each handler.

Location in the project: app/Logging/CineVerseLogFormatter.php · **Logging class** · App\Logging\CineVerseLogFormatter

Used by: cineverse_unexpected channel. Responsibility: give each log line a short, stable format.

```
new LineFormatter(
    "[%datetime%] %level_name%: %message%\n",
    "Y-m-d H:i:s",
    true,
    true
)
```

%datetime% becomes date and time, %level_name% becomes ERROR, and %message% becomes the text supplied by CineVerseLog. allowInlineLineBreaks=true lets the context JSON block preserve line breaks.

30.8 CineVerseLog::error(): main message + JSON context

CineVerseLog::error(message, context) calls Log::channel("cineverse_unexpected")->error(). Before doing so, formatContext removes keys whose value is null and transforms the rest into readable JSON with JSON_PRETTY_PRINT, JSON_UNESCAPED_UNICODE, and JSON_UNESCAPED_SLASHES.

Didactic invocation example; it is not a literal source-code block. The real method is in app/Support/CineVerseLog.php · App\Support\CineVerseLog class · error(string \$message, array \$context).

```
CineVerseLog::error($exception->getMessage(), [
    "controller" => "ReviewController",
    "method" => "store",
    "user_id" => 42,
    "movie_id" => 550,
]);
```

The array is not passed to Monolog as separate structured context: CineVerseLog converts it to text and concatenates it to the message. That is why the final file shows the cause first and a JSON block underneath.

30.9 What is physically written

```
[2026-09-19 12:30:15] ERROR: technical message
{
    "controller": "ReviewController",
    "method": "store",
    "user_id": 42,
    "movie_id": 550
}
```

The first part comes from LineFormatter. The JSON block comes from CineVerseLog::formatContext(). The browser does not automatically receive this text; the log is intended for internal diagnostics.

30.10 bootstrap/app.php: reporting and rendering are two responsibilities

Laravel separates reporting an exception from the response presented to the client. CineVerse uses withExceptions() to customize both flows.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
EXCEPTION
-> report() decides whether it is logged and with what context
```

```
-> render() decides whether a special response exists
-> final response to browser / Axios
```

The reporting function registered as a `report()` callback identifies two groups that enter the custom channel: database unavailable and unexpected internal errors. `ValidationException` and expected HTTP errors such as 403, 404, 419, or 422 are not written as ERROR through that path.

30.11 How `report()` obtains context

That callback—a function Laravel executes when it reports an exception—takes the current Request and its route. When the action has `Controller@method` form, it separates both names; in the console it uses `Console/artisan`; for a closure it identifies the route. It then adds status, `user_id`, email, username, and `movie_id` when available.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

Location in the project: `bootstrap/app.php` · `report()` callback configured in `withExceptions()`. Uses `App\Support\CineVerseLog` to record technical context before deciding the client response.

```
CineVerseLog::error($exception->getMessage(), [
    "controller" => $controller,
    "method" => $method,
    "status" => $isDatabaseUnavailable ? 503 : 500,
    "user_id" => optional($request->user())->id,
    "email" => $request->input("email"),
    "username" => $request->input("username"),
    "movie_id" => $request->route("movieId") ?? $request->input("movie_id"),
]);
```

The callback returns false after logging. In this configuration, that prevents Laravel from duplicating the same report in the default channel.

30.12 `render()`: special response when MySQL is unavailable

The specific render handler intercepts only `QueryException` whose message contains `SQLSTATE[HY000] [2002]`. If the request expects JSON or belongs to `api/*`, it returns a 503 response; if it is web navigation, it returns the `errors.database` view with the same status.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
JSON request /api/...
-> HTTP 503
-> {
    "success": false,
    "code": "DATABASE_UNAVAILABLE",
    "message": "The database is unavailable."
}

Web navigation
-> resources/views/errors/database.blade.php
-> HTTP 503
```

30.13 Complete example: MySQL is down

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Carlos opens a screen that needs MySQL
-> Eloquent attempts a query
-> QueryException SQLSTATE[HY000] [2002]
-> withExceptions report()
    -> CineVerseLog::error(...)
    -> daily file keeps the cause
-> withExceptions render()
    -> JSON 503 or Blade 503 depending on request
-> Carlos sees a controlled response, not the technical stack trace
```

This separates two audiences: the developer retains diagnostic data; the interface receives a safe, stable response.

30.14 Complete example: SMTP fails during registration

RegisteredUserController creates the User and calls `sendEmailVerificationNotification()` inside a try/catch. If delivery fails, `CineVerseLog::mailFailureMessage()` summarizes certain SMTP authentication or connection errors to avoid huge messages, logs controller/method/username/email, deletes the newly created user, and throws a `ValidationException`.

Step-by-step flow (explanatory diagram; it is not a literal block from a file):

```
Carlos clicks Create Account
-> User::create()
-> sendEmailVerificationNotification()
-> SMTP fails
-> catch(Throwable)
-> CineVerseLog::mailFailureMessage()
-> CineVerseLog::error(... context ...)
-> $user->delete()
-> ValidationException auth.email_delivery_failed
-> form displays a controlled error
```

PasswordResetLinkController applies the same idea when requesting a recovery email: it logs the technical failure and returns a validation error instead of exposing the full SMTP exception.

30.15 What is logged and what is not

A negative result does not automatically imply an ERROR log. Incorrect credentials, an out-of-range rating, a duplicate review, 404, or 422 are expected results that the system knows how to classify. The `cineverse_unexpected` channel is reserved for internal errors, database outages, and explicitly logged mail failures.

- Yes: unexpected internal exception that ends as 500.
- Yes: MySQL unavailable detected as 503.
- Yes: mail failures caught during registration or recovery.
- No through global reporting: `ValidationException`, 403, 404, 419, 422.

Context must not include passwords, CSRF tokens, or verification/reset tokens. Files in storage/logs are also not part of the public content served from public/.

31. Local Installation of CineVerse Laravel

This section starts with a developer cloning or downloading CineVerse Laravel from GitHub onto a new computer. The repository contains the code and definition files, but does not include vendor, node_modules, .env, or public/build because .gitignore excludes them. Installation rebuilds those generated dependencies and assets and prepares MySQL.

31.1 System requirements

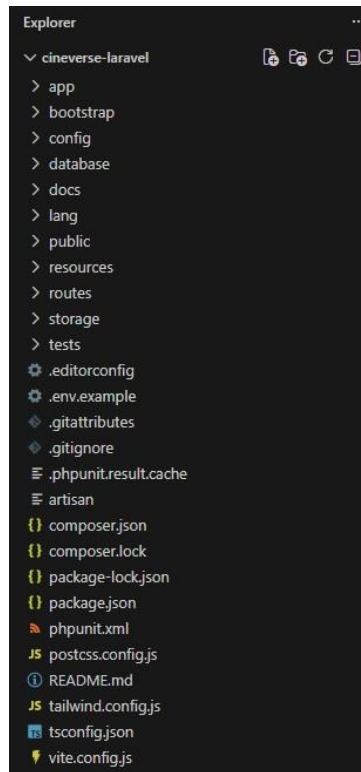
- PHP compatible with composer.json (^8.2).
- Composer available from the terminal.
- Node.js and npm available to install and run Vite.
- Accessible MySQL and a user with permissions on the local database.
- A TMDB API Read Access Token for VITE_TMDB_TOKEN.
- SMTP configuration if real verification and recovery emails are to be tested.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse-laravel
$ php -v
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies
```

Figure. PHP 8.2.12 check in the local environment.

31.2 Opening the newly obtained project

In a freshly cloned repository, `app`, `bootstrap`, `config`, `database`, `public`, `resources`, `routes`, `storage`, `tests`, `composer.json`, `package.json`, and `vite.config.js` should be visible, among others. `vendor`, `node_modules`, and `.env` appear only after the local environment is prepared.

*Figure. Local structure of the CineVerse Laravel project.*

31.2.1 What the repository contains and what is rebuilt in a clean installation

During development there may also be generated or local files, including `.env`, `public/build`, `public/hot`, `logs`, and runtime-generated data under `storage`. `.gitignore` excludes those paths from normal version control. Therefore, when working from a freshly cloned repository, `.env`, `vendor`, `node_modules`, and compiled assets are rebuilt on the machine where the project is installed.

31.3 Installing PHP dependencies

```
composer install
```

Composer reads `composer.lock` and installs the locked versions into `vendor`. It also runs Laravel's `post-autoload-dump` and `package:discover` scripts. Laravel or Inertia do not need to be installed manually package by package.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse-laravel
$ composer install
```

Figure. Running composer install from the cineverse-laravel root

31.4 Installing JavaScript dependencies

```
npm install
```

npm rebuilds `node_modules` and makes Vite, Vue, TypeScript, and the other dependencies available. Running `npm run dev` before `npm install` can produce a "vite is not recognized" error because the local executable does not exist yet.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse-laravel
$ npm install
```

Figure. Installing JavaScript dependencies from the project root

31.5 Creating .env from the template

```
cp .env.example .env
```

The .env file is local and must not be committed to the repository. The initial copy contains example values; APP_URL, MySQL, SMTP, and TMDB are then updated for the machine in use.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse-laravel
$ cp .env.example .env
```

Figure. Creating .env from .env.example

31.6 Generating APP_KEY

```
php artisan key:generate
```

The command writes APP_KEY into .env. This is not an optional documentation step: a Laravel installation needs a valid key for the framework's cryptographic operations.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse-laravel
$ php artisan key:generate

INFO Application key set successfully.
```

Figure. Laravel confirms that the application key was generated successfully

31.7 Creating the empty database

```
CREATE DATABASE cineverse_laravel
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

The database is created empty. Tables are not drawn manually in Workbench: migrations are the reproducible source of the schema.

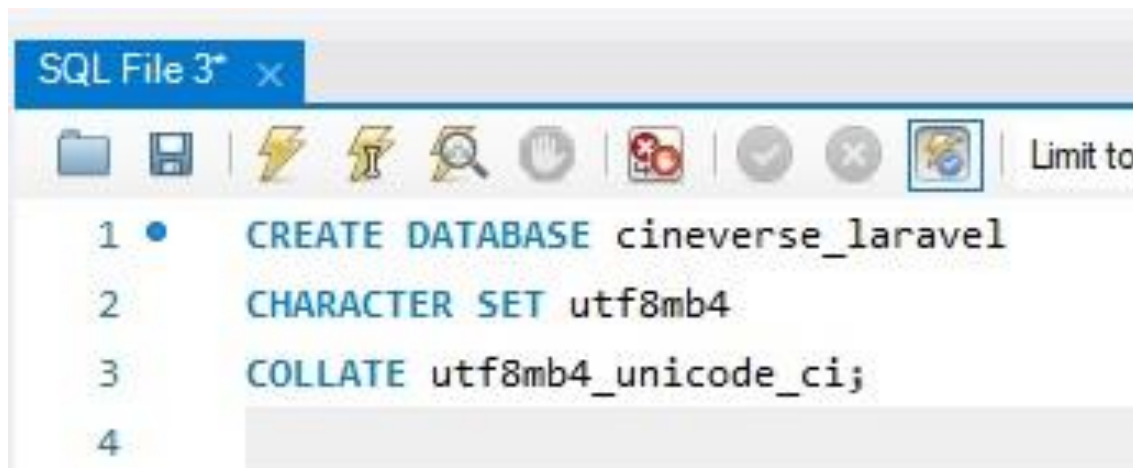


Figure. Manual creation of the cineverse_laravel database in MySQL Workbench before migrations

31.8 Configuring MySQL and TMDB in .env

Reference location: `.env.example` at the project root. This block shows the values Carlos must fill in within his local `.env` for MySQL and TMDB; username, password, and token are placeholders.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3307
DB_DATABASE=cineverse_laravel
DB_USERNAME=<username>
DB_PASSWORD=<password>

VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"
VITE_TMDB_TOKEN="<token>"
```

The values must match the actual machine. The documentation uses placeholders for username, password, and token; credentials are not published. If a `VITE_*` variable is changed while Vite is running, `npm run dev` must be restarted.

31.9 Running migrations

```
php artisan migrate
```

Laravel creates users, password_reset_tokens, sessions, cache, jobs, personal_access_tokens, reviews, and the migrations log. If this step fails because of the connection, check `DB_HOST`, `DB_PORT`, `DB_DATABASE`, username, password, and that MySQL is running.

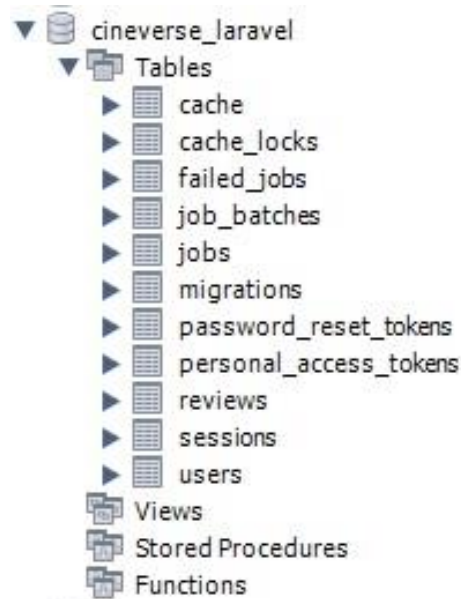


Figure. *cineverse_laravel* tables after running the migrations.

31.10 Loading test data

```
php artisan db:seed
```

`DatabaseSeeder` calls `UserSeeder` and `ReviewSeeder`. These seeders require empty tables: if they already contain data, they throw a `RuntimeException` and recommend `migrate:fresh --seed`. In a clean test environment, exactly 5,000 users and 24,322 reviews are loaded.

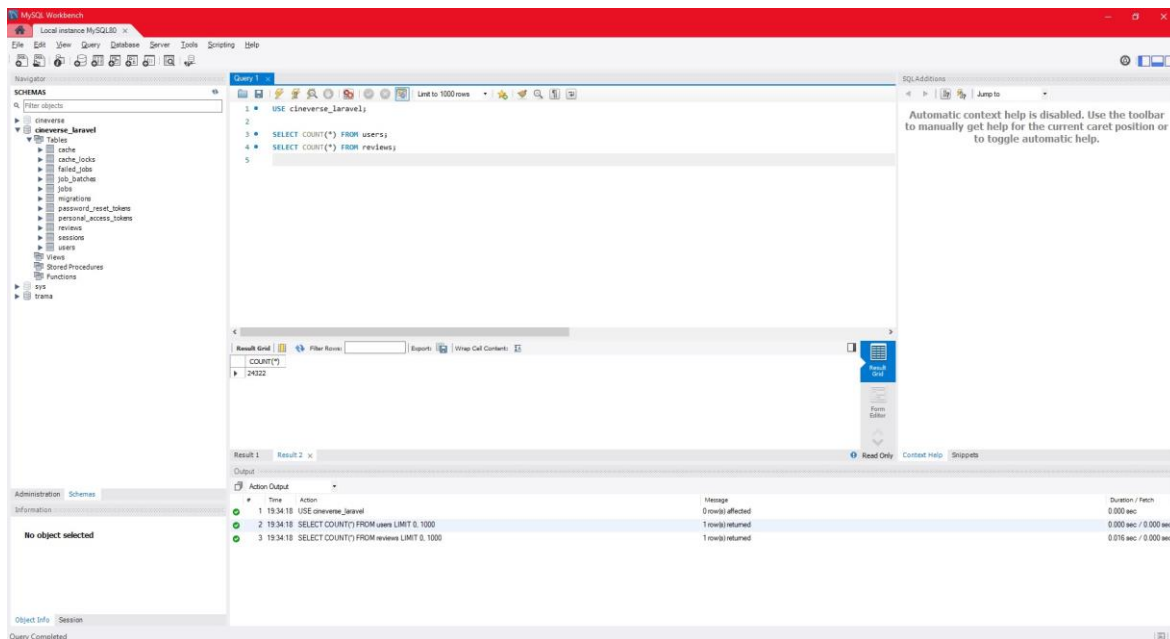


Figure. Checking the data loaded into the CineVerse Laravel database

31.11 composer setup script

composer.json includes a setup script that groups composer install, conditional creation of .env, key:generate, migrate --force, npm install, and npm run build. It is useful as a shortcut, but it does not run php artisan db:seed: the 5,000 users and 24,322 test reviews are loaded only when the seeders are run. The preceding flow is documented step by step because it shows the responsibility of each command and makes it clear where an installation fails.

```
composer run setup
```

32. Running CineVerse Laravel in Development

Once installation is complete, composer install, npm install, migrations, and seeders are not repeated every time the project is opened. Daily work mainly consists of keeping Laravel and Vite running. MySQL must also be available because the project's own data, configured cache, and other services use the database. Where sessions are stored depends on SESSION_DRIVER: .env.example proposes database; if file is configured, Laravel stores sessions in storage/framework/sessions.

32.1 Terminal 1: Laravel server

```
php artisan serve
```

Artisan starts Laravel's development server. In this project, when php artisan serve is run, the application is normally available at http://127.0.0.1:8000. That is CineVerse's main URL during development: Laravel receives browser requests, delivers the application through Inertia, and responds to authentication and API routes.

32.2 Terminal 2: Vite

```
npm run dev
```

Vite compiles and serves frontend assets with reload support during development. The port shown by Vite—for example 5173—does not replace Laravel's main URL in this integration. app.blade.php uses @vite, and the Laravel plugin knows how to connect both processes.

32.3 Both processes running simultaneously

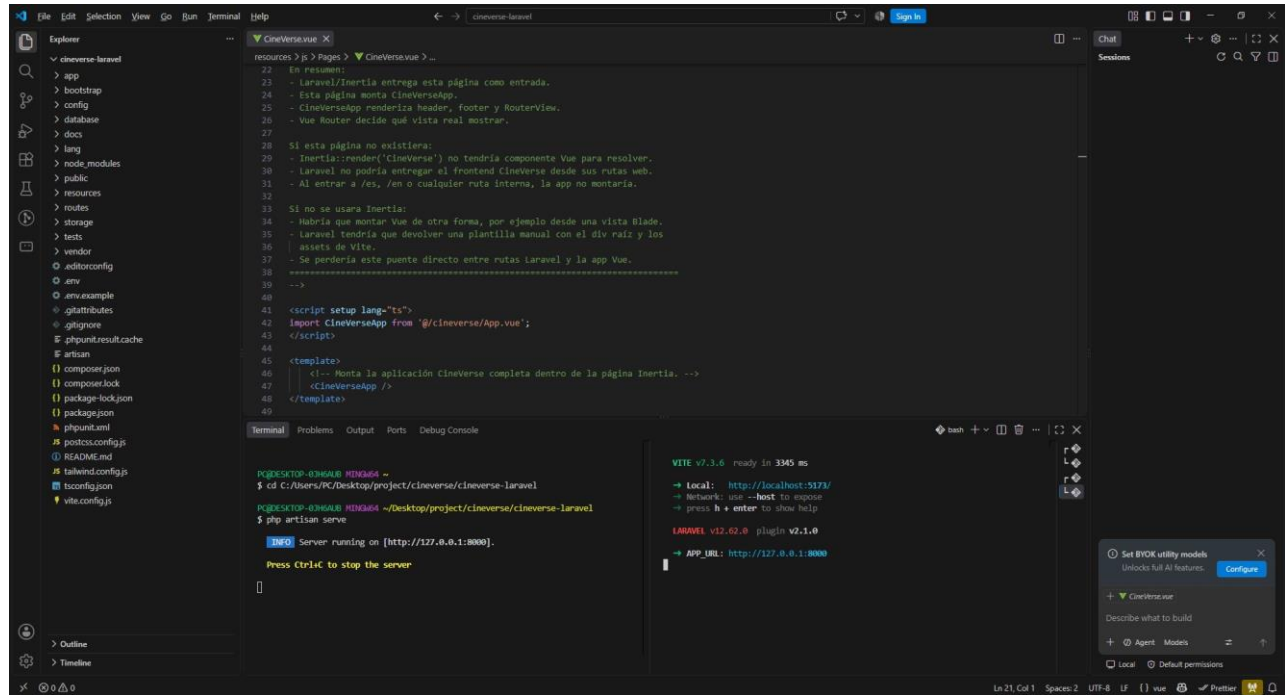


Figure. Laravel and Vite running during development.

The screenshot shows a useful characteristic of the architecture: while Laravel serves `http://127.0.0.1:8000`, Vite maintains the assets. `resources/js/Pages/CineVerse.vue` does not replace the frontend; it mounts `CineVerseApp` and `Vue Router` chooses the internal view.

32.4 Opening the application

With both processes active, open `http://127.0.0.1:8000`. `routes/web.php` redirects `/` to `/es` and the frontend loads Home. It is also valid to open a localized internal route directly because Laravel's catch-all will deliver the SPA.

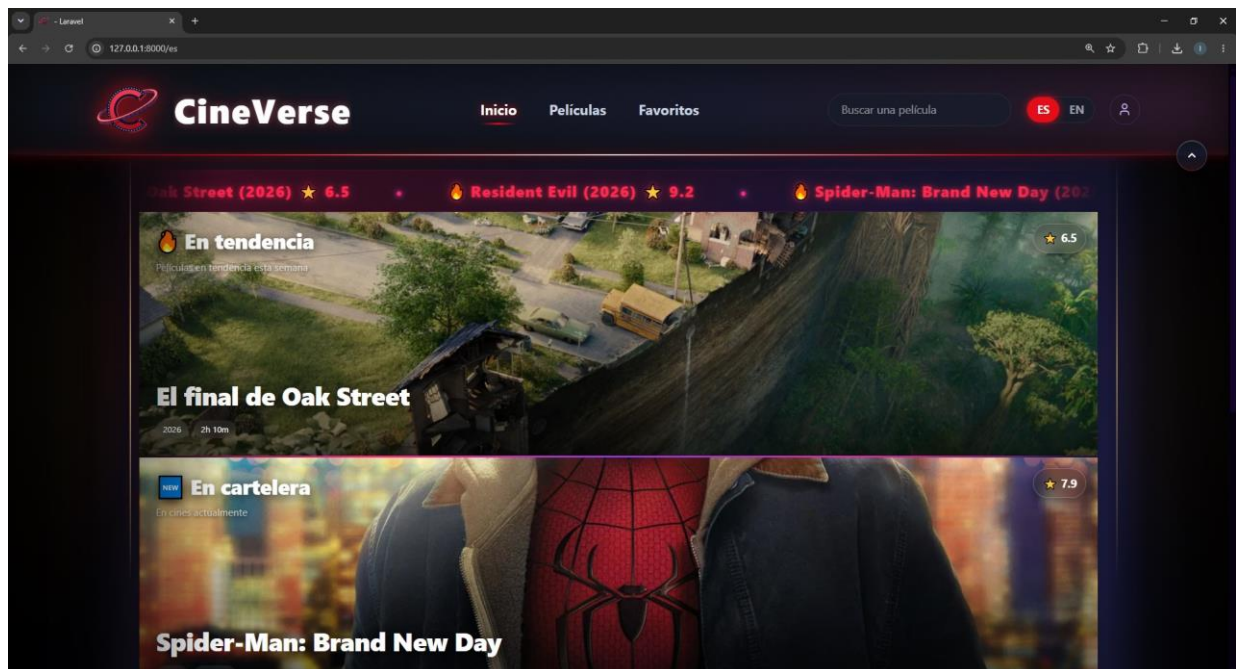


Figure. CineVerse running from the URL served by Laravel.

32.5 ViteManifestNotFoundException error

If Laravel does not detect the Vite server and `public/build/manifest.json` does not exist either, `ViteManifestNotFoundException` may appear. During development, the normal solution is to run `npm run dev` and keep the terminal open. In production, `npm run build` is executed to generate `public/build` and its manifest.

32.6 Alternative: composer run dev

`composer.json` also provides a dev script based on concurrently. It starts `php artisan serve`, `queue:listen`, `pail`, and `npm run dev` in a single run. For learning and diagnosing the project, it is clearer to begin with two separate terminals; afterward, the combined script can be convenient.

```
composer run dev
```

33. Deploying CineVerse Laravel to a Server

Deploying Laravel is not simply copying the development folder. The server must have compatible PHP, the required extensions, Composer or a prepared vendor directory, MySQL, write permissions on `storage/` and `bootstrap/cache/`, and a web root that points to `public/`. Vue assets must be compiled before the site is served without the Vite development server.

33.1 Production configuration

Recommended production-configuration example; it is not a literal block from a project file. It is based on the variables supported by `.env.example` and `config/*.php` and must be adapted to the actual hosting environment without publishing secrets. The `VITE_TMDB_*` variables must be defined before running `npm run build` because Vite embeds them in the compiled frontend.

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://domain.example

DB_CONNECTION=mysql
DB_HOST=<production-host>
DB_PORT=3306
DB_DATABASE=<production-database>
DB_USERNAME=<production-user>
DB_PASSWORD=<secret>

VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"
VITE_TMDB_TOKEN="<API Read Access Token>"

SESSION_SECURE_COOKIE=true
```

The exact values depend on the hosting environment. `APP_DEBUG` must remain `false` to avoid exposing stack traces. Credentials are not stored in Git. If the application is served over HTTPS, the session cookie must be configured consistently with the domain and HTTPS. SMTP must also be configured if email verification and password recovery will be used.

33.2 Dependencies and optimization

```
composer install --no-dev --optimize-autoloader
npm install
npm run build
```

`npm run build` runs `vue-tsc` first and then `vite build`. If TypeScript detects an error, the build stops before generating assets. Before this command runs, `VITE_TMDB_BASE_URL` and `VITE_TMDB_TOKEN` must exist with the values the published frontend will use. Vite produces `public/build`, which includes `manifest.json`. In production, `npm run dev` is not left running as a requirement for serving the application.

33.3 Migrations

```
php artisan migrate --force
```

`migrate --force` allows migrations to run in production without the interactive confirmation Laravel uses as a safeguard. `migrate:fresh` is not recommended on a server with real data because it drops tables. Test seeders should not be run automatically on a database that must preserve real information either.

33.4 Web root and rewrite

The web server's document root must point to `public/`, where `index.php` is located. Laravel receives routes that do not correspond to static files and processes them through the front controller. Exposing the entire repository root as the web root would increase the risk of serving files that should not be public.

33.5 This version does not use Inertia SSR

The project does not define an SSR entry point or an `inertia:start-ssr` process. The application mounts in the browser. Therefore, this guide does not include an SSR procedure. The required build here is the normal Vite build.

33.6 Post-deployment checks

- Open `/` and confirm the redirect to `/es`.
- Open a deep route such as `/es/movies/550` directly.
- Confirm that TMDB responds and images load.
- Register a test account and verify the email.
- Sign in and confirm that `/user` rebuilds the session.
- Create/edit/delete a test review.
- Check `storage/logs` for any 500 or email error.
- Check `/up` as a basic Laravel health check.

34. Tests and Technical Checks

The repository contains PHPUnit tests inherited largely from Breeze scaffolding and Laravel base tests. It is important to distinguish "test files exist" from "the suite is fully aligned with the current adaptation". The current suite contains several tests that still expect the original Breeze behavior and therefore must be updated before they can be treated as evidence that the entire current flow passes automatically.

34.1 What an automated test is

An automated test runs project code and checks that the observed result matches an expectation written in advance. It does not replace all manual testing, but it makes regressions detectable in a repeatable way.

34.2 PHPUnit inside Laravel

Laravel integrates PHPUnit through `php artisan test`. `composer.json` declares `phpunit/phpunit ^11.5.50` in `require-dev`. `phpunit.xml` prepares test-environment variables, and Laravel loads the application as it does for other Feature tests, but using the testing configuration.

```
composer test
-> php artisan config:clear
-> php artisan test
-> PHPUnit discovers tests
-> Laravel prepares Application
-> each test runs actions/asserts
```

34.3 Unit Tests and Feature Tests

A Unit Test attempts to verify a small, isolated piece. A Feature Test crosses more Laravel infrastructure: routes, middleware, Controllers, session, or database. The Breeze-inherited tests under `tests/Feature/Auth` are Feature tests because they simulate requests and verify authentication, redirects, or user state.

34.4 RefreshDatabase and assertions

RefreshDatabase prepares the database so a test does not depend on rows left by another test. Methods such as `assertStatus`, `assertRedirect`, `assertAuthenticated`, `assertGuest`, and database assertions express what the test expects.

Abbreviated Feature test example; it is not a complete literal block from the project's test files. The real tests are under `tests/Feature/Auth` and use Laravel testing helpers such as `post()`, `assertStatus()`, and `assertAuthenticated()`.

```
$response = $this->post("/login", [...]);
$response->assertStatus(204);
$this->assertAuthenticated();

// The expectation is useful only if it reflects the project's current behavior.
```

In CineVerse, this last point is essential: several tests were inherited from Breeze and still expect fields or redirects that the adaptation changed. The existence of a test does not make its expectation true for the current version.

34.5 Commands

```
composer test
# equivalent to:
php artisan config:clear
php artisan test
```

`phpunit.xml` prepares the test environment. Tests that use RefreshDatabase rebuild the test-environment database for each case using Laravel's tooling.

34.6 Existing tests

File/group	Purpose
AuthenticationTest	Login, invalid password, and logout.
EmailVerificationTest	Verification screen and link.
PasswordResetTest	Reset-link request and reset.
PasswordConfirmationTest	Password confirmation.
PasswordUpdateTest	Authenticated password change.
RegistrationTest	Registration.
ProfileTest	Profile, update, and account deletion.
Feature/ExampleTest	Basic application response.
Unit/ExampleTest	Minimal example unit test.

34.7 Visible misalignments in the current test suite

RegistrationTest still sends name and expects the user to become authenticated and be redirected to dashboard. The current RegisteredUserController requires username and deliberately does not sign the user in: it creates the account and sends verification. Therefore, that test describes the original Breeze behavior, not CineVerse's current behavior.

AuthenticationTest creates a factory User and expects login without explicitly reflecting the new verified-email requirement. EmailVerificationTest expects a redirect to `dashboard?verified=1`, while the current VerifyEmailController redirects to `{locale}/verify-email?verified=1`. ProfileTest still uses name in several operations, but the current ProfileUpdateRequest validates username. These differences show that the inherited suite does not fully represent the project's current behavior.

Conclusion about tests

The documentation does not claim that the current suite passes completely. The production code and inherited tests show a real misalignment. For that reason, the current PHPUnit status is not presented as definitive proof of the implemented flows.

34.8 Recommended manual checks

- Home loads trending, now-playing, and top-rated movies.
- Switching ES/EN keeps the route and updates localized text/data.
- Opening `/es/movies/{id}` directly works after F5.
- Registration rejects duplicate username/email and sends verification.
- An unverified account cannot sign in.
- A valid login rebuilds the store through GET `/user`.
- RateLimiter blocks after the configured attempts.
- Password recovery generates a link and allows a new password to be set.
- An authenticated user creates a review of up to 1000 characters.
- A second review cannot be created for the same movie.
- Another user's review cannot be edited or deleted through a manual request.
- Profile shows the review count and latest reviews.
- Favorites persist in the browser and respect `MAX_FAVORITES=20`.
- With MySQL stopped, the application returns the expected 503 contingency response.

34.9 Technical Conclusion

CineVerse Laravel combines a Vue interface with a Laravel backend without mixing the responsibilities of each layer. TMDB remains the source of the movie catalog, while Laravel and MySQL handle only the information that belongs to CineVerse, such as users, sessions, profiles, and reviews.

The resulting architecture keeps external data, application-specific logic, and presentation separate, making the project easier to maintain and allowing new features to be added without unnecessarily changing the other layers.